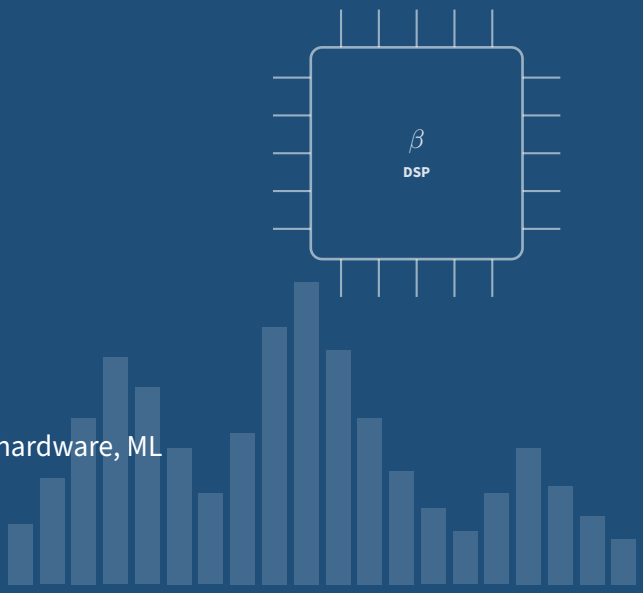
INDEPENDENT RESEARCH PROJECT GUIDE

From Intent to Silicon



Taking intention-based speech enhancement into DSP hardware, ML acceleration and VLSI. A step-by-step guide.

Prepared for	Aron Burrell
Builds on	*Intention-Based Speech Enhancement for Hearing Aids Using NLP*, BSc thesis, DTU Electrical Engineering, in collaboration with Oticon [1]
Target areas	VLSI-based digital signal processing · hardware for ML inference · FPGA prototyping · open-source ASIC design
Version	1.0 — October 2026

Every number about the thesis system in this guide was measured from the thesis code (see appendix A) or is marked as an estimate. Tool versions and board details change; check each tool's own documentation before you install.

How to Use This Guide

The project in sixty seconds

Your thesis showed that plain language can control a real-time speech enhancer. A sentence such as "the background noise is unbearable" becomes an intent class and a severity score, and those become DSP parameter updates that take effect without interrupting the audio. It ran on a laptop, in 64-bit floating point, with a 110-million-parameter language model.

This project asks the hardware version of the same question: *what is the smallest, lowest-power hardware that can still do intent-controlled speech enhancement, and what does each design decision cost?* You will answer it by:

1. turning the thesis code into a bit-exact fixed-point model;
2. designing the audio datapath in SystemVerilog;
3. verifying it against that model;
4. running it in real time on an FPGA, with your NLP pipeline controlling it;
5. going deeper on either ML acceleration or the ASIC flow.

Along the way you will practise the core skills of VLSI-based DSP and ML hardware: fixed-point design, microarchitecture, RTL, verification, FPGA prototyping, and measuring area, power and latency.

How the guide is organised

Part I — Foundations (read first)
Chapter 1 sets out the research questions, the plan and the deliverables. Chapter 2 is a primer on the hardware concepts you will need. Skim it now and come back to each section when a phase uses it. Chapter 3 measures your thesis system as a hardware workload. It is the chapter that makes everything else concrete, so do not skip it.

Part II — The project, step by step
One chapter per phase, Chapter 4 to Chapter 10. Every phase chapter has the same shape:

1. a goal;
2. the inputs you need and the outputs you will produce;
3. a sequence of numbered *steps*, each saying what to do, how to do it, how you know it worked, and roughly how long it takes;
4. the pitfalls;
5. exit criteria and a checklist.

Part III — Doing it well
Evaluation methodology (Chapter 11), writing up and presenting (Chapter 12), and the timeline, milestones and risk register (Chapter 13).

Appendices
The measured fact sheet (appendix A), the hardware register map (appendix B), a command cheat

sheet (appendix C), a glossary (appendix D), copy-paste templates for your lab notebook and reports (appendix E), and an annotated reading list (appendix F).

Suggested reading paths

- **"I want to start tomorrow."** Read Chapters 1, 3 and 13, then begin Chapter 4.
- **"Hardware is new to me."** Add Chapter 2, one section per evening, alongside Phase 0. Keep appendix D open.
- **"I have less time than the full plan assumes."** Chapter 13 has cut-down versions of the project for different credit loads. Decide your scope before you start Phase 1.

The boxes

The guide uses a small set of boxes consistently. This is what each one means:

Step N.M An action for you to take

Numbered per chapter. It says what to do, how to do it, how to check it worked, and how long it usually takes. Steps are meant to be done in order.

Done when

Exit criteria. When everything here is true, the phase is finished and its milestone is reached.

Why this matters for your project

Connects a step or concept to your thesis and to the research questions, so you always know why you are doing something.

Concept: Background theory

A compact explanation of a concept, usually with the one equation that matters.

Research question or hypothesis

A question the project answers, or a claim it tests.

Tip

Practical advice that saves time.

Pitfall

A mistake people really make at this point, and how to avoid it.

Read

Where to read more. Full references are in the bibliography and in appendix F.

Checklists look like this; tick them in your printed copy or in your notebook:

- ☐ Read Chapter 1.
- ☐ Reproduce the measurements in Chapter 3.

Conventions

- **Measured or estimated.** Any number about the thesis system is either *measured* from the thesis code or marked as an *estimate*. Never quote an estimate as a result in your own report. Measure it.
- **Paths.** `src/intent_se/...` refers to the thesis repository. Paths such as `golden/fixedpoint.py` or `rtl/bin_pipeline.sv` refer to the new project repository, `intent-se-hw`, which you create in Chapter 4.
- **Weeks and effort.** Week markers such as [Weeks 3–7] assume about 12–15 hours of work per week. Effort estimates assume you are new to the tool in question. Expect the first step with any tool to take longer than estimated and later steps to take less.
- **Tools change.** Commands and versions were checked when the guide was written, but tools move fast. When a command fails, read the tool's current documentation before assuming you made a mistake. Record the tool versions you actually used, as Chapter 11 explains.

This guide is a living document

The guide is a LaTeX project inside your thesis repository (`docs/research_guide/`). Build it with `make` in that directory (see the `README.md` there). Edit it as the project evolves: when you change the plan, change the guide, and note why in your lab notebook. A plan you keep up to date is worth far more than a perfect one you stop reading.

Contents

How to Use This Guide ii

I Foundations 1

1 Project Overview 2
1.1 Where you are starting from . . . 2
1.2 The core idea . . . 2
1.3 Research questions and hypotheses . . . 3
1.4 Scope and non-goals . . . 3
1.5 Deliverables . . . 4
1.6 Roadmap at a glance . . . 4
1.7 Two tracks after the prototype . . . 6
1.8 Skills you will gain . . . 6
1.9 Why this matters beyond the project . . . 6

2 A Hardware Primer for a DSP and ML Engineer 8
2.1 From algorithm to hardware: what changes . . . 8
2.2 Number representation . . . 8
2.3 Arithmetic building blocks and their cost . . . 9
2.4 The FFT in hardware . . . 10
2.5 Architecture transformations . . . 10
2.6 Memories . . . 11
2.7 Power and energy . . . 11
2.8 Implementation targets . . . 12
2.9 Design entry: RTL, HLS or a generator . . . 12
2.10 Interfaces . . . 12
2.11 Verification basics . . . 12
2.12 ML inference hardware basics . . . 13

3 Your Thesis as a Hardware Workload 14
3.1 Two planes, two rates . . . 14
3.2 The frame budget . . . 15
3.3 Operation inventory of the data plane . . . 15
3.4 Nonlinear functions: where the real hardware cost is . . . 17
3.5 Memory: the dominant cost . . . 18
3.6 Latency anatomy . . . 19
3.7 The control plane . . . 20
3.8 Decisions that flip under hardware constraints . . . 20
3.9 Summary of hardware opportunities . . . 21
3.10 Do it yourself . . . 21

II The Project, Step by Step 23

4 Phase 0 — Setting Up Your Research Environment 24
4.1 Steps . . . 24
4.2 Pitfalls . . . 28

5 Phase 1 — Algorithm–Hardware Co-Design and the Fixed-Point Golden Model 29
5.1 Steps . . . 30
5.2 Pitfalls . . . 34

6 Phase 2 — Microarchitecture and RTL Design 36
6.1 Top-level architecture . . . 36
6.2 The frame schedule . . . 37
6.3 Designing the blocks . . . 37
6.4 RTL coding guidelines . . . 39
6.5 Build order . . . 40
6.6 Pitfalls . . . 41

7 Phase 3 — Verification 43
7.1 The verification plan . . . 43
7.2 How the testbenches are built . . . 43
7.3 Steps . . . 44
7.4 When the RTL and the model disagree . . . 47

8 Phase 4 — Real-Time FPGA Prototype on the PYNQ-Z2 49
8.1 Hardware and host . . . 49
8.2 Steps . . . 50
8.3 Pitfalls . . . 52

9 Phase 5A — ML Acceleration Track 54
9.1 Characterise the control-plane workload . . . 54
9.2 Route A (required): the `head_engine` . . . 55
9.3 Then choose one research route . . . 56
9.4 Reporting . . . 57

10 Phase 5B — VLSI and ASIC Track 59
10.1 The flow . . . 59
10.2 What to harden . . . 60
10.3 Steps . . . 60
10.4 Pitfalls . . . 62

III Doing It Well 64

11 Evaluation Methodology and Research Rigour 65
11.1 What to measure . . . 65
11.2 Baselines . . . 65
11.3 Experimental design . . . 66
11.4 Statistics: is the difference real? . . . 66
11.5 Reproducibility . . . 66
11.6 Presenting results . . . 67
11.7 Threats to validity . . . 67

11.8 Ethics and data . . . 68

12 Writing It Up, Publishing and Presenting **69**
12.1 Write as you go . . . 69
12.2 Structure of the report . . . 69
12.3 Where it could go . . . 70
12.4 Release the repository . . . 70
12.5 CV lines and interview stories . . . 70
12.6 Talking to industry . . . 71

13 Timeline, Milestones and Risk Management **72**
13.1 The schedule . . . 72
13.2 Milestones and exit criteria . . . 72
13.3 Weekly rhythm . . . 72
13.4 Fitting the project to your credit load . . . 74
13.5 Decision points . . . 74
13.6 Risk register . . . 74
13.7 When you get stuck . . . 76

Appendices **77**

A Fact Sheet: Measured Properties of the Thesis System **78**

B Register Map of the Control Interface **82**

C Command Cheat Sheet **85**

D Glossary **87**

E Templates **90**

F Annotated Reading List **92**

Bibliography **94**

Part I

Foundations

Chapter 1

Project Overview

This chapter sets out what the project is and why it is worth doing. It defines the research questions you will answer and the plan for answering them. Read it first, and come back to it whenever you lose sight of why a step matters.

1.1 Where you are starting from

Your thesis [1] built a real-time speech enhancer for hearing aids that is controlled by plain language. It runs two pipelines concurrently:

- **An audio pipeline.** It enhances the microphone signal frame by frame: a sliding STFT with $\sqrt{\text{Hann}}$ windows and weighted overlap-add, IMCRA noise estimation [2], and a parametric Wiener filter $G = (\xi/(1+\xi))^{\beta}$ with a decision-directed a priori SNR [3].
- **An NLP pipeline.** It turns a typed complaint into an intent class and a severity score. It uses `all-mpnet-base-v2` sentence embeddings [4], [5], a linear SVM (test macro-F1 0.957) and a Ridge regressor (CV MAE 0.100).

A parameter controller maps the result to DSP updates that take effect without interrupting the audio.

The thesis answered its question: natural language *can* drive the DSP, and the path from sentence to sound is intact. It also listed limitations honestly. One of them is the starting point for this project:

> `all-mpnet-base-v2` *is 110M parameters and effectively wants a GPU — fine for a laptop proof-of-concept, far beyond embedded hearing-aid hardware.*

The same is true of the audio pipeline, for less obvious reasons. It runs in 64-bit floating point, keeps about 23,600 words of state between frames, performs 12 divisions and two transcendental functions per frequency bin, and delays every sample by 21.33 ms (Chapter 3). None of that matters on a laptop. All of it matters on a device that has to run all day on a hearing-aid battery, where the signal processor gets a power budget measured in milliwatts or less.

1.2 The core idea

Chapter 3 shows that your system already has the structure of a real hardware system. A *data plane* (the audio path) runs every 10.667 ms under a hard deadline. A *control plane* (the intent path) runs only when the user speaks. The interface between them is tiny: four numbers (β, gain floor, output gain, tilt).

In hardware that interface becomes even smaller. Because β and the floor change only when the user complains, the whole gain curve can be precomputed by the control plane into a small lookup table. The datapath then never evaluates `pow` at all. Tilt and output gain become two registers. The NLP

side controls the DSP by *writing a table and two registers*, which is exactly how fitting parameters are programmed into real hearing-aid chips.

That gives the hardware version of your thesis question:

The project question

What is the smallest, lowest-power hardware that can still do intent-controlled speech enhancement, and what does each design decision cost in quality, latency, memory, area and energy?

1.3 Research questions and hypotheses

The project question breaks down into four research questions. Each one has a measurable target, so you will know when you have answered it.

RQ1 — Precision

What minimum arithmetic precision (word lengths, number formats, function approximations) keeps the IMCRA + parametric-Wiener pipeline perceptually equivalent to the floating-point thesis reference?
Target: $|\Delta\text{PESQ}| \leq 0.1$ and $|\Delta\text{STOI}| \leq 0.01$ against float on the VoiceBank+DEMAND test set [6]. **Answered in** Chapter 5 (Phase 1).

RQ2 — Latency

How far can algorithmic latency be cut from 21.33 ms toward hearing-aid territory (10 ms or less, ideally about 8 ms), and what does each step cost in quality and memory?
Answered in Chapter 5 (Phase 1), with hardware cost confirmed in Chapters 6 and 8.

RQ3 — Cost

What are the area, power, energy per frame and memory costs of the resulting datapath on an FPGA and in an open-source ASIC flow? How do they trade against clock frequency and parallelism?
Answered in Chapters 6, 8 and 10 (Phases 2, 4 and 5B).

RQ4 — The intent path at the edge

How small and how low-precision can the intent path be while keeping macro-F1 close to the thesis's 0.957? The options are a quantised classifier head, a smaller encoder, or an intent model that works directly on audio.
Target: within 0.02 macro-F1 for the quantised head; for encoders, report the full trade-off curve.
Answered in Chapter 5 (first result) and Chapter 9 (Phase 5A).

The research questions come with four hypotheses that give the project its first concrete experiments (Table 1.1). A hypothesis is a prediction you could be wrong about. If one fails, you have still learned something you can report.

1.4 Scope and non-goals

A research project needs edges. These are deliberately outside it:

- **The perceptual calibration study.** The thesis left the mapping from severity to parameter step

Table 1.1: Hypotheses, and how each is tested.

	Hypothesis	Test	Where
H1	A datapath of 16 bits or fewer, with LUT-based nonlinear functions, meets the RQ1 target.	Word-length sweep of the bit-accurate model	Chapter 5
H2	Removing the 2× zero padding changes PESQ by less than 0.05, while halving FFT work and per-bin state.	Frame-configuration study	Chapter 5
H3	Because the six classes are linearly separable, an int8 classifier head loses less than 0.01 macro-F1.	Post-training quantisation of the heads	Chapters 5 and 9
H4	With resource sharing (one butterfly, one per-bin pipeline), the design meets the real-time deadline at a clock of 5 MHz or less.	Cycle count in simulation, then on the FPGA	Chapters 6 and 8

size uncalibrated, and it needs a listening study with hearing-aid users. This project does not attempt that. It does make that study cheaper later: once parameter updates are just table writes, a calibration experiment can sweep them on real hardware.

- **Speech recognition.** The control plane keeps typed text, or replaces it with a small audio-domain intent model (Chapter 9). It does not include a full ASR system.
- **Analog front end and acoustics.** There is no microphone preamplifier, ADC design, or feedback cancellation.
- **A product.** The aim is measured evidence and transferable skills, not a certified medical device.

1.5 Deliverables

1. **A public repository**, `intent-se-hw`, containing: a bit-accurate Python golden model; SystemVerilog RTL; cocotb testbenches; FPGA project scripts; and, on the ASIC track, LibreLane configuration and reports.
2. **A specification** (`spec-v1`): number formats, frame configuration and approximation choices, each backed by an experiment.
3. **A real-time FPGA demonstration** on a PYNQ-Z2. Typed complaints change the audio you hear, through hardware registers. Record a short video.
4. **A results set**: quality versus precision; latency versus quality and memory; FPGA resources and power; and either ML-head accelerator numbers or ASIC area, timing and power.
5. **A written report** in paper format (Chapter 12), suitable for a student conference submission or as the basis of an MSc thesis proposal.

1.6 Roadmap at a glance

The plan assumes about 12–15 hours per week for 26 weeks, including two buffer weeks (Figure 1.1 and Table 1.2). Chapter 13 turns it into a week-by-week schedule with a risk register, and shows how to shrink it if you have less time.

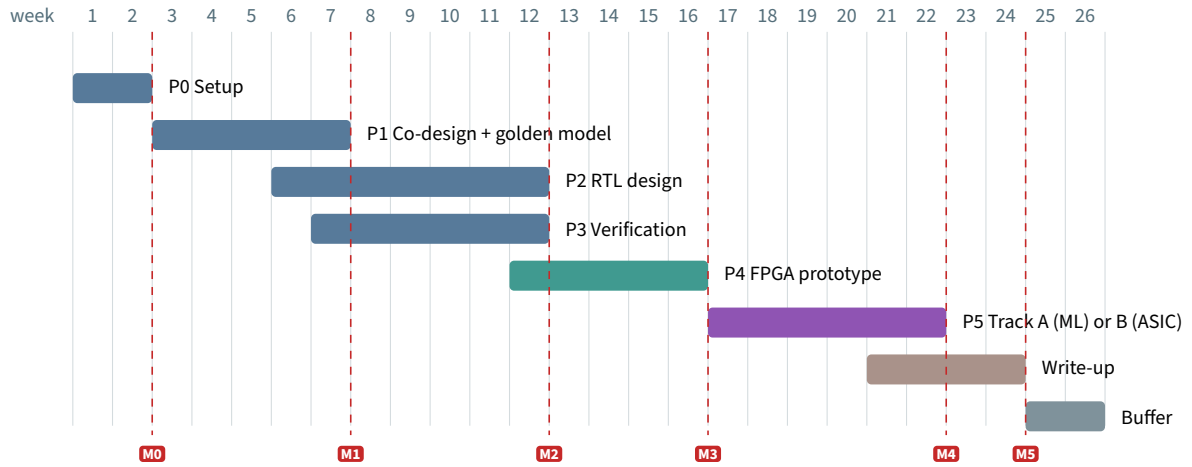


Figure 1.1: The project at a glance: phases (bars), weeks, and milestones M0–M5 (red, at the end of the marked week).

Table 1.2: Phases, chapters and milestones.

Phase	Chapter	Weeks	Milestone	Key deliverable
0 Setup	Chapter 4	1–2	M0	Toolchain, data, float baseline metrics
1 Co-design	Chapter 5	3–7	M1	Bit-accurate model, design-space results, golden vectors, `spec-v1`
2 RTL	Chapter 6	6–12	M2	RTL bit-exact against the golden model in simulation
3 Verification	Chapter 7	7–12		
4 FPGA	Chapter 8	12–16	M3	Real-time demo; latency, resource and power measurements
5A ML accel.	Chapter 9	17–22	M4	Int8 head engine plus encoder or audio-intent study
5B ASIC	Chapter 10	17–22	M4	Area, timing and power from an open PDK flow
Write-up	Chapter 12	21–24	M5	Report, release v1.0, demo video

1.7 Two tracks after the prototype

After the FPGA demo (M3) you choose one of two directions for six weeks. Choose around week 16, based on what you enjoyed and what you want to do next:

Track A — ML acceleration (Chapter 9)
Bring the intent path into hardware: an int8 matrix–vector engine for the classifier and severity heads, then either a compressed text encoder or a small model that recognises spoken intent directly from audio features computed by your own FFT. Choose this if you are aiming at AI-accelerator or edge-ML roles.

Track B — VLSI and ASIC (Chapter 10)
Take the per-bin datapath through an open-source RTL-to-GDS flow and report area, timing and power in a real process. Optionally submit a small block to Tiny Tapeout. Choose this if you are aiming at digital IC design or DSP-ASIC roles.

Both tracks reuse everything from Phases 0–4, and both answer a research question (RQ4 or RQ3) with numbers.

1.8 Skills you will gain

Table 1.3: Skills, where you practise them, and the evidence you can show.

Skill	Where in the project	Evidence for employers
Fixed-point and word-length design	Phase 1 golden model and sweeps	Quality-versus-bits plots; a justified spec
Algorithm–architecture co-design	Phase 1 (LUT gain, log-domain trackers, frame study)	"Decisions that flip" story; Pareto plots
Microarchitecture and RTL (SystemVerilog)	Phase 2	Lint-clean RTL; resource log; block diagrams
Verification (cocotb, Verilator, assertions)	Phase 3	Bit-exact regression in CI; coverage report
FPGA prototyping (Zynq, AXI, I2S)	Phase 4	Working demo video; timing and utilisation reports
ML hardware (quantisation, MAC arrays)	Track A	Int8 accuracy study; cycles and energy per inference
Physical design (synthesis, place and route, STA, power)	Track B	GDS layout; area, timing and power reports
Research method and writing	Throughout	Lab notebook, reproducible results, paper draft

1.9 Why this matters beyond the project

Hearing aids are one of the most demanding applications of low-power digital signal processing. The devices are tiny, run all day on a small battery, and have to process audio with very little delay. Hearing-aid manufacturers therefore design their own low-power DSP chips. Neural networks now run on the device itself: Oticon, for example, marketed Oticon More (2021) as the first hearing aid with an on-board

deep neural network. Denmark is home to several of the large hearing-aid companies, including Demant (Oticon), GN and WS Audiology.

Your thesis was done with Oticon. A project that goes from that thesis to measured, verified hardware gives you a concrete story for exactly the engineering teams that build these chips: fixed-point DSP, low-power architecture, verification, and ML at the edge.

Before moving on

- ☐ You can state the project question and RQ1–RQ4 in your own words.
- ☐ You know which chapter answers each research question.
- ☐ You have read Chapter 3 (the measured workload) and skimmed Chapter 13 (the schedule).
- ☐ You have discussed scope with your supervisor, especially Track A versus Track B and the credit load (Chapter 13).

Chapter 2

A Hardware Primer for a DSP and ML Engineer

You already know the signal processing and the machine learning. This chapter gives you the hardware concepts this project uses, each tied to a specific line of your thesis code, with pointers to where to read more. Read it once quickly, then come back to each section when a phase needs it.

2.1 From algorithm to hardware: what changes

In Python, you pay for time, and everything else is nearly free. In hardware, you pay for *area* (silicon, or FPGA resources), *energy* (battery life) and *design effort*, and you meet time as a hard deadline counted in clock cycles. Four habits change:

- **Numbers have a width you choose.** There is no free float64. Every bit costs area and energy (Section 2.2).
- **Operations are not equal.** An addition is cheap, a multiplication moderate, a division or exp expensive (Section 2.3).
- **Memory costs more than arithmetic.** Fetching an operand from memory can cost far more energy than computing with it (Section 2.6).
- **Everything happens in parallel.** Hardware is a circuit, not a sequence of statements. You decide how much of it to build and how to share it over time (Section 2.5).

2.2 Number representation

Concept: Fixed point and Q-format

A W-bit two's-complement integer q represents the real value $x = q \cdot 2^{-F}$, where F is the number of fractional bits. The notation **Q**m**.**F means m integer bits, including the sign, and F fractional bits, with $W = m + F$. Signed Q$m.F$ covers $[-2^{m-1},\ 2^{m-1} - 2^{-F}]$ with resolution 2^{-F}. The unsigned form UQ$m.F$ covers $[0,\ 2^{m} - 2^{-F}]$. Examples: audio in Q1.15 covers $[-1, 1)$; a gain in UQ3.13 covers $[0, 8)$ with resolution 1.2×10^{-4}.
When a result exceeds the range, **overflow** either *wraps* (two's complement arithmetic, which is catastrophic for audio) or *saturates* (clamps to the limit, which costs a comparator). When you discard low bits, **rounding** either truncates (biased; free) or rounds to nearest (one adder).

Concept: Quantisation noise and SQNR

Rounding to F fractional bits adds an error that is roughly uniform on $[-2^{-F-1}, 2^{-F-1}]$, with variance $\sigma^2 = 2^{-2F}/12$, provided the signal is "busy" enough [7]. For a full-scale sine quantised to N bits, this gives the familiar

$$\mathrm{SQNR} \approx 6.02\,N + 1.76 \text{ dB}. \tag{2.1}$$

Each extra bit buys about 6 dB. The catch is "full-scale": signals that use only part of the range waste bits. That is why the dynamic range profiling in Chapter 5 comes first.

Where this appears in your thesis code

The Wiener gain at `wiener.py:102` lies between the floor (0.05) and 1 before tilt and output gain, and up to 4 after. In UQ3.13 its quantisation step is 1.2×10^{-4}, about 0.001 dB near unity gain but about 0.02 dB at the 0.05 floor, because the step is fixed in absolute terms. A logarithmic format would keep the relative step constant. Questions like this are exactly what Phase 1 measures.

Concept: Block floating point, small floats and logarithmic numbers

- **Block floating point**: a group of values, such as an FFT frame, shares one exponent. This is cheap and very effective in FFTs.
- **Small custom floats**, for example a 10-bit mantissa and a 6-bit exponent: the relative precision is constant over a huge range, which is good for power spectra.
- **Logarithmic number systems**: store $\log_2 |x|$. Multiplication becomes addition and division becomes subtraction, while addition becomes harder. Comparisons and minima are exact, since $\log$ is monotone. Your IMCRA minimum trackers are a natural fit.

2.3 Arithmetic building blocks and their cost

Table 2.1: Relative cost of operations in hardware (qualitative; it depends on width and technology).

Operation	Typical implementation	Cost
Add, subtract, compare	ripple or carry-lookahead adder	low: linear in width
Shift by a constant	wiring	free
Multiply	array or Booth multiplier; a DSP slice on an FPGA	moderate: roughly quadratic in width
Divide	iterative (one bit per cycle), or reciprocal + multiply	high, or slow
exp, log, x^β, $\sqrt{\cdot}$	table + interpolation, CORDIC, or polynomial	high; depends on accuracy
Table lookup	ROM or RAM	cheap if small; cost grows with size

Concept: Reciprocal by Newton–Raphson

To compute $y = 1/d$, apply Newton's method to $f(y) = 1/y - d$:

$$y_{k+1} = y_k\,(2 - d\,y_k). \tag{2.2}$$

The relative error squares at every step: if $y_k = (1 - e)/d$ then $y_{k+1} = (1 - e^2)/d$. So a seed from a small table with 8 correct bits becomes 16 after one step, at the cost of two multiplications. Normalise d first with a leading-zero count, so the table only has to cover $[1, 2)$, then shift the result back [8].

Concept: Tables, interpolation and CORDIC

A smooth function on a bounded range can be stored as a table of 2^a samples. Linear interpolation between neighbouring entries costs one multiply and greatly improves accuracy for the same table size. $\log_2$ and 2^x are especially cheap: the integer part comes from the leading-zero count (a priority encoder) and only the fractional part needs a table. *CORDIC* computes trigonometric, hyperbolic, exponential and logarithmic functions with only shifts and additions, one bit of accuracy per iteration [9]. It is compact, but needs many cycles.

Where this appears in your thesis code

Per bin and per frame, the thesis computes 12 divisions, one `exp` (`imcra.py:153`) and one `pow` (`wiener.py:102`) (Chapter 3). Phase 1 replaces them with shared reciprocals, small tables and a gain lookup table rebuilt by the control plane.

2.4 The FFT in hardware

Concept: The radix-2 butterfly

The Cooley–Tukey FFT [10] splits an N-point DFT into two $N/2$-point DFTs, recursively, joined by *butterflies*: $A' = A + W_N^k B$ and $B' = A - W_N^k B$. An N-point transform has $\log_2 N$ stages of $N/2$ butterflies. Each butterfly is one complex multiplication (four real multiplications and two additions) and two complex additions. Hardware needs the *twiddle factors* W_N^k, which are stored in a ROM; by symmetry a quarter wave is enough. It also needs *bit-reversed* ordering at the input or output.

There are two families of FFT architecture:

- **Memory-based (in place).** One or a few butterfly units read and write a RAM over $\log_2 N$ passes. It is small and slow, which is ideal when you have many cycles per frame, as you do.
- **Pipelined**, such as single-path delay feedback (R2SDF, R2^{2}SDF). There is one butterfly per stage, connected by delay lines, and the unit accepts one sample per cycle continuously [11], [12]. It is larger, and the choice for streaming at high sample rates.

Values can grow by one bit per stage. Either provide $\log_2 N$ guard bits, scale by $1/2$ at every stage (which loses precision for small signals), or use block floating point, scaling only when growth is detected. A **real-input FFT** of size N can be computed with an $N/2$-point complex FFT and a cheap post-processing step, which halves the work for audio [13].

2.5 Architecture transformations

Concept: Throughput, latency and the initiation interval

Latency is how many cycles one input takes to produce its output. *Throughput* is how many inputs are accepted per cycle. A design with initiation interval II accepts a new input every II cycles. Pipelining (inserting registers) shortens the critical path, allowing a faster clock, and keeps II at 1, at the cost of more latency and registers.

Concept: Folding and unfolding

Folding (resource sharing) maps several operations of the algorithm onto one hardware unit over several cycles: less area, more cycles. *Unfolding* (parallelism) does the opposite. *Retiming* moves registers to balance paths without changing function. These are the main tools of VLSI DSP architecture [14].

A worked example from your thesis

The per-bin processing is about 112 operations for each of 1025 bins (Chapter 3).

- A *fully pipelined* unit (II = 1, all operations in hardware) processes the bins in about 1,025 cycles plus its pipeline depth.
- A *fully folded* unit, one arithmetic unit doing about one operation per cycle, needs about 115,000 cycles.

At 10 MHz the frame budget is 106,667 cycles, so the folded design just misses it, while the pipelined one uses about 1%. Between these extremes lies a whole family of designs. Finding the one with the least energy per frame is part of RQ3.

2.6 Memories

- **Flip-flops** hold one bit each, can be read in parallel, and are expensive per bit. Use them for pipelines and small state.
- **SRAM** is dense, with one or two ports, each reading or writing one word per cycle. On an FPGA it appears as block RAM, which on the Zynq-7020 comes in 36 Kbit blocks, or as distributed "LUT RAM".
- **ROM** holds constant tables. On FPGAs it is block RAM or logic; in ASICs, logic or a ROM macro.

Accessing memory typically costs considerably more energy than an arithmetic operation on the same data, and the gap grows with memory size [15]. Two rules follow:

- make memories small, which is why word lengths and bin counts matter;
- touch them rarely, which is why caching the tracker minima matters (Chapter 3).

2.7 Power and energy

Concept: Dynamic and static power

CMOS logic consumes dynamic power each time a node switches:

$$P_{\text{dyn}} = \alpha\, C\, V_{DD}^2\, f, \tag{2.3}$$

with activity factor α, switched capacitance C, supply voltage V_{DD} and clock frequency f. *Leakage* (static) power flows whenever the circuit is powered, and is ever more important in small processes.

Since the work per frame is fixed, the right metric is **energy per frame**, not power. Running faster does not save dynamic energy, since the same nodes switch either way. But a design that can run *slower* can run at a *lower voltage*, and energy scales with V^2. That is why low-power DSP uses parallel or pipelined hardware at a low clock and a low voltage [16]. Clock gating stops switching in idle blocks, and power gating removes leakage from them. Your datapath is idle for most of each frame, so both matter.

2.8 Implementation targets

Table 2.2: Where a signal-processing algorithm can run.

Target	Strength	Weakness	In this project
DSP processor or MCU	Flexible software; fast to develop	Energy per operation well above dedicated hardware	The comparison point; the control plane on the ARM
MCU + small NPU	Neural networks at low power	Fixed set of supported operators	Context for Track A
FPGA	Reconfigurable; real-time prototyping	Larger power and area than an ASIC	Phase 4 prototype
ASIC	Lowest energy and area	Expensive, slow, unforgiving	Track B, with an open PDK

2.9 Design entry: RTL, HLS or a generator

- **RTL in SystemVerilog** [17]: you describe registers and logic explicitly. This is the industry standard and what VLSI and ASIC roles expect you to read and write.
- **High-level synthesis (HLS)**: C++ compiled to hardware. It is fast for dataflow and ML kernels (hls4ml [18] builds on it), but harder to control at the bit and cycle level.
- **Hardware generators** such as Chisel, Amaranth or SpinalHDL: hardware described in Scala or Python. They are powerful, but less common in DSP-ASIC teams.

Recommendation: hand-written SystemVerilog for the DSP datapath, so you learn the craft that roles ask for. HLS is optional for Track A.

2.10 Interfaces

- **Valid/ready handshake.** A transfer happens on a cycle when both `valid` (the producer has data) and `ready` (the consumer can take it) are high. It is the universal streaming interface inside designs.
- **AXI4-Stream**: valid/ready plus data and optional side-band signals, used with DMA engines on Zynq.
- **AXI4-Lite**: a simple memory-mapped register interface, with separate address and data channels for writes and reads. It is how the ARM writes your registers (appendix B).
- **I2S**: a serial audio interface with a bit clock, a word-select (left/right) clock and a data line, used between the codec and the FPGA.

2.11 Verification basics

A *testbench* drives a design and checks its outputs against a *reference model*. When the model is bit-accurate, the check is exact equality. *Code coverage* measures which lines and toggles were exercised. *Functional coverage* measures which situations you intended to test actually occurred. *Assertions* state invariants that simulation checks continuously. *Formal verification* proves them for all possible inputs, which is practical for small blocks. Chapter 7 puts all of this into practice with cocotb [19], Verilator [20] and SymbiYosys.

2.12 ML inference hardware basics

- **MAC arrays.** Neural-network inference is dominated by multiply–accumulate operations. Accelerators are arrays of MAC units fed from local memories.
- **Systolic arrays.** A grid of MAC cells passes data to its neighbours every cycle, so each value fetched from memory is reused many times [21]. A TPU is built around one [22].
- **Dataflows.** *Weight-stationary* keeps weights in the cells; *output-stationary* keeps partial sums; *row-stationary* balances both [23]. The choice decides which memory traffic you save.
- **Quantisation.** Int8 weights and activations with int32 accumulation are standard, with a small loss in accuracy for most models [24].
- **Roofline.** Achievable performance is the minimum of peak compute and memory bandwidth times arithmetic intensity, the operations per byte moved [25]. Batch-1 matrix–vector products have low intensity, so they are memory-bound.

Read — the essential four

Harris and Harris, for digital design [26]. Parhi, for VLSI DSP architecture [14]. Sze et al., for ML hardware [27]. Widrow and Kollár, for quantisation [7]. Appendix F has the full annotated list.

You have what you need from this chapter when you can

- ☐ read Q3.13 and compute its range and resolution in your head;
- ☐ explain why one Newton–Raphson step doubles the correct bits;
- ☐ sketch a radix-2 butterfly and say how many there are in a 1024-point FFT;
- ☐ explain folding versus pipelining with the bin-processing example;
- ☐ explain why energy per frame, not power, is the right metric here.

Chapter 3

Your Thesis as a Hardware Workload

Before you design any hardware, you have to know exactly what you are asking it to do: which operations, how many per second, how much memory must persist between frames, and under what deadline. Hardware designers call this *characterising the workload*. It is the bridge between your thesis and everything that follows. Without it, design decisions are guesses. With it, most decisions in Phases 1 and 2 follow almost mechanically.

This chapter characterises the thesis system [1]. Every number marked *measured* comes from `docs/research_guide/scripts/measure_workload.py`, which instruments the thesis code directly. You will rerun it yourself in step 3.1. Numbers marked *estimate* are derived by hand from the code and are only as good as the counting. Appendix A collects all of them on one page.

Why this matters for your project

Most student hardware projects fail quietly here. They start writing RTL before they know where the cost is, then optimise the wrong block. Your thesis has an unusual advantage: its code is short, readable and already organised into the same blocks a chip would have. This chapter turns that code into a cost model.

3.1 Two planes, two rates

Your thesis already has the split that hardware designers use to organise real systems (Figure 3.1). The *data plane* is the audio path. It processes every sample, never stops, and has a hard deadline: if a frame is late, the user hears a glitch. The *control plane* is the intent path. It runs only when the user says something, and its only output is a handful of parameters.

Table 3.1: The two planes have completely different hardware requirements.

	Data plane (audio)	**Control plane (intent)**
Rate	93.75 Hz frames, 48 kHz samples	once per utterance (seconds to minutes apart)
Deadline	hard: 10.667 ms per frame, every frame	soft: a few hundred milliseconds feels instant
Dominant cost	memory for per-bin state; divisions and transcendental functions	the sentence encoder (10^8–10^9 operations per utterance)
Right metric	energy per frame, area, latency	energy per inference, model size
Thesis code	`src/intent_se/audio/`	`src/intent_se/nlp/`, `src/intent_se/control/`
Hardware style	fixed-function streaming datapath	small accelerator, or software on a CPU or phone

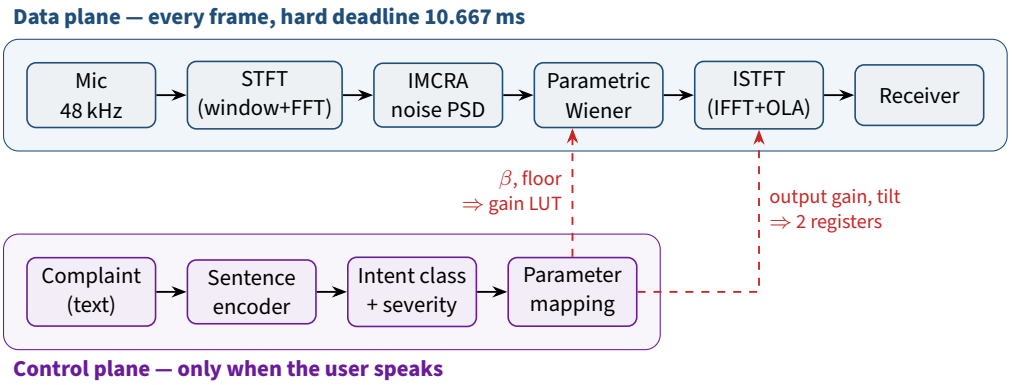


Figure 3.1: The thesis system redrawn as a data plane and a control plane. The dashed red arrows are the *entire* interface between them. In hardware it shrinks to a small lookup table and two registers (opportunities O1 and O2 in Section 3.9).

3.2 The frame budget

The hop length H fixes how often a frame must be produced. With $f_s = 48\,\text{kHz}$ and $H = 512$ (`src/intent_se/config.py`):

$$f_{\text{frame}} = \frac{f_s}{H} = 93.75\,\text{Hz}, \qquad T_{\text{frame}} = \frac{1}{f_{\text{frame}}} = 10.667\,\text{ms}. \tag{3.1}$$

A chip running at clock frequency f_{clk} therefore has $N_{\text{cyc}} = f_{\text{clk}}\, T_{\text{frame}}$ clock cycles to finish each frame (Table 3.2). That cycle count is the most important number in Phase 2. It tells you how much hardware you can *share*: if a frame needs 100,000 operations and you have 1,000,000 cycles, a single arithmetic unit can do everything, ten times over.

Table 3.2: Clock cycles available per frame (measured frame period).

f_{clk}	cycles per frame
100 MHz	1,066,667
50 MHz	533,333
10 MHz	106,667
1 MHz	10,667

Concept: Throughput is easy; energy is the problem

The thesis code already runs in about 0.2 ms per frame in Python on an ordinary computer (measured: real-time factor ≈ 0.02). Speed is not the challenge. The challenge is doing the same work in a hearing-aid power budget. Dynamic power scales as $\alpha C V^2 f$ [16], so the winning strategy is usually to run *slowly* and at low voltage, and to touch memory as little as possible. Keep that in mind whenever this guide asks "how few cycles can we use?": the real question is "how little energy per frame?"

3.3 Operation inventory of the data plane

Table 3.3 walks through one frame of the data plane, stage by stage, in the order the code runs it. The per-bin counts come from reading the code as written, with no algebraic simplification yet. Treat them as estimates accurate to within a few operations. Two things stand out. First, *every* stage after the FFT is per-bin and independent across bins, apart from the 5-tap frequency smoothing. That makes it ideal

for a streaming pipeline that processes one bin per clock cycle. Second, the expensive operations are divisions (12 per bin) and two transcendental functions, not multiplications.

Table 3.3: Per-bin operations per frame, counted from the code (estimate). Columns: multiplications, additions/-subtractions, divisions, comparisons (including `min`, `max`, `clip`), and transcendental functions. Line numbers refer to the thesis repository.

Stage	Source	×	±	÷	cmp	exp/pow
Power spectrum $\lvert X_k\rvert^2$	`pipeline.py:179`	2	1	–	–	–
Frequency smoothing (5 taps)	`imcra.py:117`	5	4	–	–	–
Recursive smoothing S	`imcra.py:118`	2	1	–	–	–
Minimum tracker 1	`imcra.py:40-51`	–	–	–	9	–
$\gamma_{\min}$, ζ (shared denominator)	`imcra.py:121-123`	1	–	2	1	–
Noise-only indicator	`imcra.py:126`	–	–	–	2	–
Second smoothing $\tilde{S}_f$	`imcra.py:129-131`	6	8	1	1	–
Recursive smoothing $\tilde{S}$	`imcra.py:133`	2	1	–	–	–
Minimum tracker 2	`imcra.py:134`	–	–	–	9	–
$\tilde{\gamma}_{\min}$, $\tilde{\zeta}$ (shared denominator)	`imcra.py:136-138`	1	–	2	1	–
Speech-absence probability q	`imcra.py:141-146`	1	1	–	6	–
A posteriori SNR γ, ν	`imcra.py:149-152`	1	1	2	1	–
Likelihood ratio, presence prob. p	`imcra.py:153-154`	2	3	2	2	exp
Noise PSD update λ_d	`imcra.py:159-164`	4	3	–	1	–
Wiener: γ, decision-directed ξ	`wiener.py:89-98`	2	2	2	3	–
Wiener: gain, floor, tilt, clip	`wiener.py:102-112`	2	1	1	3	pow ($+10^x$)
DD memory, apply gain	`wiener.py:115`, `pipeline.py:196`	4	–	–	–	–
Total per bin		**35**	**26**	**12**	**39**	**2 (+1)**

Adding the transforms gives a per-frame total (Table 3.4). The real 2048-point FFT can be computed as a 1024-point complex FFT plus a post-processing pass. A radix-2 complex FFT of size N needs $\frac{N}{2}\log_2 N = 5{,}120$ butterflies for $N = 1024$, and each butterfly is roughly 10 real operations [10], [13].

Table 3.4: Operations per frame and per second (estimate).

Block	ops per frame	share
Forward real FFT (2048 points)	$\approx 56{,}000$	25 %
Inverse real FFT (2048 points)	$\approx 56{,}000$	25 %
Analysis window, synthesis window, overlap-add	$\approx 3{,}000$	1 %
Per-bin processing ($1025\times \approx 112$)	$\approx 115{,}000$	50 %
Total	$\mathbf{\approx 230{,}000}$	
Per second ($\times$ 93.75 Hz)	$\mathbf{\approx 21 \times 10^6}$	

What the inventory tells you

At 100 MHz you have 1,066,667 cycles for roughly 230,000 operations, so throughput is not the constraint. Two architectures bracket the design space you will explore in Chapter 6 (both estimates):

- **Fast and lean.** One radix-2 butterfly per cycle plus a fully pipelined per-bin unit (one bin per cycle, initiation interval 1) needs about $2 \times (5{,}120 + 512) + 1025 + \text{overhead} \approx 12{,}500$ cycles per frame. That meets the deadline at only ≈ 1.2 MHz.
- **Fully folded.** Keep the single-butterfly FFT, but process the bins with one shared arithmetic

unit doing about one operation per cycle (about 100 cycles per bin). That needs $\approx 115{,}000$ cycles per frame, so ≈ 11 MHz.

Hypothesis H4 (Chapter 1) says that the shared-resource design meets the deadline at 5 MHz or less. The interesting research question is which point between these extremes gives the lowest *energy per frame*.

3.4 Nonlinear functions: where the real hardware cost is

In Python, a division or a `**` costs about the same as an addition. In hardware they are in different leagues. A multiplier maps to one FPGA DSP slice. A divider, an `exp` or a `pow` is either a long iterative circuit or a lookup table plus arithmetic (Chapter 2). Table 3.5 lists every nonlinear operation in the data plane and what you will replace it with.

Table 3.5: Nonlinear operations per bin per frame and their hardware replacements.

Operation	Count	Where	Hardware replacement
Division	12	9 in IMCRA, 3 in Wiener	Only 9 distinct denominators, so share reciprocals (O4). Caching $H = \xi/(1+\xi)$ (O3) removes one more. Log domain turns most of them into subtractions (O5).
$(\xi/(1+\xi))^{\beta}$	1	`wiener.py:102`	Gain lookup table indexed by ξ in dB, rebuilt by the control plane when β changes (O1). The datapath never computes `pow`.
$e^{-\nu}$	1	`imcra.py:153`	Small LUT or piecewise-linear approximation over $\nu \in [0, 16]$ (O7), or remove it entirely with the log-domain rewrite below.
$10^{\text{tilt}\cdot f/20}$	1	`wiener.py:110`	A geometric sequence across bins: one multiply per bin by a ratio r the control plane writes (O2).
min over 9 values	2	minimum trackers	Comparators. Values can be stored in log domain with few bits (O5).

Concept: An example of an algebraic rewrite for hardware

The speech-presence probability (`imcra.py:152-154`) is

$$p = \frac{1}{1 + \dfrac{q}{1-q}\,(1+\xi)\,e^{-\nu}}, \qquad \nu = \gamma\,\frac{\xi}{1+\xi}. \tag{3.2}$$

As written, that is three divisions and an exponential. Take logarithms of the ratio in the denominator and it becomes exactly

$$p = \sigma\Big(\nu - \ln(1+\xi) - \operatorname{logit}(q)\Big), \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \quad \operatorname{logit}(q) = \ln\frac{q}{1-q}. \tag{3.3}$$

In hardware, $\ln(1+\xi)$ is a small table indexed by ξ in dB. Conveniently, that is the same index as the gain LUT. $\operatorname{logit}(q)$ is a small table over q, and σ is one more small table. Three divisions and an `exp` become three table lookups and two additions. The rewrite is exact in real arithmetic. Whether it survives quantisation is precisely the kind of question Phase 1 answers (Chapter 5). One edge case: $q = 0$ means $\operatorname{logit}(q) = -\infty$ and $p = 1$, which has to be handled explicitly.

3.5 Memory: the dominant cost

The recursive estimators carry state from one frame to the next. Introspecting a running `SpeechEnhancer` gives the inventory in Table 3.6 (measured).

Table 3.6: Arrays that persist between frames (measured by introspection).

Array	Shape	words/bin	Role
`_tracker._store`	8×1025	8	subwindow minima, tracker 1
`_tracker._running`	1025	1	running minimum, tracker 1
`_tracker_tilde._store`	8×1025	8	subwindow minima, tracker 2
`_tracker_tilde._running`	1025	1	running minimum, tracker 2
`_s`, `_s_tilde`	1025 each	2	smoothed periodograms S, $\tilde{S}$
`_lambda_d`	1025	1	noise PSD estimate
`_prev_clean_power`, `_xi`	1025 each	2	Wiener decision-directed memory
`_norm_freq`	1025	(1)	constant ramp for tilt — ROM or computed, not state
Mutable state		**23**	$23 \times 1025 = 23{,}575$ words
`window`, `_in_buffer`, `_out_buffer`	3×1024		window ROM, input and overlap-add buffers

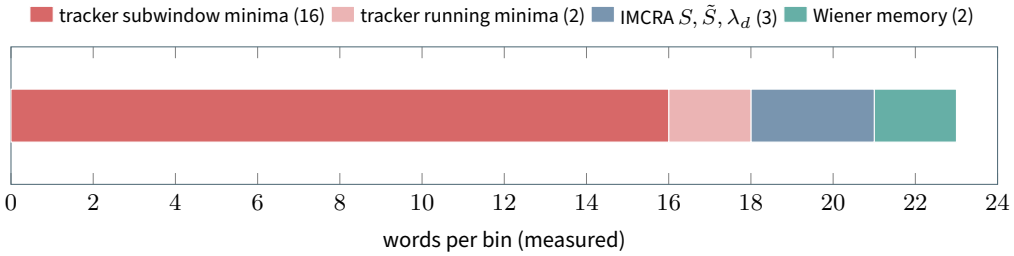


Figure 3.2: Where the 23 words of per-bin state go: 18 of them are inside the two minimum trackers.

At 23,575 words this is 46.0 KiB at 16 bits, 69.1 KiB at 24 bits and 92.1 KiB at 32 bits (measured). That is a lot of on-chip memory for a hearing aid. Reading and writing SRAM also typically costs more energy than the arithmetic done on the data, so memory is likely to dominate both area and energy. Three observations shape Phase 1:

1. **The trackers are the target.** 18 of the 23 words per bin hold minima. A minimum is unchanged by any monotone transform, so the trackers can store coarse logarithmic values (O5). As an illustration (not yet a result): storing the 18 tracker words at 8 bits and the other 5 at 16 bits gives $18 \times 8 + 5 \times 16 = 224$ bits per bin, against $23 \times 24 = 552$. That is a 2.5× reduction before you change anything else.
2. **The number of bins multiplies everything.** Removing the 2× zero padding (O6) halves the bins from 1025 to 513. Moving to 16 kHz with a 128-sample window leaves 65 bins (Section 3.6). With the 224-bit layout above, 65 bins need about 1.8 KiB in total.
3. **Memory *accesses* matter as much as memory size.** As written, each tracker reads all 9 stored values for every bin in every frame (`imcra.py:51`), even though the 8 subwindow minima only change once every $V = 15$ frames. If you cache their minimum in one extra word, the per-frame reads drop to 2 per tracker. The full 8-word scan is then needed only when a slot is overwritten. That is about $2 + 8/15 \approx 2.5$ reads per frame instead of 9, paid for with 1 extra word per bin per tracker. This kind of trade (slightly more storage for far fewer accesses) appears constantly in low-power design.

Pre-roll needs no buffer

The thesis stores 5 whole pre-roll frames and then takes their per-bin minimum (`imcra.py`, `seed`). Since only the minimum is used, hardware can keep a running minimum in the λ_d state memory that is about to be seeded anyway. That costs 0 extra words instead of 5×1025.

3.6 Latency anatomy

Figure 3.3 shows why every sample is delayed by exactly one window length. A block of 512 samples must be fully captured before its frame can be analysed. The overlap-add output for that block is only complete once the *next* frame has been added in. The total is 1024 samples, or 21.33 ms at 48 kHz (measured), before any converter or operating system buffering. Hearing-aid research generally treats total delays above roughly 10 ms as problematic, and open fittings are less tolerant still, because the direct and processed sound interfere and cause comb filtering [28], [29].

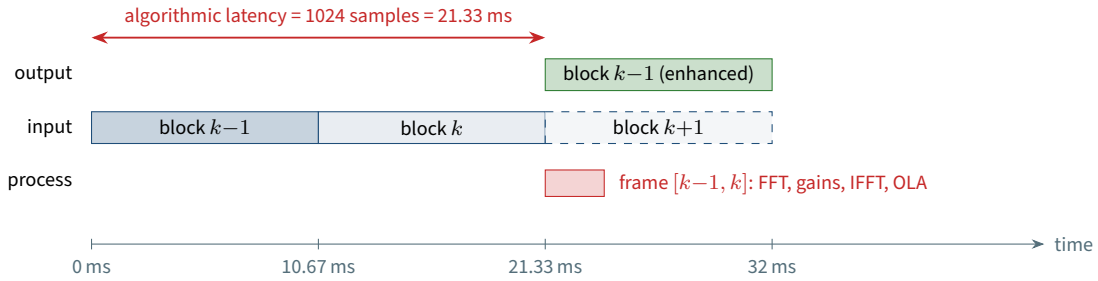


Figure 3.3: Why the thesis pipeline delays every sample by one full window. Block $k-1$ cannot leave until block k has arrived, because its overlap-add output needs the next frame. Processing time, drawn short here, adds to this.

For a frame-based design with 50 % overlap, algorithmic latency equals the window length. Latency is therefore a design variable you can trade against frequency resolution and memory (Table 3.7, measured state sizes). This is research question RQ2.

Table 3.7: Alternative frame configurations (no zero padding, hop = window/2).

f_s	window	bins	latency	frame rate	state (words)
48 kHz	2048 FFT*	1025	21.33 ms	93.75 Hz	23,575
48 kHz	1024	513	21.33 ms	93.75 Hz	11,799
48 kHz	512	257	10.67 ms	187.5 Hz	5,911
16 kHz	256	129	16.00 ms	125 Hz	2,967
16 kHz	128	65	8.00 ms	250 Hz	1,495
16 kHz	64	33	4.00 ms	500 Hz	759

* The thesis configuration: 1024-sample window zero-padded to 2048.

Time constants are defined in frames

IMCRA's smoothing factors ($\alpha_s = 0.9$, $\alpha_d = 0.85$), the decision-directed factor (0.98) and the minimum-search window ($D = U \times V = 120$ frames, about 1.28 s) are all expressed *per frame*. If you halve the hop and keep these values, every estimator suddenly adapts twice as fast. A frame-configuration study that forgets this compares different algorithms, not different frame sizes. To keep the time constants fixed when the hop changes from H to H', rescale them: $\alpha' = \alpha^{H'/H}$, and $D' = D\,H/H'$ frames. Chapter 5 builds this into the sweep.

3.7 The control plane

The intent path runs rarely, but each run is large (Table 3.8). Almost all of the cost is in the sentence encoder. The classifier and severity heads are tiny matrix–vector products. This is a direct consequence of your thesis finding that the classes are linearly separable in the embedding space.

Table 3.8: Control-plane cost per utterance. Encoder figures are estimates (2× non-embedding parameters × about 14 tokens; the complaint sentences average 7.8 words). Head figures are exact from the model shapes.

Component	Parameters	Compute per utterance	Weights (fp32 / int8)
`all-mpnet-base-v2` [4], [5]	≈110 M	≈2.4 GFLOP	≈440 MB / ≈110 MB
`all-MiniLM-L6-v2` [30]	≈22.7 M	≈0.3 GFLOP	≈90 MB / ≈23 MB
Linear SVC head (one-vs-one)	15 × 768	11,520 MAC + voting	45 KiB / 11.25 KiB
Logistic regression head	6 × 768	4,608 MAC + argmax	18 KiB / 4.5 KiB
Ridge severity head	768	768 MAC + clip	3 KiB / 0.75 KiB

A detail that matters: the selected classifier is one-vs-one

The thesis selected `SVC(kernel="linear", probability=True)` (`src/intent_se/nlp/classifier.py`). scikit-learn's `SVC` trains one binary classifier for *every pair* of classes: 15 hyperplanes for 6 classes, so 15 rows rather than 6. Its probability output, which the confidence gate uses, comes from Platt scaling followed by an iterative pairwise-coupling solve [31], [32]. That is awkward to put in hardware. Logistic regression tied the SVM on cross-validated macro-F1 (0.964). It needs 2.5× fewer multiply–accumulates, and its confidence is simply its largest score. The thesis broke the tie on generalisation, which was right for a laptop. On a chip, the tie may break the other way. Phase 1 measures this (opportunity O9).

3.8 Decisions that flip under hardware constraints

Several thesis decisions were correct for a laptop proof of concept but should be revisited for hardware (Table 3.9). Explaining these flips clearly is one of the most valuable things you can say in an interview, because it shows you understand that "optimal" depends on the cost model.

Table 3.9: Thesis decisions to revisit, and where in this guide each is tested.

Thesis decision	Thesis reason	Under hardware constraints	Tested in
mpnet (110 M) over MiniLM	"Inference speed is not a bottleneck"	The encoder is the dominant control-plane cost	RQ4, Chapter 9
2× zero padding (2048-pt FFT)	Smoother gain curves	Doubles FFT work and per-bin state	H2, Chapter 5
48 kHz, 1024 window	Soundcard block size, frequency resolution	21.33 ms latency, 1025 bins of state	RQ2, Chapter 5
float64 everywhere	Free in Python	Word length sets area, energy and memory	RQ1, Chapter 5
Linear SVM (one-vs-one, Platt)	Best val→test generalisation	LR ties on F1, needs 2.5× fewer MACs and has a simple confidence	H3, Chapters 5 and 9
Tilt computed per bin per frame	Simple code	Precompute at control rate (O2)	Chapters 5 and 6
Store 5 pre-roll frames	Simple code	Running minimum in place, no buffer	Chapter 6

3.9 Summary of hardware opportunities

The rest of the guide refers to these opportunities by label (Table 3.10). Each one is a hypothesis about cost and accuracy, not a guaranteed win. Phase 1 measures the accuracy side and Phases 2–5 measure the cost side.

Table 3.10: Hardware opportunities found in the thesis code.

ID	Idea	What it saves	Validated in
O1	Gain LUT $G(\xi_{\mathrm{dB}}) = \max\left((\xi/(1+\xi))^{\beta}, G_{\mathrm{floor}}\right)$, rebuilt by the control plane, double-buffered	a pow and a division per bin; turns the NLP→DSP interface into a table write	Chapters 5 and 6
O2	Tilt as a geometric accumulator $g_k = g_{\mathrm{out}}\, r^k$, $r = 10^{\mathrm{tilt}/(20(K-1))}$	a 10^x per bin	Chapters 5 and 6
O3	Cache $H = \xi/(1+\xi)$ from the Wiener stage for next frame's IMCRA	a reciprocal per bin	Chapter 5
O4	Share reciprocals across the 9 distinct denominators; LUT seed + Newton–Raphson	3 of 12 divisions, then cheap reciprocals	Chapters 5 and 6
O5	Log-domain values in the trackers and SNR comparisons	bits in 18 of 23 state words; divisions become subtractions	Chapter 5
O6	Remove the 2× zero padding	half the FFT work and half the state	Chapter 5
O7	$e^{-\nu}$ by LUT or piecewise-linear (or the sigmoid rewrite, Equation (3.3))	an exp per bin	Chapter 5
O8	Block-floating-point FFT and power computation	word length in the FFT	Chapters 5 and 6
O9	Logistic-regression or margin-based confidence; int8 head with Ridge folded in as a 7th row	2.5× fewer MACs; no iterative probability solve	Chapters 5 and 9

3.10 Do it yourself

Step 3.1 Reproduce this chapter's measurements

From the root of the thesis repository run:

```
PYTHONPATH=src python docs/research_guide/scripts/measure_workload.py
```

Check: the output reports 23 mutable words per bin (18 in the trackers), 23,575 words of state, a frame period of 10.667 ms and a latency of 21.333 ms. These numbers must match appendix A exactly. The Python time per frame varies from machine to machine.
Effort: 15 minutes.

Step 3.2 Turn the operation estimate into a measurement (optional)

Table 3.3 was counted by hand. You can count automatically instead. Wrap the numpy arrays inside `IMCRA.update` and `ParametricWienerFilter.compute_gain` in a thin `ndarray` subclass whose `__array_ufunc__` increments a counter per ufunc (`multiply`, `divide`, `minimum`, `exp`, ...) by the number of elements, then process one frame. Divide the counts by 1025 to get per-bin numbers.
Check: your divisions per bin equal 12 and your exp/power calls equal 2. If not, find the discrepancy. It will teach you something about the code.
Effort: 2–3 hours. Put the result in your lab notebook. It is a good first entry.

You are ready for the project chapters when

- ☐ You can explain, without notes, why memory rather than arithmetic speed is the main hardware problem for this system.
- ☐ You can name the two most expensive nonlinear operations per bin and how each will be replaced.
- ☐ You know where the 21.33 ms of latency comes from and what changing the window does to latency, memory and IMCRA time constants.
- ☐ You have reproduced the measured numbers on your own machine.

Part II

The Project, Step by Step

Chapter 4

Phase 0 — Setting Up Your Research Environment

[Weeks 1–2] Milestone M0

Phase 0 builds everything the rest of the project stands on: a repository with a sensible structure, a working hardware toolchain, evaluation data, a trusted metrics harness, and a measured *float baseline*. Every later result is compared against that baseline. It is unglamorous work, but a day saved here costs a week later.

Why this matters for your project

RQ1 and RQ2 are questions about *differences*: fixed point against float, 8 ms against 21 ms. A difference is only meaningful if the reference is measured carefully and reproducibly, with the same data, the same metric code and the same alignment. That reference is the thesis pipeline, so Phase 0 measures it properly for the first time on a standard benchmark.

Inputs	Outputs
The thesis repository; a computer (macOS, Linux or Windows); about 30 GB of free disk space; access to an x86-64 Linux or Windows machine for Vivado later	`intent-se-hw` repository with CI; working toolchain; VoiceBank+DEMAND on disk; `metrics/quality.py` with tests; baseline results table; lab notebook

4.1 Steps

Step 4.1 Start a lab notebook

Create `docs/notebook/` in the new repository (next step) and write one Markdown file per working day, named `YYYY-MM-DD.md`. Each entry records the goal, what you did, the commands you ran, the commit hash, the results and the next step. A template is in appendix E.

Why so formal? In week 14 you will need to know exactly which configuration produced the plot in week 5. Memory will not tell you, and your notebook will.

Check: today's entry exists and says what Phase 0 must achieve. **Effort:** 30 minutes now, then 5 minutes per session for the rest of the project.

Step 4.2 Create the `intent-se-hw` repository

Keep the thesis repository as an honest record of the thesis, and start a new one for the hardware project. It imports the thesis package as its floating-point reference.

```
intent-se-hw/
  golden/        bit-accurate Python model (Phase 1)
  metrics/       PESQ, STOI, SI-SDR, delta-SNR
  experiments/   one folder per experiment: config.yaml, results.csv, notes.md
  nlp_hw/        classifier-head quantisation, encoder studies
  rtl/           SystemVerilog (Phase 2)
  tb/            cocotb testbenches (Phase 3)
  fpga/pynq_z2/  Vivado Tcl scripts, overlay, notebooks (Phase 4)
  asic/          LibreLane configuration, constraints, reports (Phase 5B)
  docs/          spec, notebook, figures
  .github/workflows/  CI
```

```
mkdir intent-se-hw && cd intent-se-hw && git init
python3 -m venv .venv && source .venv/bin/activate
# the thesis code as an editable dependency (adjust the path)
pip install -e ../Intention_based_speech_enhancment_using_NLP
pip install numpy scipy soundfile pandas matplotlib pytest ruff pesq pystoi pyyaml
pip freeze > requirements-lock.txt
```

Add a `pyproject.toml` for your own packages (`golden`, `metrics`, `nlp_hw`) and a minimal `README.md` describing the goal and layout. Commit.
Check: `python -c "from intent_se.audio import SpeechEnhancer; print(SpeechEnhancer().self_test())"` prints a value around 4×10^{-16}. **Effort:** 1–2 hours.

Step 4.3 Install the open-source hardware toolchain

You need a simulator and linter (Verilator [20]), a Python testbench framework (cocotb [19]), a second simulator as a cross-check (Icarus Verilog), a waveform viewer (GTKWave or Surfer), and a synthesis tool for quick area estimates (Yosys [33]).
The easiest route on Linux and macOS is the **YosysHQ OSS CAD Suite**. It is a single download that bundles Yosys, Verilator, Icarus Verilog, GTKWave, SymbiYosys and more. Unpack it and source its environment script. On macOS, Homebrew (`brew install verilator icarus-verilog yosys`) also works. Distribution packages of Verilator are often too old for current cocotb, so check the minimum Verilator version in the cocotb documentation.
Install cocotb into your project's virtual environment so it uses the same Python as your golden model: `pip install cocotb`.
Smoke-test everything with a ten-line counter:

Listing 4.1: `sandbox/counter.sv`

```
module counter (input logic clk, input logic rst, output logic [7:0] count);
  always_ff @(posedge clk) begin
    if (rst) count <= '0;
    else     count <= count + 8'd1;
  end
endmodule
```

Listing 4.2: `sandbox/test_counter.py`

```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles

@cocotb.test()
async def counts_up(dut):
    cocotb.start_soon(Clock(dut.clk, 10, "ns").start())
    dut.rst.value = 1
```

```
    await ClockCycles(dut.clk, 3)
    dut.rst.value = 0
    await ClockCycles(dut.clk, 2)
    start = int(dut.count.value)
    await ClockCycles(dut.clk, 5)
    assert int(dut.count.value) == start + 5
```

```
verilator --version && yosys -V && iverilog -V | head -1
verilator --lint-only -Wall sandbox/counter.sv          # no warnings expected
yosys -p "read_verilog -sv sandbox/counter.sv; synth -top counter; stat"
```

Run the cocotb test with cocotb's Makefile flow or its Python runner. The variable names changed between cocotb 1.x and 2.x (for example `TOPLEVEL` became `COCOTB_TOPLEVEL`), so copy the quick-start example for your installed version from the cocotb documentation [34].
Check: the lint is clean, Yosys reports a cell count, and the cocotb test passes on Verilator *and* on Icarus. **Effort:** 3–5 hours. Tool installation is where newcomers lose time, so timebox it and ask for help early.

Step 4.4 Secure a Vivado host now (you need it in week 12)

AMD Vivado, needed for the PYNQ-Z2 in Phase 4, runs on x86-64 Linux and Windows only [35]. It does not run on macOS, and Apple-silicon Macs cannot run it natively. If your main computer is a Mac, arrange one of these *now*:

- a lab or project PC at your university;
- a remote Linux machine you can reach by SSH or remote desktop;
- a cloud x86-64 Linux VM. Vivado needs several CPU cores, plenty of RAM and a large disk.

The free edition of Vivado supports the Zynq-7020 on the PYNQ-Z2. Check which Vivado version your chosen PYNQ image release expects before downloading (Chapter 8); the installer is very large.
Check: you can log in to the machine and have enough disk space for Vivado. **Effort:** 1–2 hours to arrange; the installation itself can happen in the background later.

Step 4.5 Get the evaluation data

Download **VoiceBank+DEMAND** [6], [36] from the University of Edinburgh DataShare (DOI `10.7488/ds/2117`). It is the most widely used benchmark for single-channel speech enhancement. It is recorded at 48 kHz, the same rate as your thesis system, and it provides clean and noisy versions of every utterance.

- **Test set:** 824 utterances from two unseen speakers, at SNRs of 2.5, 7.5, 12.5 and 17.5 dB. This is where you report results.
- **Training set:** 11,572 utterances from 28 speakers, at SNRs of 0, 5, 10 and 15 dB. You do not train anything on the audio side, but use a fixed subset of it as a *development* set for tuning (LUT sizes, word lengths), so the test set is touched only once. This is the same discipline as your thesis's classifier selection.

Also generate a small set of **synthetic signals** for hardware testing: silence, a full-scale sine, a quiet sine in white noise, an impulse, a chirp, DC, and clipped noise. Real speech hides bugs that these signals expose immediately (Chapter 7).
Check: a script counts 824 clean and 824 noisy test files, all at 48 kHz, and the synthetic signals are generated deterministically from a fixed seed. **Effort:** 2 hours plus download time.

Step 4.6 Build and test the metrics harness

Write `metrics/quality.py` with four metrics. Each one takes the clean reference and the processed signal, already aligned (step 4.7).

- **PESQ**, wideband, ITU-T P.862.2 [37], [38]. The common Python package `pesq` accepts only 8 or 16 kHz signals, so resample from 48 to 16 kHz first.
- **STOI** [39], via `pystoi`.
- **SI-SDR** [40], in a few lines of numpy. It has no dependencies, so it is your fallback if a package breaks.
- **ΔSNR**: the output SNR minus the input SNR, both against the clean reference.

Listing 4.3: The core of `metrics/quality.py`.

```python
import numpy as np
from math import gcd
from scipy.signal import resample_poly
from pesq import pesq
from pystoi import stoi

def to_16k(x, fs):
    g = gcd(16000, fs)
    return resample_poly(x, 16000 // g, fs // g)

def si_sdr(ref, est):
    ref = ref - ref.mean(); est = est - est.mean()
    target = (np.dot(est, ref) / np.dot(ref, ref)) * ref
    noise = est - target
    return 10 * np.log10(np.sum(target**2) / np.sum(noise**2))

def quality(ref, est, fs=48000):
    return {
        "pesq_wb": pesq(16000, to_16k(ref, fs), to_16k(est, fs), "wb"),
        "stoi": stoi(ref, est, fs, extended=False),
        "si_sdr": si_sdr(ref, est),
    }
```

Write unit tests (`metrics/test_quality.py`) that pin down each metric's behaviour:

- identical signals give PESQ near its maximum, STOI near 1, and a very high SI-SDR;
- a clean signal plus white noise at a known SNR gives an SI-SDR within about 0.1 dB of that SNR;
- scaling the estimate leaves SI-SDR unchanged, since that is the "scale-invariant" part.

Check: `pytest metrics` passes. **Effort:** 3–4 hours.

Step 4.7 Measure the float baseline

Run the thesis `SpeechEnhancer` over the whole test set and record the metrics per utterance. Do this for the unprocessed noisy input and for $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ with the other parameters at their defaults.
Two details matter:

1. **Alignment.** Offline, `process_signal` returns output delayed by exactly 512 samples (one hop) relative to its input. This was measured with an impulse. Shift the output by 512 samples before computing metrics, otherwise PESQ and SI-SDR punish a delay that is not a quality problem. Verify the shift yourself by cross-correlation on one file.
2. **Pre-roll.** The first 5 frames (about 53 ms) pass through unprocessed while IMCRA is seeded.

Keep this behaviour, since the hardware will have it too, but note it in your results.

Save everything under `experiments/000-baseline/`:

- `config.yaml`: parameters, commit hash, package versions;
- `results.csv`: one row per utterance and setting;
- `summary.md`: means and 95% confidence intervals per setting and per input SNR (Chapter 11).

Check: enhancement improves SI-SDR and ΔSNR over the noisy input. If PESQ or STOI get worse at some β values, do not panic: it is a known property of Wiener-type filters, and a finding worth writing down. The whole run is reproducible from one command. **Effort:** 4–6 hours (the run itself takes minutes).

Step 4.8 Set up continuous integration

Add a GitHub Actions workflow (`.github/workflows/ci.yml`) that installs the Python environment and runs `ruff` and `pytest` on every push. In Phase 3 you will add Verilator and the cocotb regression to the same workflow.
Check: the CI badge is green on the main branch. **Effort:** 1–2 hours.

4.2 Pitfalls

Pitfalls in Phase 0

- **Unaligned signals.** Forgetting the 512-sample delay makes every metric look worse than it is, and the error changes with the frame configuration in Phase 1, which corrupts your comparisons.
- **Tuning on the test set.** Choose every parameter on the development subset. Touch the test set only to report results.
- **Unpinned versions.** PESQ implementations and resamplers differ slightly. Pin package versions and record them with each result.
- **Leaving Vivado until week 12.** A Mac cannot run it. Solve this now.
- **Polishing the setup forever.** Phase 0 has two weeks. A working, tested harness is enough; elegance can come later.

Milestone M0 is reached when

- ☐ `intent-se-hw` exists with the agreed layout, a lock file and green CI.
- ☐ Verilator, Icarus, Yosys, cocotb and a waveform viewer pass their smoke tests.
- ☐ A Vivado host is arranged.
- ☐ VoiceBank+DEMAND and the synthetic signals are on disk.
- ☐ `metrics/quality.py` is implemented and unit-tested.
- ☐ The float baseline is measured, aligned, saved and summarised.
- ☐ The lab notebook has an entry for every working day so far.

Chapter 5

Phase 1 — Algorithm–Hardware Co-Design and the Fixed-Point Golden Model

[Weeks 3–7] Milestone M1

This is the most important phase of the project. You turn the floating-point thesis code into a *hardware specification*: a sample rate, frame size, number format and function approximation for every signal, each justified by an experiment. You also build a Python model that reproduces, bit for bit, what the hardware will compute. Phase 2 then implements that model in RTL, and Phase 3 proves the RTL matches it.

Why this matters for your project

Phase 1 answers RQ1 (precision) and RQ2 (latency), tests H1 and H2, and gives a first answer to RQ4 (H3, the quantised classifier head). It is also where most of the *research* in the project happens. RTL implements decisions; this phase makes them. A hardware engineer who can say "16 bits here, because this sweep shows that 14 costs 0.08 PESQ" is worth far more than one who picked 16 because it is a round number.

Inputs	Outputs
M0: metrics harness, float baseline, development and test sets; Chapter 3 and appendix A	`golden/` bit-accurate model; sweep results and plots; a specification document; golden test vectors; NLP head quantisation results; repository tag `spec-v1`

Concept: Bit-accurate versus bit-exact

A *bit-accurate* model computes with exactly the integers the hardware will hold. Every rounding, saturation and table lookup is explicit, and none of it is left to numpy's floating point. When the RTL later produces the same integers on the same inputs, the two are *bit-exact*. That is what verification checks (Chapter 7). Floating-point code can only ever be "close", and "close" gives you nothing to test against when debugging hardware.

5.1 Steps

Step 5.1 Instrument the float model and profile dynamic range

Before choosing any number format you need to know the range of every signal. Subclass or wrap `IMCRA` and `ParametricWienerFilter` so each intermediate quantity is recorded per frame, without changing any result: `power`, S, $S_{\min}$, $\gamma_{\min}$, ζ, the indicator, $\tilde{S}$, $\tilde{S}_{\min}$, q, γ, ν, p, λ_d, ξ, G, and the output samples.
Run the development set through it. For each signal, record the minimum, the maximum, the 0.1st and 99.9th percentiles, and a histogram of $\log_2 |x|$.
Check: with the probes enabled, the output is identical to the uninstrumented model (assert this). You can fill the first two columns of the specification table (Table 5.1). Expect power-like quantities to span more than 100 dB. That is the first sign they want a logarithmic or floating-point format rather than plain fixed point.
Effort: 1–2 days.

Step 5.2 Frame-configuration study (RQ2, H2)

Sweep the configurations in Table 3.7, plus the thesis configuration with and without zero padding. The thesis `AudioConfig` already accepts other sizes, provided `hop = window/2` and `block = hop`. For 16 kHz runs, resample the input and the clean reference with `resample_poly`.
The step people forget is rescaling the time constants. IMCRA's smoothing factors, the decision-directed factor and the minimum-search window are all defined per frame. Keep their *time constants* fixed when the frame period $T = H/f_s$ changes to T':

Listing 5.1: Keeping time constants fixed across frame configurations.

```
from dataclasses import replace

def rescale(imcra, wiener, T_old, T_new):
    r = T_new / T_old                    # ratio of frame periods
    a = lambda alpha: alpha ** r         # tau = -T/ln(alpha) stays fixed
    v_new = max(1, round(imcra.v / r))   # keep U = 8 sub-windows, D*T fixed
    return (replace(imcra, alpha_s=a(imcra.alpha_s), alpha_d=a(imcra.alpha_d), v=v_new),
            replace(wiener, dd_alpha=a(wiener.dd_alpha)))
```

For each configuration, record PESQ, STOI and SI-SDR on the development set, together with the algorithmic latency and the state memory. Plot quality against latency and mark the configurations that dominate the others: they form the Pareto front (Figure 5.1, right).
Check: the thesis configuration reproduces the baseline exactly. You can state H2's outcome: what removing the zero padding costs in PESQ. You have chosen a design point and written down why (decision point, week 4, Chapter 13).
Effort: 3–4 days.

Step 5.3 Write a bit-true fixed-point library

Every fixed-point operation in the model goes through a few helper functions. The RTL later implements exactly the same rounding and saturation. Use numpy `int64` arrays, which have enough headroom for the products of two 32-bit values.

Listing 5.2: The heart of `golden/fixedpoint.py`.

```
import numpy as np

def limits(word, signed):
```

```
    return (-(1 << (word - 1)), (1 << (word - 1)) - 1) if signed else (0, (1 << word) - 1)

def saturate(q, word, signed=True):
    lo, hi = limits(word, signed)
    return np.clip(q, lo, hi).astype(np.int64)

def quantize(x, word, frac, signed=True):
    """Real value(s) -> integer(s) in Q(word-frac).frac, round half up, saturate."""
    q = np.floor(np.asarray(x, dtype=np.float64) * (1 << frac) + 0.5)
    return saturate(q, word, signed)

def requant(q, frac_in, word_out, frac_out, signed=True):
    """Change the binary point of integer q, rounding half up, then saturate."""
    shift = frac_in - frac_out
    if shift > 0:
        q = (q + (1 << (shift - 1))) >> shift      # arithmetic shift = floor
    else:
        q = q << (-shift)
    return saturate(q, word_out, signed)

def mul(a, fa, b, fb, word_out, frac_out, signed=True):
    """Exact integer product, then one rounding and one saturation."""
    return requant(a.astype(np.int64) * b.astype(np.int64), fa + fb,
                   word_out, frac_out, signed)

def to_real(q, frac):
    return q.astype(np.float64) / (1 << frac)
```

Choose *one* rounding mode and use it everywhere. Round-half-up (add half an LSB, then shift right) costs one adder in hardware, and that is what the code above does.
Check: unit tests for exact representable values, rounding at the half-LSB boundary for positive and negative numbers, saturation at both limits, and `mul` against exact rational arithmetic (Python `fractions`) on random inputs. **Effort:** 1 day.

Step 5.4 Build the bit-accurate model, block by block, in hardware order

Write one module per hardware block: `golden/fx_stft.py`, `golden/fx_imcra.py`, `golden/fx_wiener.py`, and `golden/fx_pipeline.py` to connect them. Quantise one block at a time and compare against float after each change. If you quantise everything at once and the quality collapses, you will not know why.

1. **Window and framing.** The $\sqrt{\text{Hann}}$ window becomes a ROM of quantised coefficients. By symmetry only half of it needs storing.
2. **FFT.** Model the *hardware* algorithm exactly, not `np.fft`. Write a radix-2 decimation-in-time FFT [10] with quantised twiddle factors, and round after every butterfly. Add *block floating point* (O8): after each stage, if any value has grown into the guard bits, shift the whole block right by one and increment a shared exponent. The inverse FFT reuses the forward FFT through conjugation: $\mathrm{IFFT}(X) = \overline{\mathrm{FFT}(\overline{X})}/N$.
3. **Power.** $|X_k|^2$ doubles the dynamic range in bits. This is where you move to a logarithmic or small floating-point format for power-like quantities (step 5.5).
4. **IMCRA and Wiener.** Implement O3 (cache $\xi/(1+\xi)$), O4 (shared reciprocals via seed table + Newton–Raphson) and O1 (gain LUT, step 5.6).
5. **Tilt and output gain** (O2), the gain multiply, the inverse FFT, the synthesis window and overlap-add.

Check: with very wide formats (for example 32 fractional bits everywhere and large tables), the

model matches float to within about 0.01 PESQ and 0.001 STOI. That shows the *structure* is right before you start narrowing word lengths.
Effort: 1.5–2 weeks. The FFT and block floating point take the most time.

Step 5.5 Decide the representation of power-like quantities (O5)

$|X|^2$, S, $\tilde{S}$, their minima and λ_d span a huge range. There are three candidate representations:

1. **Wide fixed point** (about 32 bits or more). Simple, but expensive in memory, and memory is 23 words per bin.
2. **Small custom floating point**, for example a 10-bit mantissa with a 6-bit exponent. Its relative precision is constant across the range.
3. **Logarithmic**, for example $\log_2$ as a signed Q6.10 value. Multiplications and divisions become additions and subtractions, and comparisons stay exact.

The minimum trackers, which hold 18 of the 23 state words, are a natural fit for the log domain. A minimum is unchanged by any monotone map, so $\min(\log a, \log b) = \log \min(a, b)$ exactly. The threshold tests in `imcra.py` also survive unchanged: $\gamma_{\min} < \gamma_0$ becomes $\log_2 P - \log_2(B_{\min} S_{\min}) < \log_2 \gamma_0$.
Recursive smoothing does not survive. $\alpha \log S + (1-\alpha) \log P$ is the log of a *geometric* mean, which is biased low compared with the arithmetic mean that IMCRA expects. For a single exponentially distributed periodogram bin the bias is about 2.5 dB, the Euler–Mascheroni constant in nepers. Smoothing reduces it, but it does not vanish.
Evaluate two options:

(a) keep the smoothed quantities S, $\tilde{S}$, λ_d in a small floating-point format, and store only the tracker minima as low-precision log values;

(b) smooth in the log domain and re-fit the bias factor $B_{\min}$ so the noise estimate is unbiased on the development set.

Check: for your chosen option, the noise PSD estimate on the development set has a median error under about 0.5 dB against float, and the end metrics meet RQ1. **Effort:** 3–4 days.

Step 5.6 Gain LUT and control-plane mapping (O1, O2)

Replace $(\xi/(1+\xi))^{\beta}$ by a table that the control plane computes whenever β or the gain floor changes:

Listing 5.3: Gain table generation (control plane) and lookup (data plane).

```
import numpy as np
from golden.fixedpoint import quantize

LOG2_STEP = 1 / 8            # LUT spacing: 1/8 of a log2 unit (about 0.38 dB)
LOG2_MIN = -9.0              # about -27 dB, just below xi_min = -25 dB
N_LUT = 256                  # covers 32 log2 units, about 96 dB

def make_gain_lut(beta, floor):
    log2_xi = LOG2_MIN + LOG2_STEP * np.arange(N_LUT)
    xi = 2.0 ** log2_xi
    g = np.maximum((xi / (1 + xi)) ** beta, floor)
    return quantize(g, word=16, frac=13, signed=False)     # UQ3.13

def lut_index(log2_xi_q, frac=10):
    # log2(xi) held as signed Q6.10. A power-of-two step makes the index a
    # subtraction and a shift -- no multiplier.
    idx = (log2_xi_q - int(LOG2_MIN * (1 << frac))) >> (frac - 3)
```

```
    return np.clip(idx, 0, N_LUT - 1)
```

Choosing the table spacing as a power of two in $\log_2$ units is a small co-design decision with a real payoff: the table index is a bit slice of the log-domain ξ, so no multiplier is needed. For the tilt (O2), precompute $r = 10^{\text{tilt}/(20(K-1))}$ and form $g_k = g_{\text{out}}\, r^k$ with one multiply per bin. Repeated multiplication accumulates rounding error over the K bins, so check the error at the last bin. Alternatively, add a constant per bin in the log domain.
Sweep the table size (64–1024 entries), the ξ range, and linear interpolation versus none.
Check: applying the thesis's own parameter changes (for example β from 1.0 to 1.94, then a tilt step) produces the same audio as the float model to within the RQ1 target. The table swap at a frame boundary produces no click: compare the difference signal around the switch. **Effort:** 2–3 days.

Step 5.7 Word-length and approximation sweeps (RQ1, H1)

Now narrow the formats. Use the development set, and a fixed subset of it while iterating.

1. **One factor at a time.** Vary one signal's word length (or one table's size), keeping all others wide. Find the knee where quality starts to fall (Figure 5.1, left).
2. **Joint.** Set every signal a little above its knee and run the whole model. Quantisation errors add up, so you will usually need a bit or two more than the one-at-a-time results suggest.
3. **Nonlinear functions.** Size the reciprocal seed table and the number of Newton–Raphson iterations. One iteration roughly doubles the number of correct bits. Size the $e^{-\nu}$ table (O7) over $\nu \in [0, 16]$, or the sigmoid rewrite of Equation (3.3). Derive an accuracy target for each function from the end metric, not from intuition.

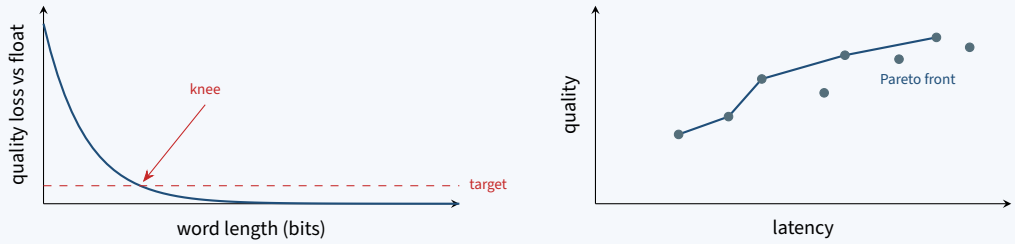


Figure 5.1: Schematic — illustrative shape, not data. Left: quality loss against word length typically falls steeply and then flattens. Pick the smallest word length past the knee that meets the target. Right: configurations that are not beaten on both latency and quality form the Pareto front.

Check: a joint configuration meets RQ1 on the development set. A table lists every signal with its format and the experiment that justifies it. You can state H1's outcome. **Effort:** 1 week (mostly computer time; run the sweeps overnight).

Step 5.8 Quantise the NLP heads (H3, first part of RQ4)

Use the thesis pipeline's cached sentence embeddings (`SentenceEmbedder` writes them to its `cache_dir`) and the thesis's fixed train/test split. Fit logistic regression and the thesis's linear SVM on the same split, then quantise the weights to int8 with one scale per row:

Listing 5.4: Post-training int8 quantisation of a linear head.

```
import numpy as np

def quantize_head(W, b, X):
    s_w = np.abs(W).max(axis=1, keepdims=True) / 127        # per-row scale
```

```
    Wq = np.clip(np.round(W / s_w), -127, 127).astype(np.int8)
    s_x = 1.0 / 127                                    # |x_i| <= 1 (L2-normalised)
    Xq = np.clip(np.round(X / s_x), -127, 127).astype(np.int8)
    acc = Xq.astype(np.int32) @ Wq.astype(np.int32).T      # int32 accumulators
    return acc * (s_w.T * s_x) + b                      # dequantised scores

# Logistic regression: 6 rows -> argmax.  Linear SVC (one-vs-one): 15 rows,
# then the same pairwise voting as sklearn -- check your voting code on the
# float weights first: it must reproduce clf.predict exactly.
```

Report macro-F1 (float and int8), the multiply–accumulate count (4,608 against 11,520), and the severity MAE with the Ridge weights quantised the same way. Replace the SVM's Platt/pairwise-coupling probability with a margin-based confidence (O9). For example, use the difference between the top two scores, and pick its threshold on the training folds so the confidence gate behaves like the thesis's.
Check: the float numbers reproduce the thesis (test macro-F1 0.957 for the SVM). You can state H3's outcome. **Effort:** 2 days.

Step 5.9 Freeze the specification and export golden vectors

Write `docs/spec.md`: the chosen frame configuration, every signal's format (Table 5.1), every table's size and contents rule, the rounding mode, the block-floating-point rule, and the parameter-update semantics (table bank swap at a frame boundary).
Then export *golden vectors* with `golden/vectors.py` for the verification phase. For each test signal (the synthetic set plus a few VoiceBank utterances), write one file per probe point: the block inputs and outputs per frame, as integers, in `.npy` or one-hex-word-per-line `.hex` files. These are the contract between Python and RTL.

Table 5.1: Specification table template — fill one row per signal and table.

Signal	Symbol	Format	Measured range	Evidence (experiment)
Input sample	$x[n]$	Q1.15		`experiments/...`
FFT data path	X_k			
Power	$\lvert X_k\rvert^2$			
Tracker minima	$S_{\min}$, $\tilde{S}_{\min}$			
Noise PSD	λ_d			
A priori SNR	ξ			
Gain	G_k	UQ3.13		
Gain LUT	—	256 × 16 bit		
…				

Check: a fresh clone can regenerate every vector and every plot from one command. Tag the repository `spec-v1`. **Effort:** 2 days.

5.2 Pitfalls

Pitfalls in Phase 1

- **Using `np.fft` in the "bit-accurate" model.** The hardware FFT rounds after every stage. If the model does not, the RTL can never match it.
- **Silent float leaks.** A single `np.log2` or `/` on a fixed-point path makes the model no longer bit-accurate. Grep for them, and assert integer dtypes at block boundaries.

- **Comparing frame configurations without rescaling time constants.** You would be comparing different algorithms.
- **Optimising one signal at a time and stopping there.** Errors add up. Always finish with a joint run.
- **Tuning on the test set.** Use the development set for every choice.
- **Scope creep.** Phase 1 is fascinating and could last forever. It ends in week 7 (Chapter 13).

Read

Fixed-point arithmetic and quantisation noise: Widrow and Kollár [7]. Word-length optimisation: Constantinides et al. [41]. Elementary function approximation (tables, polynomials, Newton–Raphson): Muller [8]. IMCRA itself: Cohen [2]. Integer-only inference: Jacob et al. [24].

Milestone M1 is reached when

- ☐ The dynamic-range profile and the frame-configuration study are complete and plotted.
- ☐ The time-constant rescaling is implemented and tested.
- ☐ `golden/` is bit-accurate (integer-only on fixed-point paths) and unit-tested.
- ☐ The chosen formats meet RQ1 on the development set; H1 and H2 have recorded outcomes.
- ☐ The gain LUT, tilt mapping and table-swap semantics are specified and tested.
- ☐ The NLP heads are quantised; H3 has a recorded outcome.
- ☐ `docs/spec.md` is complete, golden vectors are exported, and the repository is tagged `spec-v1`.

Chapter 6

Phase 2 — Microarchitecture and RTL Design

[Weeks 6–12] Milestone M2 (together with Chapter 7)

In this phase the specification becomes hardware. You first design the *microarchitecture*: which blocks exist, how data moves between them, and what happens on each clock cycle of a frame. Then you write it in synthesizable SystemVerilog, one block at a time, each verified against the golden model before you move on (Chapter 7). Build and test go together: no block is "done" until its test passes.

Why this matters for your project

Phase 2 is where you learn the core skill of VLSI DSP: mapping an algorithm onto time and hardware resources. It produces the cycle counts that test H4 and the first resource numbers for RQ3. Most of the interesting decisions already have a cost model, from Chapter 3, and a specification, from Chapter 5. Here you make them concrete.

Inputs	Outputs
`spec-v1`: formats, tables, frame configuration; golden vectors; the cocotb setup from Phase 0	Lint-clean RTL in `rtl/`; every block passing its tests; a resource log (Yosys and Vivado out-of-context synthesis) per milestone commit; a block diagram and frame schedule for the report

6.1 Top-level architecture

Figure 6.1 shows the datapath. Audio samples stream in at the sample rate. Frames are processed in bursts on a much faster system clock. Parameters arrive over a bus from the control plane.

Three decisions shape this architecture:

- **One clock domain.** The whole datapath runs on one system clock. A *sample strobe*, a one-cycle pulse at the sample rate generated by `frame_ctrl`, tells the front end when a new sample is valid. Crossing to the audio codec's own clock is handled at the edge of the FPGA design (Chapter 8), not inside `ise_top`.
- **One shared FFT.** Forward and inverse transforms never overlap within a frame, so one `fft_r2_mem` does both. The inverse uses the conjugation trick from Chapter 5.
- **Bins stream in order.** The per-bin pipeline consumes one bin per cycle, from $k = 0$ to $K - 1$. Almost everything is independent across bins, and the 5-tap frequency smoothing becomes a sliding window over a short shift register.

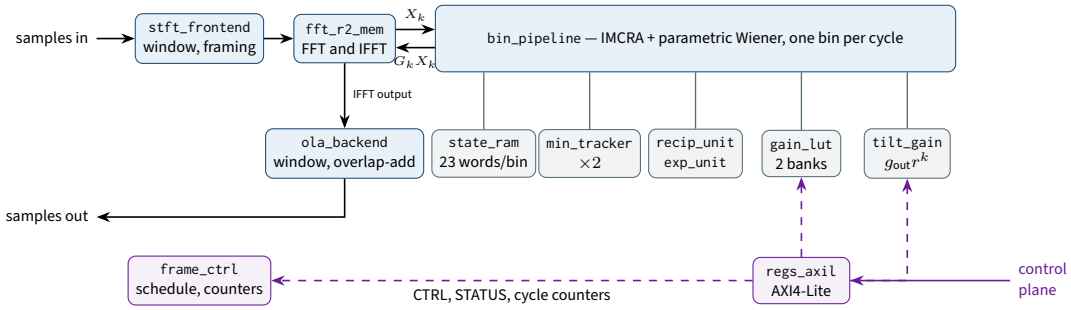


Figure 6.1: Microarchitecture of `ise_top`. Blue: data plane. Grey: helper units and memories of the per-bin pipeline. Purple, dashed: control. One FFT unit is shared between the forward and inverse transforms.

6.2 The frame schedule

A frame is processed in a burst of cycles, while the next block of samples is captured into the other half of a *ping-pong* buffer (Figure 6.2). The estimate below assumes the thesis frame configuration; for the configuration you chose in Phase 1, scale it by the number of bins.

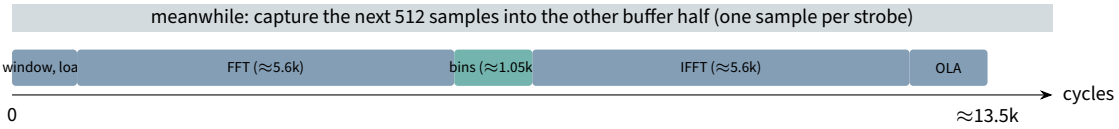


Figure 6.2: One frame of processing (estimate, thesis configuration, one butterfly per cycle, one bin per cycle). At 5 MHz the budget is 53,333 cycles per frame, so the datapath is busy for about a quarter of the frame and can be clock-gated for the rest.

This is the calculation behind hypothesis H4. Make `frame_ctrl` count the cycles of each phase in hardware and expose the total in a debug register. That makes H4 a measurement rather than an estimate.

6.3 Designing the blocks

6.3.1 The FFT (`fft_r2_mem`)

A *memory-based* radix-2 FFT keeps all N complex values in a dual-port RAM and runs one butterfly per cycle for $\log_2 N$ stages. It is the smallest FFT architecture and is fast enough here: 5,120 butterflies per 1024-point transform. Pipelined architectures such as R2SDF or R2^{2}SDF [11], [12] are faster, but they are designed for continuous streaming, which you do not need.

You need four pieces:

- **Address generation.** In stage s, butterfly j reads two addresses that differ in bit s. A counter plus a bit-insertion gives both.
- **Twiddle ROM.** $W_N^m = e^{-j2\pi m/N}$. Store a quarter cosine wave and derive sine and cosine for all m from symmetry.
- **Bit-reversed order.** Either write the input in bit-reversed order or read the output that way. Reversing an address is just wiring.
- **Block floating point.** Each stage can grow values by one bit. Detect growth into the guard bits during stage s, apply a right shift to the whole block in stage $s + 1$, and count the shifts in a shared exponent. This must follow `golden/fx_stft.py` exactly.

Start with the simplest correct version: a complex FFT that takes real input with zero imaginary part. It wastes half its work, but it is easy to verify. Only once it is bit-exact, implement the real-FFT packing (an N-point real FFT computed as an $N/2$-point complex FFT plus a post-processing pass), and only if Phase 1 modelled it.

6.3.2 The per-bin pipeline (`bin_pipeline`)

Map the Python code line by line onto pipeline stages (Table 6.1). Each stage holds only a few operations, so even long operations like the reciprocal can be pipelined to accept one bin per cycle.

Table 6.1: From Python lines to pipeline stages (an example grouping; your spec decides the details).

Stage	Work	Thesis source
1	Read state word for bin k; compute $\lvert X_k\rvert^2$ (or its log); push into the 5-bin window	`pipeline.py:179`
2	Frequency smoothing over bins $k-2 \dots k+2$; recursive smoothing of S	`imcra.py:117-118`
3	Minimum tracker 1; $\gamma_{\min}$, ζ (log domain: subtractions); indicator I_k	`imcra.py:119-126`
4	Second smoothing over $I \cdot P$ and I for bins $k-2 \dots k+2$; one reciprocal	`imcra.py:129-133`
5	Minimum tracker 2; $\tilde{\gamma}_{\min}$, $\tilde{\zeta}$; q	`imcra.py:134-146`
6	γ, ν (cached H, O3); p via `exp_unit` or the sigmoid rewrite; λ_d update	`imcra.py:149-164`
7	Wiener: decision-directed ξ (shared reciprocal of the noise PSD)	`wiener.py:89-98`
8	Gain: `gain_lut`[index(ξ)], times tilt and output gain from `tilt_gain`, clip	`wiener.py:102-112`
9	$G_k X_k$ to the IFFT input; previous clean power; write state back	`wiener.py:115`

Two details need care:

- **Look-ahead.** Smoothing at bin k needs bins $k+1$ and $k+2$. The first iteration therefore runs two bins *behind* the input, and the second smoothing, which needs the indicator of bins up to $k+2$, two bins behind that. Implement both with short shift registers. At the spectrum edges, `np.convolve(. . . , mode="same")` treats missing bins as zero, so your hardware must do exactly the same.
- **State memory.** Each bin's state is read once and written once per frame, at a different address from every other bin. There are therefore no read-after-write hazards between bins, only a latency to cover. Store each bin's state as one wide word, or as a few parallel memories, so the read takes one cycle.

6.3.3 Arithmetic and table units

`recip_unit`
Normalise the input with a leading-zero count, look up a seed for $1/d$ from the top mantissa bits, then do one Newton–Raphson step $y_1 = y_0(2 - dy_0)$, which costs two multiplies. Pipeline it so it accepts one input per cycle. Share it between the denominators that occur in the same stage.

`exp_unit`
A table over $\nu \in [0, 16]$, with or without linear interpolation, as specified.

`gain_lut`
Two banks of 256×16 bits. The datapath reads the *active* bank and the bus writes the *shadow* bank. On a commit request, the banks swap at the next frame boundary (appendix B).

`tilt_gain`
A running product: at $k = 0$ load g_{out}, then multiply by r once per bin (O2).

`min_tracker`
Combinational per-bin logic around the stored minima (Listing 6.2). The sub-window frame counter and slot pointer are global, so they live in `frame_ctrl`, not per bin.

6.4 RTL coding guidelines

Pick these rules on day one and never break them:

- Synthesizable SystemVerilog only [17]: `always_ff` for registers, `always_comb` for logic, and non-blocking assignments (<=) only in `always_ff` blocks [42].
- One reset style: synchronous, active-high `rst`.
- Naming: registers end in _q with next state _d; widths are parameters, never magic numbers.
- No latches: every `always_comb` assigns every output on every path. Assign defaults at the top.
- Lint clean with `verilator -lint-only -Wall` before every commit. A warning you do not understand is a bug you have not found yet.
- Rounding and saturation are written once, in `qmul_sat` and similar helpers, and match `golden/fixedpoint.py` exactly.

Listing 6.1: `qmul_sat`: the hardware twin of `fixedpoint.mul`.

```
// Saturating fixed-point multiply with round-half-up, bit-exact with
// golden/fixedpoint.py:mul.  Assumes SHIFT = A_F + B_F - O_F >= 1.
module qmul_sat #(
  parameter int A_W = 16, A_F = 15,
  parameter int B_W = 16, B_F = 13,
  parameter int O_W = 16, O_F = 13
) (
  input  logic signed [A_W-1:0] a,
  input  logic signed [B_W-1:0] b,
  output logic signed [O_W-1:0] y
);
  localparam int P_W   = A_W + B_W;
  localparam int SHIFT = A_F + B_F - O_F;
  localparam logic signed [P_W-1:0] HALF = 1 <<< (SHIFT - 1);
  localparam logic signed [P_W-1:0] MAXV = (1 <<< (O_W - 1)) - 1;
  localparam logic signed [P_W-1:0] MINV = -(1 <<< (O_W - 1));

  logic signed [P_W-1:0] p, r;
  always_comb begin
    p = a * b;                    // exact product
    r = (p + HALF) >>> SHIFT;     // round half up, arithmetic shift
    if      (r > MAXV) y = MAXV[O_W-1:0];
    else if (r < MINV) y = MINV[O_W-1:0];
    else               y = r[O_W-1:0];
  end
endmodule
```

Listing 6.2: `min_tracker`: per-bin logic of the two-stage minimum tracker.

```
// One bin per cycle. Log-domain values are stored with a bias so they are
// all non-negative: unsigned comparison is then exact (and cheap).
module min_tracker #(parameter int W = 8, parameter int U = 8) (
  input  logic                 sub_end,    // last frame of a V-frame sub-window
  input  logic [$clog2(U)-1:0] slot,       // sub-window slot to overwrite
  input  logic [W-1:0]         x,          // this frame's smoothed value
  input  logic [W-1:0]         running_q,  // stored running minimum
  input  logic [U-1:0][W-1:0]  store_q,    // stored sub-window minima
  output logic [W-1:0]         running_d,
  output logic [U-1:0][W-1:0]  store_d,
  output logic [W-1:0]         s_min       // = Python's np.minimum(store.min(), running)
);
  logic [W-1:0] run_new, m;
  always_comb begin
```

```
    run_new   = (x < running_q) ? x : running_q;
    store_d   = store_q;
    running_d = run_new;
    if (sub_end) begin
      store_d[slot] = run_new;    // close the sub-window
      running_d     = x;          // restart from the current value
    end
    m = running_d;
    for (int i = 0; i < U; i++)
      if (store_d[i] < m) m = store_d[i];
    s_min = m;
  end
endmodule
```

Read your RTL as hardware

The `for` loop in Listing 6.2 is not a loop in time. It describes eight comparators and multiplexers, which synthesis builds as a chain. At a few MHz a chain is fine. If timing ever fails, rewrite it as a balanced tree of depth $\log_2 9 \approx 4$. Always ask "what circuit does this describe?" [26].

6.5 Build order

Build from the leaves up. Each block gets a cocotb test, comparing it with the golden model, *before* the next block is started (Chapter 7).

Step 6.1 Skeleton, lint and synthesis flow

Create `rtl/` with empty modules and the port lists of Figure 6.1, a `Makefile` target for Verilator lint, and a Yosys script that synthesizes any module and prints `stat`. Add both to CI.
Check: `make lint` and `make synth TOP=ise_top` run cleanly on the skeleton. **Effort:** half a day.

Step 6.2 `qmul_sat` and the other arithmetic helpers

Implement Listing 6.1 and the matching add/saturate helpers.
Check: an exhaustive or large random cocotb test against `fixedpoint.mul` gives zero mismatches. Include the corner cases: both operands at their most negative value, and exact half-LSB ties.
Effort: 1 day, including your first real cocotb test.

Step 6.3 `recip_unit` and `exp_unit`

Implement the leading-zero normaliser, seed ROM, Newton–Raphson step and pipeline registers. Generate the ROM contents from Python with the same function the golden model uses, and write them as `.hex` files read with `$readmemh`.
Check: bit-exact over the full input range, or a dense random sample of it, for both units. **Effort:** 2–3 days.

Step 6.4 `gain_lut`, `tilt_gain` and `regs_axil`

Implement the register map of appendix B, the double-buffered table and the frame-boundary commit. A minimal AXI4-Lite slave has one write channel pair and one read channel. Handle the write address and data arriving in either order.
Check: a cocotb test writes a table and commits it, and the new values appear at the next frame boundary and not before. A second commit without rewriting the table is rejected or handled

exactly as specified. **Effort:** 3–4 days.

Step 6.5 `min_tracker` and `state_ram`

Implement Listing 6.2. Infer `state_ram` as a simple synchronous RAM: write the RTL so synthesis recognises a block RAM, and check the synthesis report to confirm.
Check: over many frames of golden vectors, $S_{\min}$ and $\tilde{S}_{\min}$ match the model exactly, including across sub-window boundaries and after pre-roll seeding. **Effort:** 2 days.

Step 6.6 `bin_pipeline`, fed with golden spectra

Assemble the per-bin pipeline (Table 6.1). Do not wait for the FFT: drive the pipeline directly with the golden model's spectra and compare its outputs and internal probes frame by frame.
Check: bit-exact for at least one full utterance and the whole synthetic set, including a parameter change. Record the pipeline latency and the cycles per frame. **Effort:** 1.5–2 weeks. This is the biggest block.

Step 6.7 `fft_r2_mem`

Implement the memory-based FFT: complex first, block floating point included, inverse by conjugation. Test it alone against `golden/fx_stft.py` with impulses (every output bin is equal), single tones (one bin), and random input.
Check: bit-exact forward and inverse transforms; a forward transform followed by the inverse returns the input within the documented rounding error. **Effort:** 1–1.5 weeks.

Step 6.8 `stft_frontend`, `ola_backend`, `frame_ctrl`, `ise_top`

Add the ping-pong input buffer and window ROM, the synthesis window and overlap-add, and the frame scheduler with its per-phase cycle counters. Connect everything in `ise_top`.
Check: end-to-end bit-exact on the golden vectors and on a full VoiceBank utterance. The pre-roll behaves as specified, and the cycle counters report the frame schedule. **Effort:** 1 week.

Step 6.9 Resource log

Synthesize `ise_top` with Yosys (`stat`) and with Vivado out-of-context synthesis for the Zynq-7020. Record LUTs, flip-flops, DSP slices and block RAMs per block in `docs/resources.md`, with the commit hash.
Check: the design fits comfortably in the Zynq-7020, and block RAM use matches your state-memory calculation. If it does not, find out why: memories inferred as flip-flops are a common surprise. **Effort:** half a day, then 15 minutes per milestone.

6.6 Pitfalls

Pitfalls in Phase 2

- **Writing the whole design before testing anything.** Bugs multiply when blocks are combined. Test every block alone first.
- **Signed/unsigned mix-ups.** In SystemVerilog, one unsigned operand makes the whole expression unsigned. Slices of signed vectors are unsigned. Be explicit with `$signed()` and check with lint.
- **Off-by-one at the spectrum edges.** The 5-tap smoothing at bins 0, 1, $K-2$ and $K-1$

must zero-pad exactly like `np.convolve`.

- **Parameter updates mid-frame.** Any register the datapath reads must be shadowed and latched at the frame boundary. Otherwise one frame mixes old and new parameters, and you will hear it as a click.
- **Inferred latches and flip-flop memories.** Read the synthesis report after every major change.
- **Optimising early.** Get it bit-exact first, then make it smaller.

Read

Digital design fundamentals: Harris and Harris [26]. Architecture transformations for DSP (pipelining, folding, retiming): Parhi [14] and Kaeslin [43]. FPGA arithmetic: Meyer-Baese [44]. FFT hardware architectures: Garrido's survey [12]. Practical Verilog and formal-friendly design: the ZipCPU blog [45].

Phase 2 is done (with Phase 3) when

- ☐ Every module in Figure 6.1 exists, is lint-clean, and has its own passing test.
- ☐ `ise_top` is bit-exact with the golden model on all golden vectors and at least one full utterance.
- ☐ Cycle counters report the frame schedule; H4 can be evaluated from measured cycles.
- ☐ The resource log is up to date and the design fits the Zynq-7020.

Chapter 7

Phase 3 — Verification

[Weeks 7–12, alongside Phase 2] Milestone M2

In industry, verification typically takes more effort than design. That is not inefficiency. A bug found in simulation costs minutes. Found on the FPGA, it costs days. Found in silicon, it can cost a re-spin. This project has an unusual advantage that makes verification both rigorous and pleasant: a *bit-accurate golden model* written in Python. The question is never "does this look right?" but "are the integers identical?"

Why this matters for your project

Bit-exactness against the golden model is what makes your hardware results meaningful. Once the RTL is bit-exact, every quality number from Phase 1 (RQ1, RQ2) is also a statement about the hardware. And verification skills (cocotb, coverage, assertions, formal) are among the most employable things this project teaches.

Inputs	Outputs
Golden model and vectors (`spec-v1`); RTL blocks as they are written (Chapter 6); Verilator, Icarus and cocotb from Phase 0	A verification plan; one testbench per block plus a top-level test; a regression in CI; a coverage summary; formal proofs for small blocks

7.1 The verification plan

Write the plan before the tests, in `docs/verification_plan.md`. It lists every feature that must work, how it will be tested, and what counts as passing (Table 7.1). Review it at every milestone. A feature with no test is a feature you are hoping works.

7.2 How the testbenches are built

Every testbench has the same four parts, which mirror industrial methodologies such as UVM in a lightweight Python form:

- a **driver**, which turns test data into pin activity (valid/ready handshakes, samples, bus writes);
- a **monitor**, which watches outputs and turns them back into data;
- a **reference model**, which is your golden model, imported directly;
- a **scoreboard**, which compares expected and actual results and reports the *first* mismatch with enough context to debug it.

Table 7.1: An excerpt of a verification plan. Extend it to every block and feature.

ID	Feature	Test	Pass criterion
V1	`qmul_sat` rounding and saturation	Exhaustive (narrow widths) or 10^6 random vectors	0 mismatches against `fixedpoint.mul`
V2	`recip_unit`	Full input range or dense random	0 mismatches against the golden reciprocal
V3	`min_tracker` across sub-window ends	200 frames of golden vectors	$S_{\min}$ bit-exact every frame
V4	5-tap smoothing at spectrum edges	Impulse at bins 0, 1, $K-2$, $K-1$	Bit-exact with `np.convolve(mode="same")` semantics
V5	`fft_r2_mem` with block floating point	Impulse, tones, full-scale noise	Bit-exact data and exponent
V6	Parameter commit at a frame boundary	Commit in the middle of a frame	Old parameters until the boundary, new after; no mixed frame
V7	Pre-roll	First **PREROLL_FRAMES** frames	Pass-through, then seeded state equals golden
V8	End to end	Synthetic set + VoiceBank utterances	Bit-exact output; identical PESQ/STOI
V9	Deadline	Worst-case frame	**CYCLES_LAST** below budget; no `FRAME_OVERRUN`

Because cocotb testbenches are Python [19], the reference model is not a re-implementation. It is the same code that produced your Phase 1 results.

7.3 Steps

Step 7.1 Your first real testbench: `recip_unit`

The listing below is a complete pattern you will reuse for every pipelined block. Note the separate driver and monitor coroutines. The monitor samples in the `ReadOnly` phase after the clock edge, when all signals have settled.

```
import numpy as np
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import ClockCycles, ReadOnly, RisingEdge, with_timeout
from golden.recip import recip_model          # the same code golden/ uses

async def drive(dut, values):
    for v in values:
        await RisingEdge(dut.clk)
        dut.in_data.value = int(v)
        dut.in_valid.value = 1
    await RisingEdge(dut.clk)
    dut.in_valid.value = 0

async def monitor(dut, out, n):
    while len(out) < n:
        await RisingEdge(dut.clk)
        await ReadOnly()
        if dut.out_valid.value == 1:
            out.append(int(dut.out_data.value))

@cocotb.test()
async def recip_matches_golden(dut):
    cocotb.start_soon(Clock(dut.clk, 10, "ns").start())
```

```
    dut.rst.value, dut.in_valid.value = 1, 0
    await ClockCycles(dut.clk, 3)
    dut.rst.value = 0
    rng = np.random.default_rng(1234)
    stim = [1, 2, 3, 2**15, 2**16 - 1] + list(rng.integers(1, 2**16, 5000))
    got = []
    mon = cocotb.start_soon(monitor(dut, got, len(stim)))
    await drive(dut, stim)
    await with_timeout(mon, 100, "us")
    bad = [(v, recip_model(v), g) for v, g in zip(stim, got) if recip_model(v) != g]
    assert not bad, f"{len(bad)} mismatches; first (in, expected, got): {bad[:5]}"
```

Check: the test passes on Verilator and on Icarus. Then deliberately break the RTL (flip a rounding constant) and confirm the test fails with a useful message. A test that cannot fail is not a test. **Effort:** 1 day (with the RTL).

Step 7.2 Golden-vector plumbing

Write `tb/common/vectors.py` to load the files exported by `golden/vectors.py`: per test signal, per probe point, per frame, as integer arrays. Add a scoreboard helper that compares an RTL probe stream with its golden array. On the first mismatch it reports the frame, the bin, the expected and actual values, and the five previous values.
Check: the helper reports a deliberately injected mismatch at the right frame and bin. **Effort:** half a day.

Step 7.3 A streaming testbench for `bin_pipeline`

Drive one bin per cycle from the golden spectra and compare the outputs, and the internal probes exposed through debug ports, frame by frame. Add random *back-pressure*: hold `out_ready` low on random cycles. That checks the pipeline stalls without losing or duplicating a bin, if your design has back-pressure at all. If it is purely schedule-driven, test the schedule instead.
Check: bit-exact on all synthetic signals and at least one utterance, including a parameter commit. **Effort:** 2–3 days.

Step 7.4 Frame-level test of `ise_top` with real audio

Feed a full VoiceBank utterance through `ise_top` and collect the output samples. Compare them bit for bit with the golden model. Then compute PESQ, STOI and SI-SDR on the RTL output. They must equal the golden model's numbers exactly, and that equality is the evidence connecting Phase 1 to the hardware.
Check: bit-exact output; identical metrics. Note the simulation time per second of audio for your report. **Effort:** 2 days.

Keep Python out of the per-cycle loop

A full utterance is millions of clock cycles. A cocotb coroutine that wakes on every clock edge makes the simulation slow. Wake on *events* instead: a sample strobe, a `frame_done` pulse, or a FIFO becoming non-empty. Let the simulator run the cycles in between at full speed.

Step 7.5 Directed corner cases

Real speech rarely exercises extremes. Write one directed test per item:

- silence: all zeros, which tests division by tiny noise estimates and the ε floors;

- a full-scale sine, which tests saturation in the FFT and the power computation;
- a clipped square wave and DC, which test bin 0 and the block exponent;
- an impulse and a chirp;
- a parameter commit in mid-frame (V6), two commits in quick succession, and a commit during pre-roll;
- a soft reset in the middle of the stream, then re-seeding;
- a bus read and write during frame processing.

Check: every item is in the verification plan with a passing test. **Effort:** 2–3 days.

Step 7.6 Constrained-random tests

Generate random but *legal* stimuli: noise at random levels and spectral shapes, random parameter sequences within the register ranges, and commits at random times. Seed every run and print the seed, so any failure can be reproduced exactly.
Check: 100 random seeds pass. Any failure becomes a new directed test before it is fixed. **Effort:** 1–2 days.

Step 7.7 Measure coverage

- **Code coverage.** Build with Verilator's `-coverage` option and summarise with `verilator_coverage` [46]. Look for lines and toggles never exercised.
- **Functional coverage.** Count events that matter: every gain-table index used; saturation in each block; each number of block-floating-point shifts; commits at and away from frame boundaries; tracker sub-window ends. Use simple counters in Python, or the `cocotb-coverage` package.

Check: every un-hit item is either covered by a new test or explained in the plan (for example, as unreachable by design). **Effort:** 1–2 days.

Step 7.8 Assertions and formal proofs for small blocks

Add immediate assertions inside the RTL for invariants. Verilator checks them in simulation when built with assertions enabled:

```
`ifdef FORMAL_OR_SIM_CHECKS
  always_comb begin
    assert (s_min <= running_d);
    for (int i = 0; i < U; i++) assert (s_min <= store_d[i]);
  end
`endif
```

For small blocks such as `min_tracker`, the AXI-Lite register file and any FIFO, go further: *prove* the properties for all inputs with SymbiYosys, the formal front end in the OSS CAD Suite [47]:

Listing 7.1: `tb/formal/min_tracker.sby` (adapt to the current SymbiYosys documentation).

```
[options]
mode prove

[engines]
smtbmc

[script]
read -formal -D FORMAL_OR_SIM_CHECKS min_tracker.sv
prep -top min_tracker
```

```
[files]
../../rtl/min_tracker.sv
```

Check: the proof passes. A broken variant, such as one using the wrong comparison, produces a counterexample trace. **Effort:** 2 days, and it is worth it: formal skills stand out on a CV.

Step 7.9 Regression in CI

Add a CI job that installs Verilator (the OSS CAD Suite is convenient for this), runs every cocotb test and the formal proofs, and archives the coverage summary. Keep the full-utterance tests in a nightly or manual job if they are slow.
Check: every push runs the fast regression. The badge is green at M2. **Effort:** half a day.

7.4 When the RTL and the model disagree

Mismatches are where you learn the most. Debug them systematically:

1. **Find the first divergence.** Compare the internal probes block by block, in pipeline order. The first probe that differs, in the earliest frame and lowest bin, is where the bug is, or just upstream of it.
2. **Shrink the case.** Reproduce it with the shortest input that fails, often a single frame of a synthetic signal.
3. **Look at waves.** Dump an FST trace for just that window, with Verilator's tracing options, and inspect it in GTKWave or Surfer next to the golden values.
4. **Decide who is wrong.** Sometimes it is the model. A float slipped into a fixed-point path, or the model rounds differently from the specification. Fix the specification first, then whichever side disagrees with it.
5. **Add a regression test** for the bug before fixing it.

Pitfalls in Phase 3

- **Comparing with tolerances.** "Within 1 LSB" hides systematic rounding bugs that add up over frames. Require exact equality.
- **Testing only with speech.** Speech never saturates the FFT or produces exact half-LSB ties. Synthetic and random signals do.
- **Reading outputs at the wrong time.** Sample after the edge, in `ReadOnly`. Drive on the edge, never in `ReadOnly`.
- **Unseeded randomness.** An irreproducible failure is almost worthless.
- **Verification as an afterthought.** Each block's test is written together with the block. A week of untested RTL is a week of hidden bugs.

Read

cocotb documentation and examples [34]. Verilator user's guide, for lint, tracing and coverage [46]. SymbiYosys [47]. For verification methodology in SystemVerilog, useful even though you test in Python: Spear and Tumbush [48]. Formal verification of real designs, step by step: the ZipCPU blog [45].

Milestone M2 is reached (with Phase 2) when

- ☐ The verification plan covers every block and feature, and every entry has a passing test.
- ☐ `ise_top` is bit-exact with the golden model on all golden vectors and full utterances, with identical metrics.
- ☐ Directed corner cases and 100 constrained-random seeds pass.
- ☐ Coverage is measured and every gap is closed or explained.
- ☐ Formal proofs pass for at least `min_tracker` and the register file.
- ☐ The regression runs in CI and is green.

Chapter 8

Phase 4 — Real-Time FPGA Prototype on the PYNQ-Z2

[Weeks 12–16] Milestone M3

Simulation proves the design is correct; the FPGA proves it is *real*. In this phase your RTL processes live audio in real time, and typed complaints change what you hear by writing hardware registers. That is your thesis architecture on a single chip, with real numbers for latency, resources and power.

Why this matters for your project

Phase 4 turns RQ2 and RQ3 from estimates into measurements: end-to-end latency through real converters, resource use on a real device, and measured cycles per frame (H4). It is also the demo: a 60-second video of the system working is the single most persuasive artefact in your portfolio.

Inputs	Outputs
M2: bit-exact, verified RTL; register map (appendix B); a Vivado host (Chapter 4); the hardware listed below	Scripted Vivado project in `fpga/pynq_z2/`; bitstream and overlay; a Python driver; a hardware-in-the-loop bit-exact test; latency, resource and power measurements; demo video

8.1 Hardware and host

- **TUL PYNQ-Z2 board.** A Zynq XC7Z020 SoC (a dual-core ARM Cortex-A9 processing system plus FPGA fabric), 512 MB DDR3, and an Analog Devices ADAU1761 audio codec with headphone and line/microphone jacks. It is supported by the PYNQ framework [49], [50], which runs Linux and Jupyter on the ARM and loads FPGA designs ("overlays") from Python.
- A microSD card (16 GB or more), a USB cable, an Ethernet cable, headphones, and a line-level audio source.
- Optional: your thesis's RME Fireface for loopback latency measurements, and a USB power meter.
- **Host:** Vivado on an x86-64 Linux or Windows machine (Chapter 4). Use the Vivado version that your PYNQ image release names in its documentation, so the board files and drivers match.

Concept: Processing system and programmable logic

A Zynq has two halves. The *processing system* (PS) is a dual-core ARM running Linux, with the DDR memory. The *programmable logic* (PL) is FPGA fabric where your RTL lives. They communicate over AXI buses:

- AXI4-Lite for registers, so your `regs_axil` appears as a block of memory addresses to Python;
- AXI4-Stream plus a DMA engine for bulk data.

This split matches your architecture: control plane on the PS, data plane in the PL [51].

8.2 Steps

Step 8.1 Bring up the board and prove the audio path

Flash the PYNQ image for the PYNQ-Z2 to the microSD card, boot, and open Jupyter from your browser. Run the base overlay's audio example: record a few seconds and play them back through the headphones.
This proves the codec, cables and jacks before any of your own hardware is involved. Read how the base overlay's Python audio driver configures the ADAU1761. You will reuse that configuration sequence.
Check: you hear your own recording. Note the PYNQ image version and the Vivado version it requires in your notebook. **Effort:** half a day.

Step 8.2 Hardware-in-the-loop first: `ise_top` behind an AXI DMA

Before touching real-time audio, get your design on the chip in the simplest possible system:

- the Zynq PS;
- an AXI DMA engine that streams samples from DDR into `ise_top` and streams results back;
- `regs_axil` on AXI4-Lite.

Wrap `ise_top` in a thin adapter with AXI4-Stream input and output, and package it as IP.
Create the whole Vivado project from a Tcl script in `fpga/pynq_z2/build.tcl`, never by clicking only, so it is version controlled and reproducible. A good habit is to build the block design once in the GUI, then export it with `write_bd_tcl` and commit the script.

Listing 8.1: Hardware-in-the-loop test from a PYNQ notebook (API names as in current PYNQ; check your version).

```
import numpy as np
from pynq import Overlay, allocate

ol = Overlay("ise_hil.bit")                    # loads bitstream + .hwh
dma = ol.axi_dma_0
x = np.load("golden/vectors/utt01_input.npy").astype(np.int16)
y_ref = np.load("golden/vectors/utt01_output.npy").astype(np.int16)

src = allocate(shape=x.shape, dtype=np.int16); src[:] = x
dst = allocate(shape=y_ref.shape, dtype=np.int16)
dma.recvchannel.transfer(dst); dma.sendchannel.transfer(src)
dma.sendchannel.wait(); dma.recvchannel.wait()
print("bit-exact:", np.array_equal(np.asarray(dst), y_ref))
```

Check: the FPGA output is bit-exact with the golden model, the same result as in simulation, now on silicon. **Effort:** 3–5 days. The first Vivado project always takes longer than expected.

Step 8.3 Timing closure and utilisation

Run implementation at your chosen PL clock (100 MHz from the PS is a convenient start), then read two reports:

- `report_timing_summary`: worst negative slack (WNS) must be ≥ 0;
- `report_utilization`: LUTs, flip-flops, DSP slices and block RAM tiles, both per block and in total.

Compare block RAM use with your state-memory calculation from Chapter 3.
Check: timing is met with margin. Utilisation is recorded in `docs/resources.md`. If timing fails, add one pipeline register on the failing path. Your design only needs a few MHz of real throughput, so it does not need to run at the top clock speed. **Effort:** 1 day.

Step 8.4 Real-time audio: I2S to the codec

Now replace the DMA path with the codec. You need three things:

1. **Codec configuration.** Send the ADAU1761 register sequence over I2C from the PS. Reuse what the base overlay's driver does, together with the codec datasheet, to set the sample rate (48 kHz, or the rate chosen in Phase 1), the I2S format and the signal routing.
2. **I2S receiver and transmitter** in the PL (`rtl/i2s_rx.sv`, `rtl/i2s_tx.sv`). They deserialise and serialise samples using the bit clock and word-select (LRCLK) signals.
3. **Clock-domain crossing.** The I2S clocks are not your system clock. Bring samples across through a small asynchronous FIFO, or capture each sample in the I2S domain and pass a synchronised strobe (a two-flip-flop synchroniser) with the data held stable. Never sample a multi-bit bus directly across clock domains.

Check: with `CTRL.BYPASS` set, audio passes through unaltered, apart from delay. With `BYPASS` clear and default parameters, you hear noise reduction. With β raised through the registers, you hear stronger suppression, as in the thesis. **Effort:** 1–1.5 weeks.

Step 8.5 Close the loop: complaints control the hardware

Write a small Python service on the PYNQ that receives parameter updates and applies them with the driver in appendix B. Then choose where the NLP runs:

Option A (recommended first): NLP on your laptop
Run the thesis pipeline unchanged: mpnet, SVM and Ridge, and the parameter mapping. Send the resulting parameters to the board over the network. This mirrors a real product architecture, where language processing runs on a paired phone and the hearing aid receives only parameters.

Option B (stretch): NLP on the board
Run a small encoder on the Cortex-A9. Be realistic: 512 MB of RAM rules out fp32 mpnet, and many ML runtimes ship only 64-bit ARM builds while the Zynq-7000 is 32-bit. Check the availability of your runtime before you commit to this. If it works, measure the latency per sentence. This feeds directly into Track A (Chapter 9).

Check: typing "the background noise is unbearable" audibly increases suppression within a fraction of a second, with no clicks at the switch. **Effort:** 2–3 days.

Step 8.6 Measure latency, cycles and power

Latency
Use the thesis's RME Fireface. Play a chirp or click into the board's line input while recording both the source and the board's output on two Fireface inputs. The lag of the cross-correlation peak is the end-to-end latency. Then measure again in `BYPASS` mode. The difference is your algorithm and datapath; the bypass value is the converters and I2S buffering. Compare with the algorithmic latency predicted for your frame configuration.

Cycles
Read **CYCLES_LAST** over many frames. Its maximum, against the frame budget, is your measured answer to H4.

Power
Report Vivado's `report_power` estimate for the PL (static and dynamic), ideally using switching activity from a simulation of real audio rather than the default activity rates. Then measure board power with a USB meter, with `CTRL.ENABLE` on and off. Expect the board total to be dominated by the ARM, the DDR memory and the regulators. That is exactly why the Vivado PL estimate and energy per frame, $E_{\text{frame}} = P_{\text{dyn}}\, T_{\text{frame}}$, are the meaningful numbers.

Measurement	Value	Method	Notes
End-to-end latency (processing)		loopback cross-correlation	
End-to-end latency (bypass)		loopback cross-correlation	
Max cycles per frame / budget		**CYCLES_LAST**	H4
LUT / FF / DSP / BRAM		`report_utilization`	
PL dynamic / static power		`report_power`	activity source?
Board power, enabled / disabled		USB meter	
Energy per frame		$P_{\text{dyn}} T_{\text{frame}}$	

Check: the table is filled in, every value has its method and conditions in the notebook, and the latency matches your prediction within a few milliseconds of converter delay. **Effort:** 2–3 days.

Step 8.7 Record the demo

Script a 60–90 second video:

1. noisy speech playing;
2. type a complaint; the parameters change;
3. the audio audibly improves;
4. type "that's perfect now" (`BALANCED`) and the parameters decay back towards their defaults;
5. finish on the resource and latency numbers on screen.

Record with good audio: use the Fireface, not a phone microphone next to headphones.
Check: someone who has never seen the project understands it from the video alone. **Effort:** 1 day.

8.3 Pitfalls

Pitfalls in Phase 4

- **Vivado and PYNQ versions that do not match.** Board files and overlay metadata differ between releases. Use the documented pairing.
- **Starting with real-time audio.** Get the bit-exact DMA test working first. Then any problem in real time is in the I2S, clocking or codec setup, not in your datapath.
- **Clock-domain crossing by accident.** Every signal crossing between the I2S and system clocks needs a deliberate synchroniser or FIFO. Such bugs appear intermittently and are miserable to find.

- **Codec configuration.** A wrong I2S word length or a missed routing register gives silence or noise. Compare your register sequence with the working base overlay.
- **Address map surprises.** Check the address Vivado assigned to `regs_axil` and read **VERSION** first as a sanity check.
- **Comparing board power to a hearing aid.** The PYNQ board is not a low-power platform. Report PL estimates and energy per frame, and explain the difference (Chapter 11).

Read

PYNQ documentation [50] and its support forum [52]. *The Zynq Book*, for the PS/PL architecture and AXI [51]. FPGA DSP arithmetic and resource mapping: Meyer-Baese [44].

Milestone M3 is reached when

- ☐ The Vivado project is rebuilt from a committed Tcl script.
- ☐ The hardware-in-the-loop test is bit-exact on the FPGA.
- ☐ Timing is met; utilisation is recorded.
- ☐ Live audio is enhanced in real time; bypass works; parameter changes are click-free.
- ☐ Typed complaints control the hardware (option A at least).
- ☐ The latency, cycle, resource and power table is filled in with methods.
- ☐ The demo video is recorded.

Chapter 9

Phase 5A — ML Acceleration Track

[Weeks 17–22] Milestone M4 (Track A)

Track A brings the *control plane* into hardware. Your thesis found that the six intent classes are linearly separable in the sentence-embedding space, which means the classifier and severity heads are tiny matrix–vector products. The expensive part is the encoder that produces the embedding. This track builds a small, verified int8 accelerator for the heads, then attacks the encoder with one of two research routes.

Why this matters for your project

This track answers RQ4 and completes H3. It also teaches the core of ML hardware: quantisation, multiply–accumulate arrays, dataflows, memory bandwidth, and the roofline view of performance. These are the same concepts used in commercial neural-network accelerators [22], [27], practised at a scale you can finish in six weeks.

Inputs	Outputs
M3 (working FPGA system); Phase 1 head quantisation results (H3); the thesis NLP pipeline and dataset; cocotb flow	`head_engine` RTL, verified and integrated; an encoder or audio-intent study with an accuracy–cost trade-off curve; cycles, latency, memory and energy-estimate tables

9.1 Characterise the control-plane workload

Start from Table 3.8. Encoder compute is about 2.4 GFLOP per sentence for mpnet and about 0.3 GFLOP for MiniLM (estimates). The heads need a few thousand multiply–accumulates. Two consequences shape this track:

- **Throughput is irrelevant; latency and energy per inference matter.** A user complains at most every few seconds. A few hundred milliseconds per sentence feels instant. What matters is how much energy each inference costs, and whether the model fits in on-chip memory.
- **Memory bandwidth dominates large models.** The Zynq-7020 has roughly 630 KB of block RAM. Even int8 MiniLM (about 23 MB) cannot live on chip, so its weights must stream from DDR for every sentence. The roofline model [25] makes this precise. Performance is bounded by $\min(\text{peak compute},\ \text{bandwidth} \times \text{arithmetic intensity})$. A matrix–vector product reuses each weight once, about one MAC per weight byte at int8, so it is bandwidth-bound. Processing all of a sentence's roughly 14 tokens together reuses each weight 14 times.

Concept: Int8 affine quantisation

A real value r is represented by an integer q via $r \approx s\,(q - z)$, with a scale s and a zero point z [24]. For weights, symmetric quantisation ($z = 0$) with one scale per output row is standard. A dot product becomes an int8×int8 product accumulated in int32, then rescaled once: $\sum_i r_{w,i} r_{x,i} \approx s_w s_x \sum_i q_{w,i} q_{x,i}$. The rescale can itself be a fixed-point multiply and shift, so inference is integer-only.

9.2 Route A (required): the `head_engine`

Step 9.1 Choose the head and freeze its integer model

From your Phase 1 results (Chapter 5), choose between logistic regression (6 rows) and the one-vs-one SVM (15 rows plus voting). Unless the SVM is clearly more accurate after quantisation, choose logistic regression: it has 2.5× fewer MACs and a simple margin confidence (O9). Fold the Ridge severity weights in as a seventh row, giving one 7×768 int8 matrix of 5.25 KiB.
Write `nlp_hw/head_model.py`: a bit-exact integer model that takes the same int8 inputs as the hardware and returns the class, a margin and a severity in Q0.15 (clipped to $[0, 1]$). This is the head's golden model.
Check: on the thesis test split, the integer model's macro-F1 and MAE equal your Phase 1 int8 results. **Effort:** 1 day.

Step 9.2 Design and build the engine

A simple, effective design:

- **Weight memory.** One block RAM, P int8 values wide. Load it once from the ARM over AXI.
- **Input buffer.** 768 int8 values, written by the ARM per sentence.
- **Datapath.** A `mac_array` of P lanes ($P = 8$ or 16). Each cycle it multiplies P weights by P inputs and adds them, with an adder tree, into one int32 accumulator for the current row.
- **Post-processing.** After each row: add the bias, rescale with a fixed-point multiplier, and track the running argmax and the second-best score for the margin.

Cycles per inference $\approx 7 \times 768/P$: 672 cycles for $P = 8$, which is 6.7 µs at 100 MHz. Expose **start**, **done**, **class**, **margin** and **severity** registers.
Check: cocotb against `nlp_hw/head_model.py` on all 1106 sentences: identical class, margin and severity. Lint-clean; resource log updated. **Effort:** 1–1.5 weeks.

Step 9.3 Integrate on the PYNQ

Add `head_engine` to the FPGA design next to `ise_top`. The ARM, or your laptop in option A, now sends only the 768-value embedding. The hardware returns class and severity, and the parameter mapping writes the enhancer registers.
Check: end-to-end, typed complaint to audible change, now with the heads in hardware. Measured latency of the head is negligible next to the encoder. Say so in your report, with numbers. **Effort:** 2–3 days.

9.3 Then choose one research route

Route B — compress the text encoder
Keep typed or recognised text, and make the encoder small enough for edge hardware. Lower risk: everything can be measured on a laptop, and the hardware deliverable is a matrix-multiply engine.

Route C — recognise intent directly from audio
Replace text plus encoder with a small network that classifies spoken complaints from audio features computed by *your own* STFT. This is closer to a real hearing aid and removes the speech-recognition step the thesis listed as a limitation. It is also more data-hungry and riskier.

9.3.1 Route B steps

Step 9.4 Swap and quantise the encoder

Re-embed the thesis dataset with `all-MiniLM-L6-v2` [30], retrain the heads on the thesis split, and measure macro-F1. Apply dynamic int8 quantisation to the encoder's linear layers, with PyTorch or ONNX Runtime, and measure again. Integer-only transformer inference is possible [53], [54], but start with the easy wins.
Check: a table of encoder, precision, macro-F1, MAE, parameter count, model size and CPU latency per sentence. This is the start of your RQ4 trade-off curve. **Effort:** 2–3 days.

Step 9.5 Distil a tiny domain-specific student

Your domain is narrow: complaints about hearing-aid sound. Train a small student, for example 2–4 transformer layers of width 128, or even a small CNN over word embeddings, to *reproduce the mpnet embeddings* (an MSE or cosine loss on the embedding). Use a large set of unlabelled, in-domain sentences, generated in the same way as the thesis dataset. Then put the unchanged thesis heads on top.
A domain vocabulary of a few thousand words keeps the embedding table small enough for on-chip memory. That is a co-design decision worth reporting.
Check: a curve of macro-F1 against parameter count, from mpnet through MiniLM down to your students. Note where it falls below the RQ4 target. **Effort:** 1–1.5 weeks.

Step 9.6 A matrix-multiply engine for the student

Map the student's matrix multiplies onto a small *weight-stationary systolic array* [21], [23]. For example, a 4×4 or 8×8 grid of int8 MAC cells: weights stay in the cells, and activations flow through. Implement *one tile* in RTL, verify it with cocotb against numpy, and estimate cycles for the whole student from the tile's throughput and the layer shapes.
Tool-based alternatives: hls4ml [18], [55] and FINN with Brevitas [56], [57]. Check their current support for the layer types you use, especially attention, before relying on them.
Check: the tile is bit-exact. The cycle and memory estimate for the full student is backed by the tile measurement. **Effort:** 1.5 weeks.

9.3.2 Route C steps

Step 9.7 Build a spoken-intent dataset

No spoken hearing-aid-complaint dataset exists, so you will build one:

1. synthesise the 1106 complaint sentences with several text-to-speech voices;
2. mix them with noise at several SNRs, for example DEMAND noises as in Voice-Bank+DEMAND;
3. record a small *real* test set from volunteers, with informed consent (Chapter 11).

Consider pre-training on a public spoken-language-understanding dataset such as Fluent Speech Commands [58], or on keyword spotting with Speech Commands [59].
Check: train, development and test splits with no speaker or sentence overlap, and a real-speech test set kept separate. **Effort:** 1 week.

Step 9.8 A tiny audio-intent model on your own features

Compute log-mel features from the STFT magnitudes your enhancer already produces, with the mel filterbank as a sparse ROM. Note the tension: the enhancer's short low-latency frames give coarse frequency resolution. Test whether that is enough, or whether a separate longer analysis window is needed.
Train a depthwise-separable CNN of the kind used for keyword spotting [60], with tens of thousands of parameters rather than millions. Use quantisation-aware training (for example Brevitas), and deploy it with hls4ml or FINN, or with your own MAC array.
Check: accuracy against SNR on synthetic speech, and separately on real recordings. Report the gap honestly; generalising from TTS to real voices is the main risk. **Effort:** 2 weeks.

9.4 Reporting

Model	Params	Precision	macro-F1	MAE	Latency (cycles)	Weights
mpnet + SVM (thesis)	110 M	fp32	0.957	0.100	laptop	440 MB
mpnet + LR, int8 head (`head_engine`)						
MiniLM + LR						
MiniLM int8 + LR						
Your student + LR						
Audio DS-CNN (Route C)						

To estimate energy, use published figures for the energy of each arithmetic operation and each memory access, for example those collected by Horowitz [15]. State clearly that they are 45 nm estimates, not measurements, and use them only to compare designs with each other.

Pitfalls in Track A

- **Optimising the heads too much.** They are not the bottleneck. The `head_engine` is a learning vehicle and a verified building block; the research question is in the encoder or the audio model.
- **Leaking the test split.** Distillation and TTS data must never contain test sentences.
- **Mismatched quantisation between training and hardware.** If your quantised model and your RTL round differently, F1 in Python means nothing. Keep a bit-exact integer model, as in Phase 1.
- **Counting FLOPs and ignoring memory.** For batch-1 inference, moving weights usually

costs more time and energy than arithmetic.

Read

The definitive tutorial on efficient DNN hardware: Sze et al. [27], and the book [61]. MIT's *Hardware Architectures for Deep Learning* course materials [62]. Quantisation methods: Gholami et al. [63]. Keyword spotting on microcontrollers: Zhang et al. [60]. Systolic arrays: Kung [21].

Milestone M4 (Track A) is reached when

- ☐ `head_engine` is bit-exact on all sentences, integrated, and measured on the FPGA.
- ☐ One route (B or C) has a complete accuracy–cost trade-off curve.
- ☐ One hardware block for that route (matrix tile, or the audio model's accelerator) is verified.
- ☐ RQ4 is answered in the results table, with estimates clearly marked.

Chapter 10

Phase 5B — VLSI and ASIC Track

[Weeks 17–22] Milestone M4 (Track B)

Track B takes part of your verified RTL through a complete open-source *RTL-to-GDS* flow: synthesis to standard cells, floorplanning, placement, clock-tree synthesis, routing, timing sign-off and physical verification. The result is a layout that could be manufactured, with area, timing and power from a real 130 nm process. Optionally, a small block goes to an actual shuttle via Tiny Tapeout.

Why this matters for your project

FPGA numbers say little about a chip. LUTs are not gates, and FPGA power is dominated by the programmable fabric. This track answers RQ3 in the units chip designers use: square millimetres, nanoseconds of slack, and microjoules per frame. It also gives you hands-on experience of every step of a digital IC back end, which is the core of digital VLSI roles.

Inputs	Outputs
M3: verified, FPGA-proven RTL; the cocotb regression; a Linux machine (or macOS) where you can install Nix or Docker	LibreLane configuration in `asic/`; synthesis area sweeps; a DRC/LVS-clean layout of the chosen block; timing and power reports, with power from real switching activity; an FPGA versus ASIC comparison

10.1 The flow

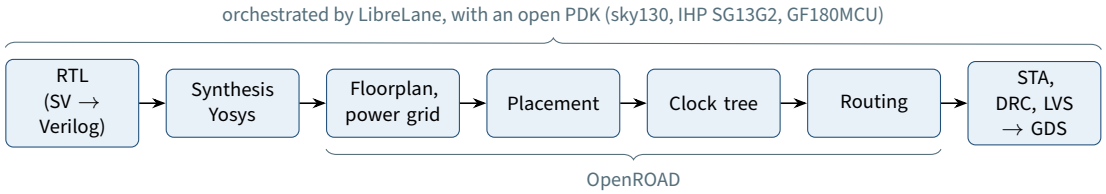


Figure 10.1: The open-source RTL-to-GDS flow.

LibreLane [64], [65], the successor of OpenLane 2, runs the whole flow with a single command. The steps are Yosys for synthesis [33], OpenROAD for floorplanning, placement, clock tree, routing and static timing analysis [66], and Magic, KLayout and Netgen for physical verification. It works with several open *process design kits* (PDKs), which describe a real manufacturing process (Table 10.1).

Table 10.1: Open PDKs you can use. Check each PDK's current documentation for details such as available SRAM macros and library corners.

PDK	Notes
SkyWater SKY130 [67]	130 nm. The most widely documented open PDK, with many examples and tutorials. Used by Tiny Tapeout.
IHP SG13G2 [68]	130 nm BiCMOS from IHP (Germany). Well integrated with LibreLane, and also offered on Tiny Tapeout shuttles.
GF180MCU	180 nm from GlobalFoundries. Larger and slower, but also supported.

Concept: What you are measuring

- **Area**: the standard-cell area and the core area after placement. Utilisation is their ratio.
- **Timing**: setup and hold slack at your target clock, from static timing analysis with real wire parasitics after routing. Positive slack means the design meets timing.
- **Power**: dynamic power, $\alpha C V^2 f$ summed over all nets (Chapter 2), plus leakage. Dynamic power depends on the *switching activity* α. With default activity the number is a guess; with activity from simulating real audio it is an estimate worth reporting.

10.2 What to harden

Do not try to harden the whole `ise_top`. Its state memory (tens of kilobytes) would be built from flip-flops unless you use SRAM macros, and that would dominate the area and runtime and teach you little. Choose instead:

- **`bin_pipeline`**, with the state memory left *outside*: expose its read and write ports as pins. This hardens the arithmetic, which is the interesting part for RQ1 and RQ3.
- and/or **`head_engine`** (Chapter 9) with a small weight memory.

For the memory itself, estimate area from an SRAM macro of the chosen PDK, or from a memory compiler such as OpenRAM [69], and state which you used. Check what each PDK currently provides.

10.3 Steps

Step 10.1 Install LibreLane and run the example

Follow the LibreLane documentation. The recommended installation uses Nix: install Nix, clone the LibreLane repository, enter its shell, then run the smoke test and the bundled example. Docker is an alternative.

```
git clone https://github.com/librelane/librelane
nix-shell librelane/shell.nix        # or the current command from the docs
librelane --smoke-test
librelane --run-example spm          # a small serial-parallel multiplier
```

The PDK is downloaded automatically on first use.
Check: the example finishes and you can open its final layout in KLayout. Find the timing, area and power reports in the run directory and read them. **Effort:** half a day to a day.

Step 10.2 Make your RTL digestible for the flow

Yosys's built-in SystemVerilog support covers a subset of the language. Test your block with `yosys -p "read_verilog -sv ...; synth -top ..."`. If it fails, convert to Verilog with sv2v, or use a full SystemVerilog front end plugin for Yosys. Re-run your cocotb regression on the converted Verilog: if the conversion changed behaviour, you want to know now.
Check: the converted design passes the same tests as the original. **Effort:** half a day to two days, depending on the constructs you used.

Step 10.3 Synthesis-only design-space exploration

Before running the full flow, use fast synthesis runs to answer "how much area does each bit cost?" Synthesize `bin_pipeline` against the PDK's standard-cell library for several word-length configurations from Phase 1, and record the cell area:

```
yosys -p "read_verilog bin_pipeline.v; synth -top bin_pipeline; \
          dfflibmap -liberty $LIB; abc -liberty $LIB; stat -liberty $LIB"
```

Plot area against word length next to the Phase 1 plot of quality against word length. Together they show the price of each bit of quality, which is the heart of RQ1 and RQ3.
Check: a table and plot of area for at least four configurations. **Effort:** 1–2 days.

Step 10.4 Configure and run the full flow

Create `asic/bin_pipeline/config.json` (JSON or YAML; see the LibreLane configuration reference for current variable names):

```
{
  "DESIGN_NAME": "bin_pipeline",
  "VERILOG_FILES": "dir::src/*.v",
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 50
}
```

Start with a relaxed clock (50 ns, i.e. 20 MHz). Your design needs only a few MHz, so you are learning the flow, not chasing frequency. Run it (`librelane config.json`, adding `--pdk` for a non-default PDK), then read the reports.
Check: the flow completes with no DRC or LVS errors and positive setup and hold slack. Record the core area, the utilisation and the critical path. **Effort:** 2–4 days, including the first failures.

Step 10.5 Power from real switching activity

Default activity factors make power numbers meaningless for a design whose datapath is idle most of the frame. Instead:

1. simulate the synthesized gate-level netlist, using the PDK's cell models, on a few frames of real audio from your golden vectors, and dump a VCD;
2. load the final netlist, its parasitics and the VCD into OpenSTA (part of OpenROAD), and report power.

Convert to *energy per frame*: $E_{\text{frame}} = P \times T_{\text{frame}}$, with P averaged over whole frames, idle time included.
Check: you can explain the split between dynamic and leakage power, and why the idle part of each frame matters. Repeat with default activity and report how different the two numbers are.
Effort: 2–3 days.

Step 10.6 Clock, parallelism and voltage

Use your synthesis and power results to explore the classic low-power trade-off [16]:

- a folded design at a higher clock versus a parallel design at a lower clock, compared on area and on energy per frame;
- the effect of supply voltage, where the PDK provides library corners at more than one voltage. Open PDKs offer only a few corners. Where a voltage is missing, use the first-order $E \propto V^2$ scaling and say so clearly.

Check: a small table that ranks at least two architectures on energy per frame, with the method stated. **Effort:** 2 days.

Step 10.7 Compare FPGA and ASIC

Put the FPGA numbers from Chapter 8 next to the ASIC numbers: resources against area, PL power against ASIC power, and energy per frame. Explain the differences in a paragraph: programmable routing and look-up tables against fixed standard cells. This comparison is one of the most useful paragraphs in your report.
Effort: 1 day.

Step 10.8 Optional: tape out a small block with Tiny Tapeout

Tiny Tapeout [70] pools many small designs on one shared chip. A design occupies one or more *tiles* (a single tile is on the order of $160\,\mu\text{m} \times 100\,\mu\text{m}$ in sky130) and has a small fixed set of input, output and bidirectional pins. That is far too small for the datapath with its memory, but enough for a single unit: the `recip_unit`, the `qmul_sat` plus gain-table interpolation, or a slice of the `head_engine`'s MAC array with a serial interface.
The process uses a template repository whose GitHub Actions run the hardening flow for you. Check current shuttles, deadlines, tile sizes, pin limits and costs on the Tiny Tapeout website, as they change between rounds.
Check: your block hardens in the template's CI and passes its gate-level test. Submit it if the timing and budget work for you. **Effort:** 1 week.

10.4 Pitfalls

Pitfalls in Track B

- **Hardening memory as flip-flops by accident.** Check the cell report. Thousands of flip-flops means a RAM became registers.
- **Chasing frequency.** You need megahertz, not hundreds of megahertz. Use a relaxed clock and spend the time on power and area.
- **Reporting power from default activity.** Use real activity, or label the number clearly as a rough estimate.
- **Hold violations after clock-tree synthesis.** These are normal in early runs. The flow inserts hold buffers, but read why they appear.
- **Over-claiming.** 130 nm open-PDK numbers are not hearing-aid numbers. Commercial hearing-aid chips use smaller, low-leakage processes, custom memories and aggressive power gating. Report your results as relative comparisons and trends.

Read

CMOS VLSI design: Weste and Harris [71]; Rabaey et al. [72]. Low-power design: Chandrakasan et al. [16]. A structured, hands-on course on the open-source ASIC flow: Zero to ASIC [73]. LibreLane documentation [65]; OpenROAD [66].

Milestone M4 (Track B) is reached when

- ☐ The chosen block passes the full flow with no DRC or LVS errors and positive slack.
- ☐ The area-versus-word-length sweep is complete and plotted next to the Phase 1 quality sweep.
- ☐ Power and energy per frame come from real switching activity.
- ☐ At least two architectures are compared on energy per frame.
- ☐ The FPGA versus ASIC comparison is written.

Part III

Doing It Well

Chapter 11

Evaluation Methodology and Research Rigour

Your thesis already showed good instincts: stratified cross-validation, a test set touched exactly once, and ties broken on generalisation. This chapter carries the same discipline into the hardware project. It covers what to measure, what to compare against, how to tell a real difference from noise, and how to make every number reproducible.

11.1 What to measure

Table 11.1: Metric catalogue. Use the same definitions everywhere in the project.

Group	Metric	Notes
Speech quality	PESQ (wideband) [38], STOI [39], SI-SDR [40], ΔSNR	Against the clean reference, after alignment; per utterance; PESQ at 16 kHz
Implementation fidelity	Bit-exactness against golden; output SNR of fixed point against float	Separates "the hardware is wrong" from "the precision is too low"
Intent	Macro-F1, per-class F1, severity MAE	Thesis split; test set used once
Timing	Cycles per frame (max and mean), algorithmic and end-to-end latency (ms)	Measured, not estimated, by M3
FPGA cost	LUT, FF, DSP, BRAM; WNS at the target clock	From Vivado reports
ASIC cost	Cell and core area (mm^2), setup and hold slack	From LibreLane reports
Power and energy	Dynamic and static power (mW); energy per frame (µJ)	State the activity source
Memory	State and table size (KiB); accesses per frame	From the spec and RTL

Concept: Know what your metrics can and cannot say

PESQ and STOI were designed and validated for particular conditions. They predict average listener judgements for normal-hearing listeners on telephone and enhancement tasks. Neither is a perfect proxy for what a hearing-aid user perceives. Use them to compare *configurations of the same algorithm*, which is exactly what RQ1 and RQ2 need, rather than to claim absolute perceptual quality.

11.2 Baselines

Every result needs a reference point:

- **Unprocessed noisy input**: does enhancement help at all?

- **Float64 thesis model**: the reference for RQ1 and RQ2.
- **Float32 thesis model**: a sanity check of how much precision actually matters.
- **Standard Wiener filter** ($\beta = 1$, no tilt): isolates the effect of the parametric control.
- For intent: **the thesis's mpnet + SVM** at macro-F1 0.957 and MAE 0.100.

11.3 Experimental design

- **Development versus test.** Make every choice (word lengths, table sizes, frame configuration, quantisation scales) on the development subset. Run the test set once, at the end, for the reported numbers. This is the same rule as your thesis's test-once discipline.
- **One factor at a time, then jointly.** Single-factor sweeps find each knee. A joint run checks that the errors do not add up beyond the target.
- **Report by condition.** VoiceBank+DEMAND's test set has four input SNRs. Effects often differ between low and high SNR, so report each condition as well as the overall mean.
- **One change per experiment.** Each experiment folder records exactly one hypothesis, one configuration and one commit.

11.4 Statistics: is the difference real?

Most comparisons in this project are *paired*: the same 824 utterances, processed two ways. Use that pairing:

1. compute the per-utterance difference d_i, for example $\mathrm{PESQ}_{\mathrm{fixed}} - \mathrm{PESQ}_{\mathrm{float}}$;
2. report the mean $\bar{d}$ with a 95% *bootstrap* confidence interval: resample the 824 differences with replacement many times and take the 2.5th and 97.5th percentiles of the means;
3. for "is there any difference at all?", use a Wilcoxon signed-rank test on the d_i;
4. for RQ1's equivalence question, check that the *whole* confidence interval lies within the tolerance ($|\Delta\mathrm{PESQ}| \leq 0.1$). This is stronger, and more honest, than "not significantly different".

Listing 11.1: Paired bootstrap confidence interval for a mean difference.

```
import numpy as np

def paired_ci(a, b, n_boot=10_000, seed=0):
    d = np.asarray(a) - np.asarray(b)
    rng = np.random.default_rng(seed)
    idx = rng.integers(0, len(d), size=(n_boot, len(d)))
    means = d[idx].mean(axis=1)
    return d.mean(), np.percentile(means, [2.5, 97.5])
```

For the NLP results, the test set has only about 166 sentences. Report how many predictions changed between float and int8, not just the change in F1. A difference of one or two sentences is within noise.

11.5 Reproducibility

Follow the "good enough practices" of scientific computing [74], [75]:

- Every experiment has a configuration file, a fixed seed and a commit hash, stored with its raw results (CSV), not just the plots.

- One command per figure. A `make figures` target, or a script, regenerates every table and plot from raw results.
- Record tool versions: Python packages (lock file), Verilator, cocotb, Vivado, the PYNQ image, LibreLane and the PDK version.
- Tag the repository at every milestone.
- Raw audio stays out of git, but record how to obtain it (the dataset DOI and a script).

11.6 Presenting results

Plan your key tables and figures now, and fill them as you go:

- quality against word length, per signal and joint (RQ1);
- quality against latency, with state memory (RQ2): the Pareto plot;
- resources and area against word length, next to the quality plot (RQ3);
- energy per frame for the architectures you compared (RQ3, H4);
- macro-F1 against model size and precision (RQ4).

Table 11.2: Template for the headline results table (fill in at M5).

Configuration	PESQ	STOI	Latency	State	Energy/frame
Noisy input			—	—	—
Float64 thesis (48 kHz, 2048 FFT)			21.33 ms	23,575 words	—
Fixed point, chosen spec (golden = RTL)					
FPGA (measured)	= above	= above			
ASIC (Track B, estimated)	= above	= above			

11.7 Threats to validity

A strong report names its own weaknesses before a reviewer does. Discuss at least these:

- **Objective metrics are not perception**, least of all for hearing-impaired listeners. The thesis's open calibration problem remains open.
- **The data is not hearing-aid data.** VoiceBank+DEMAND is studio speech with added noise, not a hearing-aid microphone in a real room.
- **The intent data is typed and LLM-generated**, as the thesis noted, and the TTS data in Route C is not real speech.
- **FPGA power is not ASIC power**, and **130 nm open-PDK results are not modern hearing-aid silicon**.
- **Estimates are not measurements.** Every number has a label.

11.8 Ethics and data

If you record anyone's voice, for latency tests or Route C, get informed consent, say what the recordings are for, store them securely, and delete them when the project ends. Recordings of identifiable people are personal data under GDPR. Check your university's guidance before recording anyone. Keep public datasets under their licences, and cite them.

Your evaluation is sound when

- ☐ Every number in the report has a definition, a method, a dataset split and a commit.
- ☐ Every comparison has a baseline and a confidence interval.
- ☐ The test set was used once, at the end.
- ☐ Every figure regenerates from raw results with one command.
- ☐ The threats to validity are written down.

Chapter 12

Writing It Up, Publishing and Presenting

[Weeks 21–24] Milestone M5

Results that nobody can read do not exist. This chapter covers how to write the project up as a paper-style report, where it could go, how to release the repository, and how to turn it into CV lines and interview stories. Do not leave writing to the end: each milestone already produces material for one section.

12.1 Write as you go

Table 12.1: Which milestone feeds which section of the report.

Milestone	Draft this section right after reaching it
M0	Experimental setup: data, metrics, float baseline
M1	Algorithm–hardware co-design: frame study, precision study, the specification
M2	Architecture and verification: block diagram, frame schedule, bit-exactness
M3	Implementation and results on the FPGA: latency, resources, power, demo
M4	Track results: ML accelerator, or ASIC implementation
M5	Introduction, discussion, conclusion, abstract — written *last*

12.2 Structure of the report

A hardware-for-signal-processing paper usually follows this shape:

1. **Introduction.** The problem (intent-controlled hearing-aid DSP must fit a tiny power budget), the gap (the thesis ran on a laptop), the contributions in three or four bullet points, and the results in one sentence each.
2. **Background.** IMCRA, the parametric Wiener filter and the thesis system, all brief and cited.
3. **Workload characterisation.** The Chapter 3 analysis in condensed form. Reviewers love a cost model.
4. **Co-design.** Frame configuration, precision and approximations, and the gain-LUT control interface, each with the experiment that justified it.
5. **Architecture and verification.**
6. **Results.** FPGA, plus ML or ASIC, against baselines, with confidence intervals.
7. **Discussion.** What flipped under hardware constraints, threats to validity (Chapter 11), and future work, including the perceptual calibration study.

8. **Conclusion.**

Figures you must have:

- the two-plane architecture;
- the block diagram;
- the frame schedule;
- quality against bits;
- quality against latency (Pareto);
- area or resources against bits;
- the headline results table;
- a photo of the demo setup.

Writing advice that works

Write one message per paragraph, and put it in the first sentence. Put the old, known information at the start of a sentence and the new information at the end [76]. Write the outline first, then the figures, then the text [77]. For computer-science style writing in general, see Zobel [78]. Before you start, answer the Heilmeier questions: what are you trying to do, how is it done today, what is new, and who cares [79]. They make an excellent first page.

12.3 Where it could go

A well-executed project of this kind can be submitted to a student-friendly venue. Check each venue's current call for papers, deadlines and page limits:

- **IEEE NorCAS** (Nordic Circuits and Systems Conference): a regional conference with a natural fit for a Danish circuits project.
- **IEEE SiPS** (Workshop on Signal Processing Systems): exactly the signal-processing-hardware intersection.
- **IEEE ISCAS**, **EUSIPCO** or **ICASSP**: larger, more competitive venues. Check whether there are student or late-breaking tracks.
- **arXiv**: post a preprint once your supervisor agrees.

Within your university, the project is also an excellent basis for an MSc thesis proposal, or can be formalised as a project course. Ask your study administration which formats exist for your programme [80].

12.4 Release the repository

- A README with the results table at the top, one-command reproduction, and the demo video linked.
- A tagged release (`v1.0`). Archive it for a DOI if you want it to be citable, and add a `CITATION.cff` file [81].
- Licence, data instructions (how to obtain VoiceBank+DEMAND), and pinned tool versions.

12.5 CV lines and interview stories

Write CV lines with *your measured numbers*. Never fill them in before you have measured them. Templates:

- "Designed and verified a real-time speech-enhancement datapath (IMCRA + parametric Wiener) in SystemVerilog, bit-exact with a Python golden model across [N] utterances; [X] ms algorithmic latency, [Y] LUTs / [Z] BRAMs on a Zynq-7020."
- "Reduced per-bin state memory by [k]× through log-domain minimum tracking and frame reconfiguration at a PESQ cost of [d]."
- "Built an int8 matrix–vector accelerator for intent classification, preserving macro-F1 within [d] of float." (Track A)
- "Took a DSP block through an open-source RTL-to-GDS flow (LibreLane, [PDK]); [A] mm^2, [E] µJ/frame from gate-level activity." (Track B)

Prepare five stories of about two minutes each. Each one is a situation, what you did, and what you measured:

1. **A decision that flipped.** mpnet versus MiniLM, or zero padding: right for a laptop, wrong for a chip, and how you showed it (Chapter 3).
2. **The interface trick.** How language control became a table write and two registers.
3. **Bit-exact verification.** Why exact equality, and how the golden model made debugging systematic.
4. **A hard bug.** The first divergence, how you localised it, and the regression test you added.
5. **A trade-off with numbers.** Latency against quality against memory, from your Pareto plot.

12.6 Talking to industry

Hearing-aid manufacturers in Denmark, such as Demant (Oticon), GN and WS Audiology, build exactly the kind of low-power DSP and ML systems this project studies. Oticon also has a dedicated research centre, Eriksholm [82]. Your thesis collaboration with Oticon is a natural first contact: share the demo video and the results table with your thesis contacts, and ask for twenty minutes of feedback. Engineers respond well to specific, measured work and a clear question.

Milestone M5 is reached when

- ☐ The report is complete, and your supervisor has given feedback that you have addressed.
- ☐ The repository is released as `v1.0`, and reproduces the headline results from one command.
- ☐ The demo video and results table are linked in the README.
- ☐ The CV lines contain only measured numbers.
- ☐ You have rehearsed the five interview stories out loud.

Chapter 13

Timeline, Milestones and Risk Management

A research project rarely fails for technical reasons. It fails because a few weeks disappear into a problem nobody noticed growing, or because the scope never stopped expanding. This chapter is the defence: a schedule you can track, exit criteria that tell you unambiguously when a phase is done, a weekly routine, and a list of the risks most likely to hurt you, each with a plan.

13.1 The schedule

Figure 13.1 expands the roadmap from Chapter 1 into the main steps of each phase. It assumes about 12–15 hours per week. Overlaps are deliberate. You start writing RTL for the simplest blocks while the last Phase 1 sweeps run, and verification runs alongside RTL design rather than after it.

13.2 Milestones and exit criteria

A milestone is reached when its exit criteria are met and there is *evidence* for each one, not when you feel nearly done (Table 13.1). At every milestone, tag the repository, for example `git tag m1-spec-v1`, and write a one-page milestone report using the template in appendix E.

13.3 Weekly rhythm

- **Monday (15 minutes).** Choose this week's two or three goals. Each must finish with something checkable, such as "the `recip_unit` test passes", not "work on the reciprocal".
- **Every working session (5 minutes).** Start by rereading your last notebook entry, and end by writing a new one: what you did, what happened, and what you will do next. This is the habit that saves projects.
- **Friday (30 minutes).** Review the week against the plan. Update the Gantt chart and the risk register. Write the weekly status note (appendix E).
- **Every two weeks.** Meet your supervisor with the status notes. Bring a decision you need help with, not just a progress report.

Protect deep-work blocks

RTL debugging and verification need uninterrupted blocks of at least two to three hours. Twelve hours in four solid blocks gets far more done than twelve hours in twenty fragments.

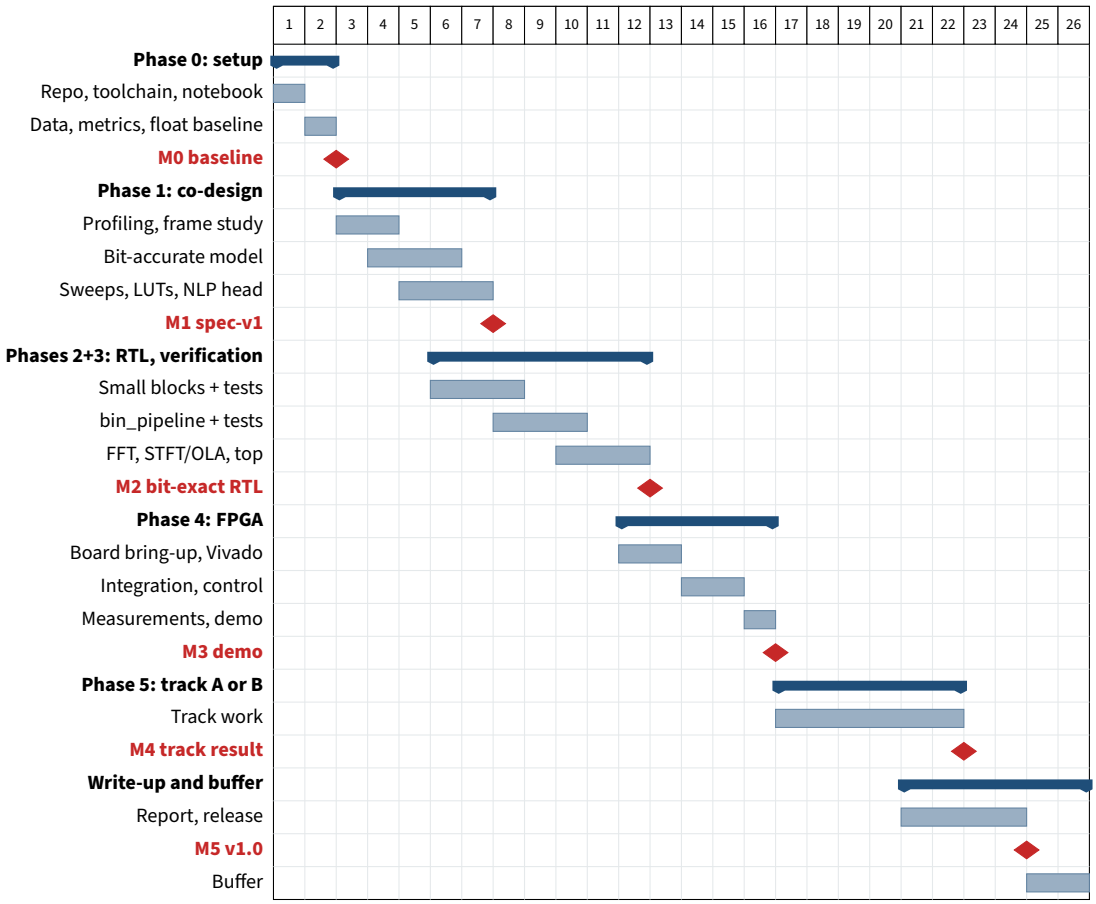


Figure 13.1: Week-by-week schedule. Groups (dark bars) are phases, light bars are work packages, red diamonds are milestones at the end of the week.

Table 13.1: Milestones, exit criteria and evidence.

	Week	Exit criteria	Evidence
M0	2	Toolchain smoke tests pass; VoiceBank+DEMAND test set downloaded; metrics harness unit-tested; float baseline measured	CI green; `experiments/000-baseline/results.csv`; baseline table in notebook
M1	7	Bit-accurate model matches float within the RQ1 target at the chosen formats; frame configuration chosen; LUT sizes chosen; NLP head quantised; golden vectors exported	`spec-v1` tag; spec document; sweep plots; golden vector files
M2	12	All RTL blocks lint-clean; `ise_top` bit-exact with the golden model on the test vectors and at least one full utterance; directed corner cases pass	cocotb regression in CI; coverage summary; Yosys resource log
M3	16	Real-time audio through the FPGA; typed complaints change parameters through registers; latency, resources and power measured	Demo video; Vivado timing and utilisation reports; latency measurement
M4	22	Track A: int8 head engine verified and integrated, plus encoder or audio-intent study. Track B: block through RTL-to-GDS with clean DRC/LVS and power from real activity	Results tables; reports; layout screenshot
M5	24	Report or paper draft complete; repository released as v1.0 with a reproducible README	PDF; release tag; DOI if archived

13.4 Fitting the project to your credit load

Under the ECTS system, one credit usually corresponds to roughly 25–30 hours of work. Check the norms and rules for your course type at your university [80], [83]. The full plan is about 26 weeks at 12–15 hours, roughly 310–390 hours. If you have less time, cut whole phases rather than doing every phase badly (Table 13.2).

Table 13.2: Three scopes. The hours are rough guides, including writing.

Scope	Hours	Contents	Research questions
Compact	≈140	Phase 0; Phase 1 (frame study, bit-accurate model, NLP head quantisation); one RTL block (`recip_unit` or `head_engine`) verified in simulation	RQ1, RQ2, first part of RQ4
Core	≈250	Compact, plus the full datapath RTL bit-exact in simulation (Phases 2–3) and synthesis estimates with Yosys and Vivado out-of-context runs	RQ1, RQ2, RQ3 (estimates), RQ4 (head)
Full	≈350	Core, plus the FPGA prototype (Phase 4) and one track (Phase 5)	All four

13.5 Decision points

Some decisions should be made at fixed points, using evidence, so they do not drift:

End of week 4 — frame configuration
Choose the sample rate, window and padding from the frame study. If no configuration under 10 ms meets the quality target, keep the best under 16 ms and report the trade-off honestly. That is still an answer to RQ2.

End of week 7 — precision (M1)
If some block needs more than 24 bits to meet RQ1, give that block a small custom floating-point or logarithmic format, or relax the target and document it. Do not let Phase 1 run into week 9.

Week 12 — FPGA readiness
If you have no working Vivado host by week 12 (Chapter 4), switch Phase 4 to a *simulation-only demo*: process recorded audio through the RTL and play the result back. Move the saved weeks into Phase 5.

Week 16 — choose the track
Choose Track A or B based on your results so far, your interests and your supervisor's advice. Write the decision and its reasons in the notebook.

13.6 Risk register

Review this table every Friday. A risk becomes an *issue* when its trigger appears, and then you act on its mitigation immediately (Table 13.3). L and I are likelihood and impact (L = low, M = medium, H = high).

Table 13.3: Risk register.

Risk	L	I	Mitigation	Early-warning trigger
Vivado cannot run on your computer (it is not supported on macOS)	H	H	Arrange an x86-64 Linux or Windows machine in week 1: a lab PC, a remote university machine or a cloud VM [35]	No Vivado host by week 8
FFT block harder than expected	M	H	Build it last among datapath blocks; start with a plain complex radix-2 FFT, and optimise only after it is bit-exact	FFT not passing tests by week 11
Phase 1 never ends ("one more sweep")	H	M	Hard stop at week 7; anything unfinished becomes future work	Week 6 with no draft spec
Fixed-point model does not match float	M	H	Compare block by block with probes; quantise one block at a time	Mismatch you cannot localise within one day
RTL and golden model disagree on rounding	H	M	Write rounding and saturation helpers once and use them everywhere; test each block exhaustively or randomly against the model	First block mismatch unexplained after 2 hours
PESQ/STOI library problems (sample-rate limits, install)	M	L	Resample to 16 kHz for PESQ; pin versions; keep SI-SDR as a dependency-free fallback	Install fails or scores look implausible
Board codec or audio IP will not work	M	M	Prove the audio path with the vendor's example first; fall back to streaming audio from the ARM through DMA	Base-overlay audio example fails
Timing closure on the FPGA	L	M	The design needs only a few MHz; add a pipeline stage or constrain a slower clock	Negative slack at 100 MHz
Tool version drift breaks scripts	M	M	Record versions; pin Python packages; use containers or Nix where offered	A tool update changes results
Underestimated effort	H	M	Weekly review; cut scope early using Table 13.2	Two consecutive weeks behind
LibreLane / PDK installation trouble (Track B)	M	M	Use the documented Nix route and the bundled example first; ask early in the community channels	Example design fails after half a day
Hardware failure or loss (board, SD card)	L	M	Keep scripts in git so the project can be rebuilt; image the SD card	—
Data access problems	L	M	Download VoiceBank+DEMAND in week 2 and keep a local copy	Download blocked
Motivation dip in the long middle (RTL debugging)	M	H	Small visible wins weekly; regular supervisor meetings; demo intermediate results to friends	Notebook entries become sparse

13.7 When you get stuck

Getting stuck is normal. Staying stuck for a week is a process failure. Use this protocol:

1. **Timebox.** Give a problem two focused hours. Write down what you tried as you go.
2. **Shrink it.** Build the smallest reproduction you can: one module, one test, one input. Half of all bugs are found while doing this.
3. **Compare with the model.** In this project you almost always have a golden model. Find the *first* signal or block where the two disagree.
4. **Ask.** Ask your supervisor, or the tool's own community: the GitHub issues and discussions of Verilator and cocotb, the PYNQ support forum [52], the Tiny Tapeout community for Track B, or a general FPGA forum. Post the minimal reproduction, the exact tool versions, and what you already tried.
5. **Route around it.** If the problem is not central to a research question, find a workaround, note it as a limitation, and move on.

You are using this chapter well if

- ☐ The Gantt chart and risk register are updated every Friday.
- ☐ Every milestone has a tag, a report and evidence for each exit criterion.
- ☐ Decisions at the four decision points are written down with reasons.
- ☐ No problem has stayed unsolved and unescalated for more than a week.

Appendices

Appendix A

Fact Sheet: Measured Properties of the Thesis System

This appendix collects, on one page, every number about the thesis system that the guide relies on. *Measured* values come from `docs/research_guide/scripts/measure_workload.py` run on the thesis code (step 3.1). *Estimates* are derived by hand from the code, and Chapter 3 shows how. Thesis results are quoted from the thesis repository [1]. If you change the thesis code, rerun the script and update this page first.

Data plane

Quantity	Value	Kind
Sample rate f_s	48 kHz	config
Block = hop H	512 samples	config
Window (periodic $\sqrt{\text{Hann}}$, analysis and synthesis)	1024 samples	config
FFT size (2× zero padding)	2048	config
Frequency bins K	1025	config
Frame rate / frame period	93.75 Hz / 10.667 ms	measured
Algorithmic latency (block buffering + WOLA overlap)	1024 samples = 21.33 ms	measured
Cycles per frame at 100 / 50 / 10 / 1 MHz	1,066,667 / 533,333 / 106,667 / 10,667	measured
Mutable state per bin	23 words	measured
of which inside the two minimum trackers	18 words ($2 \times (8+1)$)	measured
IMCRA S, $\tilde{S}$, λ_d; Wiener $\lvert\hat{S}\rvert^2_{\text{prev}}$, ξ	5 words	measured
Constant per-bin data (tilt frequency ramp)	1 word (ROM or computed)	measured
Total mutable per-bin state	23,575 words	measured
at 16 / 24 / 32 bits	46.0 KiB / 69.1 KiB / 92.1 KiB	measured
Other arrays (input buffer, OLA buffer, window)	3×1024 words + 5-tap kernel	measured
Pre-roll buffer (start-up only)	$5 \times 1025 = 5{,}125$ words	measured
Python float64 speed	≈ 0.2 ms per frame (real-time factor ≈ 0.02; varies by machine)	measured
Per-bin ops per frame: × / ± / ÷ / cmp / transcendental	35 / 26 / 12 / 39 / 2 (+1 for tilt)	estimate
Distinct denominators among the 12 divisions	9	estimate
Ops per frame (FFT + IFFT $\approx 112{,}000$, per-bin $\approx 115{,}000$)	$\approx 230{,}000$	estimate
Ops per second	$\approx 21 \times 10^6$	estimate
Minimum clock: 1 butterfly/cycle + per-bin pipeline (II = 1)	$\approx 12{,}500$ cycles/frame $\Rightarrow \approx 1.2$ MHz	estimate
Minimum clock: 1 butterfly/cycle + folded per-bin unit	$\approx 115{,}000$ cycles/frame $\Rightarrow \approx 11$ MHz	estimate

Algorithm parameters

IMCRA [2]	$\alpha_s = 0.9$, $\alpha_d = 0.85$, bias $B_{\min} = 1.47$ (named `beta` in `imcra.py`), $\gamma_0 = 4.6$, $\gamma_1 = 3$, $\zeta_0 = 1.67$, $U = 8$, $V = 15$, $D = UV = 120$ frames (≈ 1.28 s), 5-tap normalised Hann smoothing across frequency
Wiener [3], [84]	$G = \max\left((\xi/(1+\xi))^\beta, G_{\text{floor}}\right) \cdot 10^{\text{tilt}\cdot f/20} \cdot g_{\text{out}}$, clipped to $[0, 4]$; $\beta = 1$ (range $[0.1, 4]$), $G_{\text{floor}} = 0.05$, decision-directed $\alpha = 0.98$, $\xi_{\min} = -25$ dB, $g_{\text{out}} = 1$ (range $[0.1, 4]$), tilt $= 0$ (range ± 12 dB)
Pre-roll	first 5 frames pass through unprocessed; their per-bin minimum seeds IMCRA

Alternative frame configurations

No zero padding, hop = window/2, state = 23 words × bins (measured).

f_s	window	bins	latency	frame rate	state (words)
48 kHz	1024	513	21.33 ms	93.75 Hz	11,799
48 kHz	512	257	10.67 ms	187.5 Hz	5,911
16 kHz	256	129	16.00 ms	125 Hz	2,967
16 kHz	128	65	8.00 ms	250 Hz	1,495
16 kHz	64	33	4.00 ms	500 Hz	759

When the hop changes from H to H', keep the time constants by using $\alpha' = \alpha^{H'/H}$ and $D' = D\,H/H'$ frames.

Control plane

Quantity	Value	Kind
Complaint dataset	1106 sentences, 6 classes; mean 7.8 words (max 12)	measured
`all-mpnet-base-v2`: parameters, dimension, layers	≈110 M, 768, 12	published
`all-MiniLM-L6-v2`: parameters, dimension, layers	≈22.7 M, 384, 6	published
Encoder compute per sentence (mpnet / MiniLM)	≈2.4 / ≈0.3 GFLOP	estimate
Encoder weights fp32 (mpnet / MiniLM)	≈440 / ≈90 MB	estimate
Encoder weights int8 (mpnet / MiniLM)	≈110 / ≈23 MB	estimate
Selected classifier	`SVC(kernel="linear", probability=True)`, one-vs-one	code
MACs (15 hyperplanes × 768)	11,520 + 15 votes	exact
Logistic regression alternative (6 × 768)	4,608 MACs + argmax	exact
Ridge severity (768)	768 MACs + bias + clip	exact
Thesis results: LinearSVM CV / test macro-F1	0.964 / 0.957	thesis
Thesis results: logistic regression CV macro-F1	0.964	thesis
Thesis results: Ridge α, CV MAE	0.10, 0.1002	thesis

Script output

For reference, this is the output the guide was written against. Your Python timing will differ, but every other line should match exactly.

```
== Frame timing ==
fs=48000 Hz  hop=512  window=1024  n_fft=2048  bins=1025
frame rate   = 93.7500 Hz
frame period = 10.6667 ms
algorithmic latency = 1024 samples = 21.333 ms (one block of buffering + one hop of WOLA overlap)
  cycles per frame @   100 MHz =    1,066,667
  cycles per frame @    50 MHz =      533,333
  cycles per frame @    10 MHz =      106,667
  cycles per frame @     1 MHz =       10,667

mutable words per bin      = 23  (of which 18 in the minimum trackers)
constant words per bin     = 1
mutable per-bin state      = 23,575 words
  @ 16 bit = 46.0 KiB
  @ 24 bit = 69.1 KiB
  @ 32 bit = 92.1 KiB
other array words          = 3,077 (input buffer, OLA, window, kernel)
```

```
pre-roll buffer at startup = 5 x 1025 = 5,125 words (transient)

== Python float64 reference speed ==
0.204 ms per frame (budget 10.667 ms), real-time factor 0.019
```

Appendix B

Register Map of the Control Interface

The control plane talks to `ise_top` through an AXI4-Lite slave (`regs_axil`) with 32-bit registers. This map is a starting point: adjust the formats to your `spec-v1`, but keep the *semantics*. In particular, no parameter the datapath reads may change in the middle of a frame.

Commit semantics

On a write of 1 to **LUT_COMMIT**, the hardware sets `STATUS.COMMIT_PENDING`. At the next frame boundary it atomically:

1. swaps the active and shadow gain-table banks;
2. copies the shadow **OUTPUT_GAIN** and **TILT_RATIO** into the active registers the datapath reads;
3. clears `COMMIT_PENDING` and sets `COMMIT_DONE`.

After a swap, the shadow bank holds the *previous* table. The control plane must therefore write a complete table before every commit. It must never assume the shadow bank already contains the current one.

Applying an intent update

1. The control plane classifies the complaint and computes the new parameters: β, floor, g_{out} and tilt.
2. It computes the 256-entry gain table with the same function the golden model uses (`make_gain_lut`, Chapter 5), and the tilt ratio $r = 10^{\text{tilt}/(20(K-1))}$.
3. It waits until `STATUS.COMMIT_PENDING` is 0.
4. It writes **LUT_ADDR** = `0x80000000` (address 0, auto-increment), then 256 writes to **LUT_WDATA**.
5. It writes **OUTPUT_GAIN** and **TILT_RATIO**.
6. It writes 1 to **LUT_COMMIT**, then polls for `STATUS.COMMIT_DONE` and clears it by writing 1.

Listing B.1: A sketch of the Python driver on the PYNQ (check the MMIO API of your PYNQ version).

```
from pynq import MMIO
from golden.gainlut import make_gain_lut   # same code as the golden model

CTRL, STATUS, OUTPUT_GAIN, TILT_RATIO = 0x00, 0x04, 0x10, 0x14
LUT_ADDR, LUT_WDATA, LUT_COMMIT = 0x20, 0x24, 0x28
PENDING, DONE = 1 << 2, 1 << 3

class EnhancerRegs:
    def __init__(self, base_addr):
```

Table B.1: Register map. Access: RW read/write, RO read-only, WO write-only, W1C write 1 to clear. SC = self-clearing.

Offset	Name	Access	Reset	Bits	Description
`0x00`	**CTRL**	RW	`0x0`	0	`ENABLE`: process audio (0 = output silence)
				1	`BYPASS`: pass input to output unprocessed (latency preserved)
				2	`SOFT_RESET` (SC): clear all state, restart pre-roll
				3	`PREROLL_RESTART` (SC): re-seed IMCRA, keep parameters
`0x04`	**STATUS**	RO/W1C	`0x0`	0	`PREROLL_DONE` (RO)
				1	`FRAME_OVERRUN` (W1C): a frame missed its deadline
				2	`COMMIT_PENDING` (RO): a commit waits for a frame boundary
				3	`COMMIT_DONE` (W1C): the last commit took effect
`0x08`	**VERSION**	RO	build	31:24 / 23:16 / 15:0	major / minor / spec revision
`0x0C`	**FRAME_CNT**	RO	`0x0`	31:0	frames processed since reset (wraps)
`0x10`	**OUTPUT_GAIN**	RW	`0x2000`	15:0	shadow g_{out}, UQ3.13 (reset value = 1.0)
`0x14`	**TILT_RATIO**	RW	`0x800000`	23:0	shadow tilt ratio r, UQ1.23 (reset value = 1.0, i.e. no tilt)
`0x18`	**PREROLL_FRAMES**	RW	`0x5`	7:0	frames used to seed IMCRA
`0x20`	**LUT_ADDR**	RW	`0x80000000`	7:0	address in the *shadow* gain-table bank
				31	`AUTO_INC`: increment the address after each **LUT_WDATA** write
`0x24`	**LUT_WDATA**	WO	—	15:0	table entry, UQ3.13, written to the shadow bank at **LUT_ADDR**
`0x28`	**LUT_COMMIT**	WO	—	0	write 1: request a swap at the next frame boundary
`0x2C`	**LUT_BANK**	RO	`0x0`	0	currently active bank
`0x30`	**CYCLES_LAST**	RO	`0x0`	31:0	cycles used by the last frame (debug, tests H4)
`0x34`	**DBG_SEL**	RW	`0x0`	15:0 / 31:16	probe select / bin index
`0x38`	**DBG_DATA**	RO	—	31:0	value of the selected probe for the selected bin in the last frame

```
        self.mmio = MMIO(base_addr, 0x100)

    def apply(self, beta, floor, g_out, tilt_db, n_bins):
        while self.mmio.read(STATUS) & PENDING:
            pass
        self.mmio.write(LUT_ADDR, 0x80000000)              # address 0, auto-increment
        for word in make_gain_lut(beta, floor):
            self.mmio.write(LUT_WDATA, int(word))
        r = 10 ** (tilt_db / (20 * (n_bins - 1)))
        self.mmio.write(OUTPUT_GAIN, round(g_out * (1 << 13)))
        self.mmio.write(TILT_RATIO, round(r * (1 << 23)))
        self.mmio.write(LUT_COMMIT, 1)
        while not self.mmio.read(STATUS) & DONE:
            pass
        self.mmio.write(STATUS, DONE)                      # W1C
```

Why `TILT_RATIO` needs many bits

With $\pm 12\,\mathrm{dB}$ of tilt spread over about 1024 bins, r lies within about $\pm 0.14\%$ of 1, and it is raised to powers up to $K - 1$. Its rounding error compounds over the bins. That is why this map gives it 23 fractional bits. If you chose a smaller frame configuration, or a log-domain tilt, recompute the error at the last bin and resize the register.

Appendix C

Command Cheat Sheet

The commands you will type most often, grouped by phase. Tool options change between versions; when one fails, check the tool's own documentation for the version you installed (and record that version in your notebook).

Python environment and metrics (Phase 0)

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ../Intention_based_speech_enhancment_using_NLP   # thesis code
pip install numpy scipy soundfile pandas matplotlib pytest ruff pesq pystoi pyyaml cocotb
pip freeze > requirements-lock.txt
pytest metrics                       # metric unit tests
ruff check .                         # lint Python
PYTHONPATH=src python docs/research_guide/scripts/measure_workload.py   # (thesis repo)
```

Golden model and sweeps (Phase 1)

```
python -m golden.sweep --config experiments/012-wordlength/config.yaml
python -m golden.vectors --out golden/vectors/      # export golden test vectors
git tag spec-v1 && git push --tags
```

RTL: lint, simulate, synthesize (Phases 2–3)

```
verilator --lint-only -Wall rtl/*.sv --top-module ise_top
make -C tb/recip_unit SIM=verilator          # one cocotb testbench (Makefile flow)
make -C tb/recip_unit SIM=icarus             # same test, second simulator
pytest tb/                                   # if you use cocotb's Python runner
yosys -p "read_verilog -sv rtl/bin_pipeline.sv; synth -top bin_pipeline; stat"
sv2v rtl/*.sv > build/design.v               # if a tool needs plain Verilog
sby -f tb/formal/min_tracker.sby             # formal proof (SymbiYosys)
gtkwave build/dump.fst  # or: surfer build/dump.fst
```

Coverage (Phase 3)

```
# build the simulation with Verilator's --coverage, run the tests, then:
verilator_coverage --annotate build/cov_annotated coverage.dat
```

Vivado in batch mode (Phase 4)

```
vivado -mode batch -source fpga/pynq_z2/build.tcl      # rebuild project + bitstream
```

```
# useful commands inside a Vivado Tcl script or console
report_timing_summary -file reports/timing.rpt
report_utilization    -file reports/utilization.rpt
report_power          -file reports/power.rpt
write_bd_tcl -force fpga/pynq_z2/bd.tcl                # save the block design
```

On the PYNQ board (Phase 4)

```
from pynq import Overlay, MMIO, allocate
ol = Overlay("ise.bit")              # .hwh with the same name next to it
print(ol.ip_dict.keys())             # find your IP and its address
regs = MMIO(ol.ip_dict["regs_axil_0"]["phys_addr"], 0x100)
print(hex(regs.read(0x08)))          # VERSION register: a first sanity check
```

ASIC flow (Track B)

```
nix-shell librelane/shell.nix        # enter the LibreLane environment
librelane --smoke-test
librelane asic/bin_pipeline/config.json            # default PDK
librelane --pdk ihp-sg13g2 asic/bin_pipeline/config.yaml   # another PDK
klayout runs/<run>/final/gds/bin_pipeline.gds     # look at your layout
```

Git habits

```
git switch -c exp/023-log-trackers          # one branch per experiment
git commit -m "Log-domain trackers: 8-bit minima meet RQ1 on dev set"
git tag m2-bit-exact && git push --tags    # one tag per milestone
```

This guide

```
cd docs/research_guide && make              # build research_guide.pdf
tools/check_chapter.sh chapters/05-codesign # check one chapter quickly
```

Appendix D

Glossary

A priori SNR (ξ) Estimated ratio of clean-speech power to noise power in a bin; computed with the decision-directed method in the thesis.

A posteriori SNR (γ) Ratio of the observed (noisy) power to the estimated noise power in a bin.

Activity factor (α) Fraction of clock cycles in which a node switches; scales dynamic power.

ASIC Application-specific integrated circuit: a chip designed for one function.

AXI4-Lite / AXI4-Stream ARM bus protocols: memory-mapped registers / valid-ready streaming.

Bit-accurate model Software model that computes exactly the integers the hardware will hold.

Bit-exact RTL and model produce identical integers for identical inputs.

Block floating point (BFP) A group of values sharing one exponent; common in FFTs.

Block RAM (BRAM) Dedicated SRAM blocks in an FPGA (36 Kbit each on the Zynq-7000).

Butterfly The basic two-input, two-output operation of an FFT.

Clock domain crossing (CDC) Passing signals between circuits driven by unrelated clocks; needs synchronisers or FIFOs.

Clock gating Stopping the clock to idle logic to save dynamic power.

Clock tree synthesis (CTS) Building the buffered network that distributes the clock with low skew.

cocotb Python framework for writing hardware testbenches.

Control plane The part of a system that configures the data plane; here, the intent path.

CORDIC Shift-and-add algorithm for trigonometric, hyperbolic, exponential and logarithmic functions.

Critical path The slowest register-to-register path; it sets the maximum clock frequency.

Data plane The part of a system that processes the data stream; here, the audio path.

Decision-directed estimator A priori SNR estimate mixing the previous frame's clean estimate with the current observation (Ephraim–Malah).

Development set Data used for tuning choices; distinct from the test set.

DMA Direct memory access: hardware that moves data between memory and a peripheral without the CPU.

DRC / LVS Design rule check / layout versus schematic: physical verification steps before tape-out.

DSP slice Hard multiply–accumulate block in an FPGA.

Dynamic power Power from switching: $\alpha C V^2 f$.

Energy per frame Energy consumed to process one audio frame; the right efficiency metric for a fixed-rate workload.

Fixed point Integers interpreted with an implied binary point; see Q-format.

Folding Mapping several operations onto one shared hardware unit over several cycles.

Formal verification Proving properties hold for all inputs, rather than testing selected ones.

FPGA Field-programmable gate array: reconfigurable hardware.

Functional coverage Measure of which intended scenarios a test suite exercised.

Gain floor Minimum Wiener gain (0.05 in the thesis) to avoid musical noise.

GDS(II) File format of the final chip layout.

Golden model / golden vectors Reference model and the test data it produces, against which RTL is checked.

Hardware-in-the-loop (HIL) Running the real hardware on stored test data and comparing with the model.

HLS High-level synthesis: compiling C++ (or similar) into hardware.

Hold / setup slack Timing margin for the earliest / latest data arrival at a register.

I2S Serial audio interface between converters and digital logic.

II (initiation interval) Cycles between successive inputs accepted by a pipelined unit.

IMCRA Improved minima controlled recursive averaging: the thesis's noise PSD estimator.

Int8 quantisation Representing weights and activations as 8-bit integers with a scale (and zero point).

Latency (algorithmic) Delay caused by the algorithm's buffering; one window length for 50% overlap STFT processing.

Leakage Static current flowing through transistors that are off.

LibreLane Open-source RTL-to-GDS flow (successor of OpenLane 2).

Lint Static checks of RTL code for suspicious constructs.

Logarithmic number system Representing numbers by their logarithms; multiplication becomes addition.

LUT (1) Lookup table holding precomputed function values. (2) The basic logic element of an FPGA.

MAC Multiply–accumulate operation or unit.

Macro-F1 F1 score averaged over classes with equal weight.

Minimum statistics / tracker Noise estimation by tracking the minimum of a smoothed power spectrum over a window.

Musical noise Tonal artefacts from isolated spectral bins switching on and off between frames.

Newton–Raphson reciprocal Iteration $y \leftarrow y(2 - dy)$ that doubles the number of correct bits per step.

OpenROAD Open-source physical design tools (floorplan, placement, CTS, routing, STA).

Overlay A PYNQ FPGA design (bitstream plus metadata) loadable from Python.

Pareto front Set of configurations not beaten on all objectives by any other.

PDK Process design kit: everything needed to design for one manufacturing process.

PESQ Perceptual evaluation of speech quality (ITU-T P.862 / P.862.2 wideband).

Ping-pong buffer Two buffers alternating between being filled and being processed.

Pipelining Inserting registers to split a long computation into stages.

PL / PS Programmable logic / processing system of a Zynq SoC.

Q-format (Qm.F) Fixed-point notation: m integer bits (including sign), F fractional bits.

Real-time factor Processing time divided by audio duration; below 1 means faster than real time.

Retiming Moving registers across logic without changing function, to balance paths.

Roofline model Performance bound $\min(\text{peak compute}, \text{bandwidth} \times \text{arithmetic intensity})$.

RTL Register-transfer level: hardware described as registers and the logic between them.

Saturation Clamping a result to the representable range instead of wrapping.

SDF FFT Single-path delay feedback: a pipelined FFT architecture.

SI-SDR Scale-invariant signal-to-distortion ratio.

Sky130 / SG13G2 Open 130 nm PDKs from SkyWater (with Google) and IHP.

SQNR Signal-to-quantisation-noise ratio; about $6.02N + 1.76$ dB for an N-bit full-scale sine.

SRAM Static random-access memory.

STA Static timing analysis: checking all timing paths without simulation.

Standard cell Pre-designed logic gate in a PDK's library.

STOI Short-time objective intelligibility measure.

Systolic array Grid of processing elements passing data to neighbours each cycle.

Testbench Code that drives a design and checks its outputs.

Tiny Tapeout Shared-shuttle programme for small open-source chip designs.

Twiddle factor Complex exponential W_N^k used in FFT butterflies.

Valid/ready handshake Flow-control protocol: a transfer happens when both are high.

Verilator Open-source SystemVerilog simulator and linter.

VoiceBank+DEMAND Standard speech-enhancement benchmark (clean/noisy pairs at 48 kHz).

WNS Worst negative slack: the most-violating timing path (positive means timing met).

WOLA Weighted overlap-add: windowed analysis/synthesis with perfect reconstruction when the window condition holds.

Yosys Open-source synthesis tool.

Appendix E

Templates

Copy these into your repository (`docs/templates/`) and use them as-is. They are deliberately short: a template you actually fill in beats a perfect one you skip.

Lab notebook entry — `docs/notebook/YYYY-MM-DD.md`

```
# 2027-02-14 (week 5, Phase 1)

Goal today:      log-domain trackers -- does 8-bit storage meet RQ1?
Starting commit: 3f2a9c1

## Did
- Implemented fx_imcra.log_tracker (bias-encoded log2, 8 bits)
- Ran experiments/023-log-trackers on dev subset (200 utts)

## Results
- dPESQ = -0.03 [-0.04, -0.02] vs float; dSTOI = -0.002
- Noise PSD median error 0.31 dB

## Problems / surprises
- Edge bins k=0,1 biased high -- check smoothing zero-padding

## Next
- Fix edge bins; rerun; then joint run with 16-bit S, S~, lambda
```

Experiment record — `experiments/NNN-name/notes.md`

```
# Experiment 023: log-domain minimum trackers

Question:    Can the tracker minima be stored as 8-bit log2 values?
Hypothesis:  Yes; |dPESQ| <= 0.05 on the dev set (part of H1)
Config:      config.yaml (this folder)     Commit: 3f2a9c1    Seed: 0
Data:        dev subset, 200 utterances (list in dev_ids.txt)
Command:     python -m golden.sweep --config experiments/023-log-trackers/config.yaml
Results:     results.csv (per utterance), summary.md (means + 95% CIs)
Conclusion:  Supported / not supported, because ...
Next step:   ...
```

Weekly status report (one page, for your supervisor)

```
Week 9 of 26 -- Phase 2 (RTL)        Plan status: on track / 1 week behind / ...

Done this week:     - recip_unit bit-exact (V2), min_tracker bit-exact (V3)
Planned, not done:  - gain_lut commit test (blocked: AXI write ordering bug)
Next week:          - fix AXI bug; start bin_pipeline stages 1-3
Risks (from register): R2 FFT complexity -- unchanged; R7 Vivado host -- resolved
Decision needed:    - LR or SVM head for Track A? (data: Phase 1 table 3)
```

Design review checklist (before a block is called done)

- ☐ Ports, widths and formats match `docs/spec.md`.
- ☐ Lint-clean with `-Wall`; no inferred latches; memories inferred as RAM.
- ☐ One reset style; all registers reset or deliberately left without reset.
- ☐ Rounding and saturation via the shared helpers only.
- ☐ Parameters read by the datapath are shadowed and latched at frame boundaries.
- ☐ A testbench exists, passes on two simulators, and appears in the verification plan.
- ☐ Resource numbers recorded in `docs/resources.md`.

Verification plan row

ID	Feature	Test (file)	Pass criterion	Status
V*n*		`tb/...`		planned / passing / failing

Specification row

Signal	Symbol	Format	Range (measured)	Evidence (experiment)
		Q*m.F*		`experiments/NNN-...`

Milestone report (one page)

```
Milestone M2 -- RTL bit-exact in simulation       Date:        Tag: m2-bit-exact

Exit criteria (from Chapter 13)          Evidence
- [x] all blocks lint-clean               CI run #212
- [x] ise_top bit-exact, 6 utterances     tb/top/test_utterances.py, log
- [ ] coverage gaps explained             2 open items (see verification_plan.md)

What went well / what was hard:
Deviations from plan (and why):
Changes to plan for next phase:
```

Appendix F

Annotated Reading List

Grouped by phase. **[M]** must-read for this project, **[U]** useful when you reach that point, **[D]** deep-dive if you want more. Full details are in the bibliography.

Foundations (Part I)

- **[M]** Harris & Harris, *Digital Design and Computer Architecture* [26] — the clearest introduction to digital logic and HDLs.
- **[M]** Parhi, *VLSI Digital Signal Processing Systems* [14] — pipelining, folding, retiming: the vocabulary of DSP architecture.
- **[U]** Kaeslin, *Top-Down Digital VLSI Design* [43] — architecture to gates, with an engineer's view of trade-offs.
- **[U]** Horowitz, "Computing's energy problem" [15] — why memory access dominates energy.
- **[D]** Weste & Harris [71]; Rabaey et al. [72] — CMOS circuits in depth.

The thesis algorithms

- **[M]** Cohen, IMCRA [2]; Ephraim & Malah [3] — re-read with "what does this cost in hardware?" in mind.
- **[U]** Loizou, *Speech Enhancement* [84]; Martin, minimum statistics [85].
- **[U]** Stone & Moore on tolerable hearing-aid delay [28], [29] — the evidence behind RQ2.
- **[D]** Kates, *Digital Hearing Aids* [86].

Phase 1 — fixed point and co-design

- **[M]** Widrow & Kollár, *Quantization Noise* [7] — when the uniform-noise model holds and when it does not.
- **[U]** Constantinides et al., word-length optimisation [41].
- **[U]** Muller, *Elementary Functions* [8] — tables, polynomials and Newton–Raphson done properly.
- **[U]** Oppenheim & Schafer [13] — FFT algorithms, real-input FFTs.

Phases 2–3 — RTL and verification

- **[M]** Sutherland et al., *SystemVerilog for Design* [17]; Cummings on non-blocking assignments [42].
- **[M]** cocotb documentation [34] and the Verilator guide [46].

- **[U]** Garrido, survey of pipelined FFT architectures [12]; He & Torkelson, R2^{2}SDF [11].
- **[U]** HDLBits [87] for Verilog practice; Project F [88] for FPGA tutorials; the ZipCPU blog [45] for formal verification of real designs.
- **[U]** SymbiYosys documentation [47].
- **[D]** Spear & Tumbush, *SystemVerilog for Verification* [48].

Phase 4 — FPGA

- **[M]** PYNQ documentation [50] and support forum [52].
- **[U]** *The Zynq Book* [51].
- **[U]** Meyer-Baese, *DSP with FPGAs* [44].
- **[U]** Vivado supported operating systems [35].

Track A — ML acceleration

- **[M]** Sze et al., "Efficient processing of deep neural networks" [27] (and the book [61]).
- **[M]** Jacob et al., integer-only inference [24].
- **[U]** MIT 6.5930/1 course materials [62]; Gholami et al., quantisation survey [63].
- **[U]** Kung, systolic arrays [21]; Jouppi et al., TPU [22]; Chen et al., Eyeriss [23]; Williams et al., roofline [25].
- **[U]** hls4ml [18], [55]; FINN [56]; Brevitas [57].
- **[D]** MiniLM [30], DistilBERT [89], I-BERT [53], Q8BERT [54]; keyword spotting [60]; Fluent Speech Commands [58]; MLPerf Tiny [90].

Track B — ASIC

- **[M]** LibreLane documentation [65]; Zero to ASIC course [73].
- **[U]** OpenROAD [66]; OpenRAM [69]; Tiny Tapeout [70].
- **[U]** Chandrakasan et al., low-power CMOS design [16].

Research and writing

- **[M]** Wilson et al., "Good enough practices in scientific computing" [74]; Sandve et al., ten rules for reproducible research [75].
- **[U]** Gopen & Swan, "The science of scientific writing" [76]; Whitesides, "Writing a paper" [77]; Zobel [78].
- **[U]** Peyton Jones, "How to write a great research paper" [91]; Alley, scientific presentations [92].

Bibliography

[1] A. Burrell, *Intention-based speech enhancement for hearing aids using NLP*, BSc thesis, Technical University of Denmark (DTU), in collaboration with Oticon. Code: https://github.com/oliburr10/Intention_based_speech_enhancment_using_NLP, 2026.

[2] I. Cohen, "Noise spectrum estimation in adverse environments: Improved minima controlled recursive averaging," *IEEE Transactions on Speech and Audio Processing*, vol. 11, no. 5, pp. 466–475, 2003. DOI: 10.1109/TSA.2003.811544.

[3] Y. Ephraim and D. Malah, "Speech enhancement using a minimum-mean square error short-time spectral amplitude estimator," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 32, no. 6, pp. 1109–1121, 1984. DOI: 10.1109/TASSP.1984.1164453.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.

[5] K. Song, X. Tan, T. Qin, *et al.*, "MPNet: Masked and permuted pre-training for language understanding," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[6] C. Valentini-Botinhao, X. Wang, S. Takaki, and J. Yamagishi, "Investigating RNN-based speech enhancement methods for noise-robust text-to-speech," in *Proc. 9th ISCA Speech Synthesis Workshop (SSW9)*, 2016, pp. 146–152. DOI: 10.21437/SSW.2016-24.

[7] B. Widrow and I. Kollár, *Quantization Noise: Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge University Press, 2008.

[8] J.-M. Muller, *Elementary Functions: Algorithms and Implementation*, 3rd ed. Birkhäuser, 2016.

[9] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959. DOI: 10.1109/TEC.1959.5222693.

[10] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965. DOI: 10.1090/S0025-5718-1965-0178586-1.

[11] S. He and M. Torkelson, "A new approach to pipeline FFT processor," in *Proc. 10th International Parallel Processing Symposium (IPPS)*, 1996, pp. 766–770. DOI: 10.1109/IPPS.1996.508145.

[12] M. Garrido, "A survey on pipelined FFT hardware architectures," *Journal of Signal Processing Systems*, vol. 94, no. 11, pp. 1345–1364, 2022. DOI: 10.1007/s11265-021-01655-1.

[13] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.

[14] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Wiley, 1999.

[15] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.

[16] A. P. Chandrakasan, S. Sheng, and R. W. Brodersen, "Low-power CMOS digital design," *IEEE Journal of Solid-State Circuits*, vol. 27, no. 4, pp. 473–484, 1992. DOI: 10.1109/4.126534.

[17] S. Sutherland, S. Davidmann, and P. Flake, *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*, 2nd ed. Springer, 2006.

[18] J. Duarte, S. Han, P. Harris, *et al.*, "Fast inference of deep neural networks in FPGAs for particle physics," *Journal of Instrumentation*, vol. 13, no. 07, P07027, 2018. DOI: 10.1088/1748-0221/13/07/P07027.

[19] cocotb contributors, *Cocotb: A coroutine-based cosimulation library for writing VHDL and Verilog testbenches in Python*, https://www.cocotb.org, 2025.

[20] W. Snyder *et al.*, *Verilator: Open-source SystemVerilog simulator and lint system*, https://www.veripool.org/verilator/, 2025.

[21] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982. DOI: 10.1109/MC.1982.1653825.

[22] N. P. Jouppi, C. Young, N. Patil, *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12. DOI: 10.1145/3079856.3080246.

[23] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. DOI: 10.1109/JSSC.2016.2616357.

[24] B. Jacob, S. Kligys, B. Chen, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[25] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785.

[26] S. L. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*. Morgan Kaufmann, 2021.

[27] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017. DOI: 10.1109/JPROC.2017.2761740.

[28] M. A. Stone and B. C. J. Moore, "Tolerable hearing aid delays. I. estimation of limits imposed by the auditory path alone using simulated hearing losses," *Ear and Hearing*, vol. 20, no. 3, pp. 182–192, 1999. DOI: 10.1097/00003446-199906000-00002.

[29] M. A. Stone, B. C. J. Moore, K. Meisenbacher, and R. P. Derleth, "Tolerable hearing aid delays. V. estimation of limits for open canal fittings," *Ear and Hearing*, vol. 29, no. 4, pp. 601–617, 2008. DOI: 10.1097/AUD.0b013e3181734ef2.

[30] W. Wang, F. Wei, L. Dong, *et al.*, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[31] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, MIT Press, 1999, pp. 61–74.

[32] T.-F. Wu, C.-J. Lin, and R. C. Weng, "Probability estimates for multi-class classification by pairwise coupling," *Journal of Machine Learning Research*, vol. 5, pp. 975–1005, 2004.

[33] C. Wolf *et al.*, *Yosys open SYnthesis suite*, https://yosyshq.net/yosys/, 2025.

[34] cocotb contributors. "Cocotb documentation." (2026), [Online]. Available: https://docs.cocotb.org (visited on 10/08/2026).

[35] AMD. "Vivado design suite user guide: Release notes, installation, and licensing (UG973) – supported operating systems." (2026), [Online]. Available: https://docs.amd.com/r/en-US/ug973-vivado-release-notes-install-license/Supported-Operating-Systems (visited on 10/08/2026).

[36] C. Valentini-Botinhao, *Noisy speech database for training speech enhancement algorithms and TTS models*, University of Edinburgh DataShare, 2017. DOI: 10.7488/ds/2117.

[37] ITU-T, *Recommendation P.862: Perceptual evaluation of speech quality (PESQ): An objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs*, International Telecommunication Union, 2001.

[38] ITU-T, *Recommendation P.862.2: Wideband extension to recommendation P.862 for the assessment of wideband telephone networks and speech codecs*, International Telecommunication Union, 2007.

[39] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, "An algorithm for intelligibility prediction of time–frequency weighted noisy speech," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 7, pp. 2125–2136, 2011. DOI: 10.1109/TASL.2011.2114881.

[40] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR – half-baked or well done?" In *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 626–630. DOI: 10.1109/ICASSP.2019.8683855.

[41] G. A. Constantinides, P. Y. K. Cheung, and W. Luk, "Wordlength optimization for linear digital signal processing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 10, pp. 1432–1442, 2003. DOI: 10.1109/TCAD.2003.818119.

[42] C. E. Cummings, "Nonblocking assignments in Verilog synthesis, coding styles that kill!" In *Synopsys Users Group Conference (SNUG)*, 2000.

[43] H. Kaeslin, *Top-Down Digital VLSI Design: From Architectures to Gate-Level Circuits and FPGAs*. Morgan Kaufmann, 2014.

[44] U. Meyer-Baese, *Digital Signal Processing with Field Programmable Gate Arrays*, 4th ed. Springer, 2014.

[45] D. Gisselquist. "The ZipCPU blog." (2026), [Online]. Available: https://zipcpu.com (visited on 10/08/2026).

[46] W. Snyder *et al.* "Verilator user's guide." (2026), [Online]. Available: https://verilator.org/guide/latest/ (visited on 10/08/2026).

[47] YosysHQ. "SymbiYosys (sby) documentation." (2026), [Online]. Available: https://yosyshq.readthedocs.io/projects/sby/ (visited on 10/08/2026).

[48] C. Spear and G. Tumbush, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, 3rd ed. Springer, 2012.

[49] AMD Xilinx, *PYNQ: Python productivity for Zynq*, http://www.pynq.io, 2025.

[50] AMD Xilinx. "PYNQ documentation." (2026), [Online]. Available: https://pynq.readthedocs.io (visited on 10/08/2026).

[51] L. H. Crockett, R. A. Elliot, M. A. Enderwitz, and R. W. Stewart, *The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC*. Strathclyde Academic Media, 2014. [Online]. Available: http://www.zynqbook.com.

[52] PYNQ community. "PYNQ community forum, support category." (2026), [Online]. Available: https://discuss.pynq.io/c/support/ (visited on 10/08/2026).

[53] S. Kim, A. Gholami, Z. Yao, *et al.*, "I-BERT: Integer-only BERT quantization," in *Proc. 38th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 139, 2021, pp. 5506–5518.

[54] O. Zafrir, G. Boudoukh, P. Izsak, and M. Wasserblat, *Q8BERT: Quantized 8bit BERT*, arXiv:1910.06188 (EMC^2 Workshop at NeurIPS 2019), 2019.

[55] Fast Machine Learning Lab. "Hls4ml documentation." (2026), [Online]. Available: https://fastmachinelearning.org/hls4ml/ (visited on 10/08/2026).

[56] Y. Umuroglu, N. J. Fraser, G. Gambardella, *et al.*, "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 65–74. DOI: 10.1145/3020078.3021744.

[57] A. Pappalardo and Xilinx Research Labs, *Brevitas: Neural network quantization in PyTorch*, https://github.com/Xilinx/brevitas, 2021. DOI: 10.5281/zenodo.3333552.

[58] L. Lugosch, M. Ravanelli, P. Ignoto, *et al.*, "Speech model pre-training for end-to-end spoken language understanding," in *Proc. Interspeech*, 2019, pp. 814–818. DOI: 10.21437/Interspeech.2019-2396.

[59] P. Warden, *Speech commands: A dataset for limited-vocabulary speech recognition*, arXiv:1804.03209, 2018.

[60] Y. Zhang, N. Suda, L. Lai, and V. Chandra, *Hello edge: Keyword spotting on microcontrollers*, arXiv:1711.07128, 2017.

[61] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, *Efficient Processing of Deep Neural Networks* (Synthesis Lectures on Computer Architecture). Morgan & Claypool, 2020. DOI: 10.2200/S01004ED1V01Y202004CAC050.

[62] V. Sze and J. S. Emer. "6.5930/1 hardware architecture for deep learning (course website, MIT)." (2026), [Online]. Available: https://csg.csail.mit.edu/6.5930/ (visited on 10/08/2026).

[63] A. Gholami, S. Kim, Z. Dong, *et al.*, *A survey of quantization methods for efficient neural network inference*, arXiv:2103.13630 (book chapter in *Low-Power Computer Vision*), 2021.

[64] LibreLane contributors, *LibreLane: An open-source RTL-to-GDSII flow (successor of OpenLane 2)*, https://github.com/librelane/librelane, 2025.

[65] LibreLane contributors. "LibreLane documentation." (2026), [Online]. Available: https://librelane.readthedocs.io (visited on 10/08/2026).

[66] T. Ajayi, V. A. Chhabria, M. Fogaça, *et al.*, "Toward an open-source digital flow: First learnings from the OpenROAD project," in *Proc. 56th ACM/IEEE Design Automation Conference (DAC)*, 2019. DOI: 10.1145/3316781.3326334.

[67] Google and SkyWater Technology, *SkyWater SKY130 open source process design kit*, https://github.com/google/skywater-pdk, 2020.

[68] IHP GmbH, *IHP open source PDK for SG13G2 130 nm BiCMOS*, https://github.com/IHP-GmbH/IHP-Open-PDK, 2023.

[69] M. R. Guthaus, J. E. Stine, S. Ataei, *et al.*, "OpenRAM: An open-source memory compiler," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016. DOI: 10.1145/2966986.2980098.

[70] M. Venn and Tiny Tapeout contributors, *Tiny tapeout*, https://tinytapeout.com, 2025.

[71] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Addison-Wesley, 2010.

[72] J. M. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Prentice Hall, 2003.

[73] M. Venn. "Zero to ASIC course." (2026), [Online]. Available: https://www.zerotoasiccourse.com (visited on 10/08/2026).

[74] G. Wilson, J. Bryan, K. Cranston, *et al.*, "Good enough practices in scientific computing," *PLoS Computational Biology*, vol. 13, no. 6, e1005510, 2017. DOI: 10.1371/journal.pcbi.1005510.

[75] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research," *PLoS Computational Biology*, vol. 9, no. 10, e1003285, 2013. DOI: 10.1371/journal.pcbi.1003285.

[76] G. D. Gopen and J. A. Swan, "The science of scientific writing," *American Scientist*, vol. 78, no. 6, pp. 550–558, 1990.

[77] G. M. Whitesides, "Whitesides' group: Writing a paper," *Advanced Materials*, vol. 16, no. 15, pp. 1375–1377, 2004. DOI: 10.1002/adma.200400767.

[78] J. Zobel, *Writing for Computer Science*, 3rd ed. Springer, 2014. DOI: 10.1007/978-1-4471-6639-9.

[79] Defense Advanced Research Projects Agency (DARPA). "The Heilmeier catechism." (2026), [Online]. Available: https://www.darpa.mil/about/heilmeier-catechism (visited on 10/08/2026).

[80] Technical University of Denmark. "Project courses (rules for BSc and MSc students)." (2026), [Online]. Available: https://student.dtu.dk/en/rules/teaching/project-courses (visited on 10/08/2026).

[81] GitHub. "About CITATION files." (2026), [Online]. Available: https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-citation-files (visited on 10/08/2026).

[82] Eriksholm Research Centre. "About eriksholm research centre (part of Oticon)." (2026), [Online]. Available: https://www.eriksholm.com/about-us/ (visited on 10/08/2026).

[83] Technical University of Denmark. "Course base – find all DTU's courses (workload per ECTS point)." (2026), [Online]. Available: https://www.dtu.dk/english/education/course-base (visited on 10/08/2026).

[84] P. C. Loizou, *Speech Enhancement: Theory and Practice*, 2nd ed. CRC Press, 2013.

[85] R. Martin, "Noise power spectral density estimation based on optimal smoothing and minimum statistics," *IEEE Transactions on Speech and Audio Processing*, vol. 9, no. 5, pp. 504–512, 2001. DOI: 10.1109/89.928915.

[86] J. M. Kates, *Digital Hearing Aids*. Plural Publishing, 2008.

[87] "HDLBits: Verilog practice problems." (2026), [Online]. Available: https://hdlbits.01xz.net (visited on 10/08/2026).

[88] W. Green. "Project F: FPGA tutorials and verilog library." (2026), [Online]. Available: https://projectf.io (visited on 10/08/2026).

[89] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, *DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter*, arXiv:1910.01108, 2019.

[90] C. Banbury, V. J. Reddi, P. Torelli, *et al.*, *MLPerf tiny benchmark*, NeurIPS Datasets and Benchmarks Track; arXiv:2106.07597, 2021.

[91] S. Peyton Jones, *How to write a great research paper*, Talk, Microsoft Research Cambridge; slides widely mirrored online, 2010.

[92] M. Alley, *The Craft of Scientific Presentations: Critical Steps to Succeed and Critical Errors to Avoid*, 2nd ed. Springer, 2013. DOI: 10.1007/978-1-4419-8279-7.